

Bachelor thesis

Short-Term Trajectory Prediction for Inland Vessels using AIS Data

Dustin Klein (445455)

Supervisors:

1st: Prof. Dr. Holger H. Hoos

2nd: Prof. Dr.-Ing. Heike Vallery

Daily Supervisor (AIM): Wadie Skaf

Daily Supervisor (IRT): Tim Rehbronn

RWTH Aachen University

Department of Computer Science

Chair of AI Methodology (Informatik 14)

<https://www.aim.rwth-aachen.de/>

Abstract

Maritime transport on inland waterways forms a backbone of European logistics and a key application domain for autonomous and cooperative surface vessels, where reliable short-term motion prediction of surrounding traffic is essential for safe navigation. While Automatic Identification System (AIS) data is widely used to train these prediction models, existing data-based approaches often treat AIS trajectories as generic time series without utilising geographic domain knowledge, and they lack comprehensive comparisons against kinematic and filter-based baselines. These gaps prevent practitioners from reliably choosing between motion model families. Furthermore, any potential accuracy gains from encoding simple waterway-type information in data-driven motion models remain unexplored. In this thesis, we address these limitations through a comprehensive study of trajectory prediction for inland vessels based on AIS data. In particular, we examine how kinematic models, stochastic filters, and task-specific, as well as pretrained deep sequence (foundation) models compare in predicting vessel trajectories five minutes ahead under a unified evaluation framework. To this end, our work introduces three main contributions. First, we construct a large-scale dataset with more than four million samples on inland waterways. For this purpose we, develop a pipeline that extracts, cleans, segments, and labels vessel tracks, including domain labels for different waterway types. Second, we implement an AutoML-based hyperparameter optimisation setup to systematically tune models across different model families. Third, we conduct a comprehensive evaluation that compares kinematic models, stochastic filters, and deep sequence models under a unified framework. The experiments demonstrate that learned sequence models clearly outperform kinematic and filter-based models. Specifically, Long Short-Term Memory (LSTM)-based models achieve the lowest errors. They demonstrated an approximate 28% improvement compared with the Constant Velocity (CV) baseline, achieving a lower Root Mean Square Error (RMSE) by 44.37 m. Additionally, we found that foundation models reach similar accuracy to kinematic models without any task-specific training, making them attractive as low-effort baselines, though their probabilistic calibration quality varies. Moreover, training the LSTM with domain-specific splits and incorporating simple one-hot encodings of waterway domains yielded the best overall predictive performance. These results suggest that, for applications prioritising predictive accuracy, future inland multi-target tracking systems should adopt domain-aware deep sequence models as motion models.

Table of Contents

List of Figures	iii
List of Tables	v
List of Abbreviations	vi
1 Introduction	1
2 Related Work	4
2.1 Kinematic and Deep Learning Models	4
2.2 Data Preprocessing	6
2.3 Hyperparameter Optimisation	7
3 Data & Preprocessing	9
3.1 Data Sources	9
3.2 Data Preprocessing	9
3.2.1 Data Extraction and Cleaning	10
3.2.2 Geographic Domain Assignment	11
3.2.3 Resampling	13
3.2.4 Trajectory Segmentation	13
3.2.5 Feature Engineering and Normalisation	15
3.3 Dataset Characteristics	16
4 Methodology	17
4.1 Sequence-to-Sequence Models	17
4.2 Automated Machine Learning	19
4.2.1 The Search Space	19
4.2.2 Search Strategies and Pruning	20
4.3 Kinematic Models	21

4.3.1	Constant Velocity	21
4.3.2	Constant Turn Rate and Velocity	21
4.3.3	Hybrid Constant Velocity/Constant Turn Rate and Velocity	22
4.4	Stochastic Filters	22
4.4.1	Kalman Filter with Constant Velocity	22
4.4.2	Extended Kalman Filter with Constant Turn Rate and Velocity	23
5	Setup of Experiments	25
5.1	Research Questions and Experiments	25
5.2	Evaluation Framework	26
5.2.1	Train-Val-Test Split	26
5.2.2	Metrics	26
5.2.3	Statistical tests	27
5.2.4	Optimisation and Training	27
5.3	Implementation Details	29
5.3.1	Foundation Models	29
5.3.2	LSTM	30
5.4	Reproducibility and Code Availability	31
6	Results and Discussion	32
7	Conclusions	45
	References	51
A	Supplementary Figures	55
B	Supplementary Tables	67
C	Methodological Details	69
C.1	Mathematical Foundations of Sequence Models	69
C.2	Bayesian Optimisation and the Tree-Parzen-Estimator	72
C.3	Mathematical Details of the Search Space	73
C.4	CTRV-EKF Jacobian Derivation	74
D	Codebase Overview	76

List of Figures

3.1	OpenStreetMap Labels	12
3.2	Domain-assigned AIS Data	12
3.3	Preprocessed Dataset Kinematic Characteristics	16
3.4	Polar Histogram of Vessel COG after Preprocessing	16
5.1	Exp. I LSTM Intermediate Value plot	28
5.2	Proposed LSTM Architecture	31
6.1	Exp. I Friedman & Nemenyi CD-Plot	34
6.2	Exp. I ADE over Time	35
6.3	Exp. I RMSE Violin Plots	35
6.4	Exp. I LSTM Parallel Coordinate Plot	36
6.5	Exp. I RMSE Monthly Trends	36
6.6	Exp. III RMSE Violin Plot	39
6.7	Exp. III Friedman & Nemenyi CD-Plot	39
A.1	UTM coordinate overview	55
A.2	Preprocessing visualisation (1/2)	56
A.2	Preprocessing visualisation (2/2)	57
A.3	Strata Elbow-Plot	58
A.4	CV and CTRV QQ-Plot	59
A.5	Hyperparameter Importances for the LSTM	60
A.6	Forecasting examples for Channel (Kiel)	61
A.7	Forecasting examples for Harbour (Kiel)	62
A.8	Forecasting examples for Lock (Kiel)	63
A.9	Forecasting examples for River (Wedel)	64
A.10	Exp. II Friedman & Nemenyi CD-Plots (1/2)	65
A.10	Exp. II Friedman & Nemenyi CD-Plots (2/2)	66

C.1	RNN Architecture	71
C.2	LSTM Architecture	71
C.3	Bayesian Optimisation	72
C.4	Tree-structured Parzen Estimator	73
D.1	Code Overview: Preprocessing Pipeline	77
D.2	Code Overview: Model Layer	78
D.3	Code Overview: Evaluation Workflow	79

List of Tables

3.1	Overview of AIS Data Fields	10
3.2	Geographic Bounding Boxes	11
3.3	Geometric Buffers	11
3.4	AIS Reporting Intervals	13
3.5	Preprocessed Dataset Domain Distribution	15
5.1	LSTM Hyperparameter Search Space	29
6.1	Exp. I Optimised Hyperparameter Configurations	33
6.2	Exp. I Test Set Error Metrics	34
6.3	Exp. I Probabilistic Metrics for Foundation Models	34
6.4	Exp. II Optimised Hyperparameter Configurations	38
6.5	Exp. II Error Metrics	38
6.6	Exp. III Error Metrics	40
B.1	Configuration Parameters for Optuna HPO	68
B.2	Model-specific Hyperparameters	68

List of Abbreviations

ADE	Average Displacement Error
AIS	Automatic Identification System
ARIMA	AutoRegressive Integrated Moving Average
AutoML	Automated Machine Learning
Bi-LSTM	Bidirectional LSTM
BO	Bayesian Optimisation
CASH	Combined Algorithm Selection and Hyperparameter Optimisation
CD	Critical Difference
COG	Course Over Ground
CTRV	Constant Turn Rate and Velocity
CV	Constant Velocity
EKF	Extended Kalman Filter
EI	Expected Improvement
fANOVA	functional Analysis of Variance
FDE	Final Displacement Error
GNSS	Global Navigation Satellite System
HPO	Hyperparameter Optimisation
IMM	Interacting Multiple Model
KF	Kalman Filter
LLM	Large Language Model
LSTM	Long Short-Term Memory
m-LSTM	Matrix LSTM
MoE	Mixture of Experts
MLP	Multi-Layer Perceptron
MMSI	Maritime Mobile Service Identity
MTT	Multi-Target Tracking

NAS	Neural Architecture Search
OSM	OpenStreetMap
PCHIP	Piecewise Cubic Hermite Interpolating Polynomial
RNN	Recurrent Neural Network
ROT	Rate of Turn
RMSE	Root Mean Square Error
s-LSTM	Scalar LSTM
Seq2Seq	Sequence-to-Sequence
SOG	Speed Over Ground
SOLAS	International Convention for the Safety of Life at Sea
TPE	Tree-structured Parzen Estimator
UTM	Universal Transverse Mercator coordinate system
x-LSTM	Extended LSTM

Chapter 1

Introduction

Maritime transport accounts for more than two-thirds of freight transport performance in the EU (Eurostat, 2024). This makes waterway infrastructure a critical backbone of European logistics. In Germany, inland waterway transport accounts for roughly 39% of total German waterborne freight by tonnage, with the remainder carried by maritime shipping (Statistisches Bundesamt (Destatis), 2026). At the same time, inland waterways are themselves vulnerable to sudden and severe disruption: vulnerability to critical water levels on the Rhine has doubled since 2018, with projected annual losses in container throughput of 7–20% by 2050 (Bedoya-Maya et al., 2024). The significance of inland waterway transport and the requirement for safe, reliable operation are therefore likely to grow rather than fade, independent of short-term fluctuations in traffic volume.

Beyond economic pressure, human factors are a major contributor to maritime accidents (Bundesstelle für Seeunfalluntersuchung, 2025; de Vos et al., 2021), motivating the development of systems that reduce reliance on manual operation. Autonomous and cooperative surface vessels are widely discussed as a promising way to mitigate human-error-driven risks and reduce the number of crew members exposed to maritime accidents (de Vos et al., 2021). The research in this thesis is driven by the practical requirements of the envisioned cooperative traffic systems. We specifically target application scenarios like those outlined by the GALILEOnautic 2+ project at the Institute of Automatic Control (IRT) at RWTH Aachen University (Eisenblätter et al., 2025). The practical realisation of such networked, cooperative traffic systems for autonomous inland vessels relies heavily on safe manoeuvring in safety-critical environments like ports and narrow waterways. For autonomous surface vessels operating in close proximity to surrounding traffic, reliable perception and vessel motion prediction is a fundamental requirement. A key technical component enabling this is Multi-Target Tracking (MTT), the continuous association of sensor detections over time to maintain consistent trajectory estimates for all surrounding vessels (Eisenblätter et al., 2025). The performance of MTT is fundamentally dependent on the underlying motion model, which provides the *prediction* step in the tracking cycle. This model is responsible for propagating the vessel state forward in time between sensor updates. As a consequence, the ability of a tracking system to maintain lock on a target in dense traffic is highly sensitive to the accuracy of this prediction. This motivates the exploration of data-driven approaches to capture the complex dynamics that traditional models often fail to describe.

While onboard sensors such as radar, LiDAR, and cameras can provide local observations for downstream perception tasks, the Automatic Identification System (AIS) offers large-scale motion data with time-stamped position, velocity, and vessel identity across entire waterway networks (NOAA Marine Cadastre, 2025). Although raw AIS data is subject to reliability issues, such as transmission gaps or sensor noise (Zhao et al., 2018), its widespread availability offers unprecedented spatial coverage. Consequently, the AIS serves as a primary source of real vessel trajectories for the data-driven learning of motion behaviour. Kinematic motion models, such as Constant Velocity (CV) or Constant Turn Rate and Velocity (CTRV), have long served as the prediction component within tracking filters. However, they rely on hand-crafted dynamics and cannot adapt

beyond their underlying assumptions, making them unable to accurately capture sudden course changes or context-dependent vessel behaviour.

Deep learning-based motion models trained on AIS data have the potential to capture more realistic dynamics and improve prediction quality in constrained, high-traffic environments (Donandt et al., 2022). Reliable prediction in these safety-critical environments is a key challenge for the motion estimation systems that autonomous surface vessels depend on (Eisenblätter et al., 2025), motivating deeper investigation into these models. However, transitioning from physically grounded kinematic models to deep learning architectures introduces a methodological challenge: the dependence of model performance on a high-dimensional design space. Achieving high performance in machine learning requires navigating complex design choices, including model architecture and hyperparameter configurations (Baratchi et al., 2024). To ensure a fair comparison between different model types and architectures, all models must be evaluated at their full potential. To address this, we explore Automated Machine Learning (AutoML) to systematically manage Hyperparameter Optimisation (HPO), and ensure that any observed performance differences reflect the inherent ability of the models rather than arbitrary parameter choices.

The necessity for such a rigorous evaluation framework stems directly from the core limitations of existing tracking systems. Classical kinematic models such as Constant Velocity (CV) and Constant Turn Rate and Velocity (CTRV), often implemented in stochastic filtering frameworks, offer simple and efficient baselines. However, their simplified dynamics make it difficult to represent the nonlinear manoeuvres that often occur in populated inland waterways. Recent work has shown that deep learning models trained on AIS trajectories can capture complex motion patterns more accurately than simple kinematic baselines (Donandt et al., 2022; Zhang et al., 2022). Despite these advances, only a few studies focus specifically on inland waterways, so coverage of inland vessel motion modelling remains limited. Several studies explicitly mention Hyperparameter Optimisation (HPO) as open work and do not evaluate each architecture at its full potential.

While individual works do compare specific deep learning architectures to selected kinematic baselines, to the best of our knowledge, there are no studies that jointly evaluate kinematic models, stochastic filters and both task-specific and pretrained deep sequence models on inland AIS data within a single, shared preprocessing and optimisation framework. In particular, foundation models have, to our knowledge, not yet been applied to inland vessel trajectory prediction. This lack of a unified evaluation setup makes it difficult to draw controlled conclusions about the relative performance of different motion-model families on inland AIS data.

At the same time, existing approaches treat AIS trajectories as generic time series and make no use of domain knowledge about the waterway type, even though vessel dynamics could differ substantially across these domains. It has not yet been systematically investigated how this domain information can be incorporated into motion models, and to what extent domain-aware modelling improves predictive performance on inland AIS data.

We address these gaps through a unified experimental framework combining systematic data curation, AutoML based evaluation, and a comprehensive model benchmark. The main contributions of this thesis are as follows:

1. **Curating a domain-annotated inland AIS dataset:** Developing an automated AIS preprocessing pipeline that consolidates standard cleaning and resampling steps and extends them with semantic domain labels based on OpenStreetMap (OSM), handles domain-aware data preparation, and produces a reproducible large-scale dataset of over four million samples.
2. **Implementing a systematic AutoML setup for motion models:** Developing a HPO framework for kinematic models, stochastic filters, and deep learning models to ensure a fair and reproducible comparison of trajectory prediction methods for inland vessel tracking.
3. **Evaluating motion model families and domain-aware modelling:** Conducting a comprehensive empirical comparison of ten distinct motion models, spanning kinematic, stochastic, and deep learning approaches, across more than four million inland vessel samples. This uni-

fied evaluation assesses predictive capabilities and quantifies the performance gains achieved by integrating geographic domain labels into the preprocessing and training phases.

The remainder of this thesis is structured as follows: In Chapter 2, we review related work on kinematic motion models, stochastic filters, deep sequence models, AIS data preprocessing pipelines, and automated hyperparameter optimisation. In Chapter 3, we outline the preprocessing pipeline we developed to extract, clean, segment, and annotate the trajectories with geographic domain labels, followed by analysis of the resulting dataset characteristics. In Chapter 4, we present the methodology, explaining the theoretical backgrounds of Sequence-to-Sequence (Seq2Seq) models and AutoML, alongside the mathematical formalisations of the kinematic motion models and stochastic filters we implemented. In Chapter 5, we define the research questions and experiments and explain the evaluation framework, which includes data splits, evaluation metrics, and the exact model configurations, as well as optimisation and model training procedures. In Chapter 6, we present and discuss the experimental results, addressing each research question through a comparative evaluation of the models and configurations, along with an analysis of how geographic domain information impacts predictive performance. In Chapter 7, we conclude the thesis by highlighting the performance differences between models, followed by a discussion of the limitations of the study along with potential paths for future research. The appendices provide Supplementary Figures (Appendix A), Supplementary Tables (Appendix B), Methodological Details (Appendix C) and an overview of the implemented codebase (Appendix D)

Chapter 2

Related Work

Multi-target tracking for autonomous inland vessels requires predicting the future state of each vessel between sensor updates. The accuracy of this prediction directly affects the quality of state estimates of the tracker and reduces uncertainty that accumulates over the prediction horizon. Classical kinematic models remain common; however, their simplifying assumptions limit their ability to capture complex manoeuvres. Data-driven methods trained on Automatic Identification System (AIS) data have become a promising alternative.

In this section, we review the state of the art in vessel trajectory prediction to establish the theoretical foundation for our comparative evaluation and to identify the key gaps we address in this thesis. First, we survey existing motion models in Section 2.1, ranging from kinematic models and stochastic filters to task-specific deep learning architectures and recently published foundation models. Next, we describe standard AIS data preprocessing pipelines in Section 2.2. Finally, we discuss the necessity of systematic hyperparameter optimisation to enable a fair comparison between models in Section 2.3.

2.1 Kinematic and Deep Learning Models

Kinematic Models and Kalman Filters. Classical trajectory prediction in the maritime industry frequently uses kinematic models such as Constant Velocity (CV) or Constant Turn Rate and Velocity (CTRV) (Li & Jilkov, 2003). Kinematic models predict future states by projecting current geometric motion variables (e.g., speed and heading) forward in time, without explicitly modelling the physical forces that generate this motion (Li & Jilkov, 2003). In contrast, dynamic models explicitly model the forces and torques acting on a system, such as engine thrust, hydrodynamic drag, and wind. For vessel tracking, dynamic models are not practical because the forces acting on other vessels are not readily available. Given this limitation, model-based filters default to kinematic approaches. Kalman Filters (KFs), for example, are designed to minimise the mean square error of the state estimate and have been successfully applied to position estimation and trajectory prediction of linear systems (Jwo & Cho, 2010). They are computationally lightweight and well suited for short-term predictions under linear conditions. When it comes to vessel dynamics, the process noise covariance Q of a KF can be tuned to allow for deviations from the assumed linear motion. However, the underlying prediction model remains linear and therefore, their useful prediction horizons remain limited (Donandt et al., 2022). The Extended Kalman Filter (EKF) extends the KF framework to nonlinear state-space models by linearising the state transition and measurement functions using first-order Taylor expansion, yet the linearisation still introduces approximation errors under highly nonlinear motion regimes (Jwo & Cho, 2010). Both filter types are inherently model-based and require prior knowledge of the system dynamics in the form of predefined state transition equations. These two filters cannot accurately capture vessel behaviour when the under-

lying dynamics depend on unobserved spatial factors (Yang et al., 2022). In the context of inland shipping, such factors include narrow canal sections or lock approaches. Similarly, these filters struggle when a manoeuvre is inherently more complex than the underlying model assumptions allow. For example, an S-curve cannot be predicted with the singular circular arc generated by the **CTRV** model. Furthermore, these filters follow the Markov assumption, compressing all past information into the current state estimate, and therefore cannot anticipate impending manoeuvres based on historical or contextual patterns.

Deep Learning Models. With increasing **AIS** data availability, data-driven models, particularly deep neural networks, have gained popularity as alternatives or complements to model-based approaches. They are capable of handling complex navigation situations and longer prediction horizons. A comprehensive review by Zhang et al. (2022) revealed that Long Short-Term Memory (**LSTM**) networks and their variants dominate the field of trajectory prediction. Unlike standard Recurrent Neural Networks (**RNNs**), which suffer from the vanishing gradient problem when handling long time series data (Bengio et al., 1994), **LSTMs** more effectively capture long-term temporal dependencies. As a consequence, Sequence-to-Sequence (**Seq2Seq**) architectures based on **LSTMs** have become the established baseline for modern vessel trajectory modelling (Donandt et al., 2022; Zhang et al., 2022). Another key development was the Bidirectional **LSTM** (**Bi-LSTM**), which processes sequences in both forward and backward directions to better capture the full context of a trajectory segment. Yang et al. (2022) showed that **Bi-LSTMs** can significantly outperform traditional forecasting models such as AutoRegressive Integrated Moving Average (**ARIMA**) and Multi-Layer Perceptrons (**MLPs**), alongside **LSTMs** in complex maritime scenarios. The most recent shift in trajectory prediction enhances these recurrent architectures by integrating attention mechanisms (Zhang et al., 2022). Sekhon and Fleming (2020) proposed a framework combining **LSTMs** with spatial and temporal attention layers, which allow the model to predict the trajectories of multiple interacting vessels by selectively attending to both neighbouring vessels and relevant past timestamps. The spatial attention mechanism utilises concurrent **AIS** observations from neighbouring vessels as additional input. This approach directly applies to the scenarios relevant to autonomous surface vessels.

Despite these advances, existing deep learning studies remain largely limited to open-ocean or coastal settings. Yang et al. (2022) evaluated their **Bi-LSTM** model exclusively on large vessels in open coastal waters around Taiwan, where trajectories are comparatively straight and predictable. Their evaluation is limited to trajectories of only ten vessels. Sekhon and Fleming (2020) evaluated on only 8,676 samples from a single US harbour and did not motivate their hyperparameter choices. Additionally, inland-specific studies are rare, and there is little consensus on appropriate temporal window sizes for trajectory segmentation. For instance, You et al. (2020) applied a 10-minute context to predict 5 minutes ahead for vessels on the Yangtze River, relying on a data collection period of ten days. Donandt et al. (2022) noted that context lengths in inland literature typically range from 4 to 10 minutes. In their own work, they used 5 to 10 minutes of **AIS** history to predict 15 minutes ahead, and utilised unusually long 30-minute prediction horizons during training to improve generalisation. However, their evaluation scope was restricted to a single 90-kilometre section of the Danube River, where they used between 250k and 280k trajectory sequences. Subsequent work by Legel et al. (2025) extended the attention mechanism presented by Sekhon and Fleming (2020) to an interaction-aware multi-vessel **LSTM** explicitly adapted for the inland domain; their work is, however, limited to a 16km stretch of the Rhine.

In summary, existing deep learning approaches either rely on limited spatial coverage, small datasets, or the inputs of multiple vessels. They rarely include systematic Hyperparameter Optimisation (**HPO**).

Foundation Models. With the rising popularity of Large Language Models (**LLMs**), foundation models have been explored in time series forecasting (Wang et al., 2026). Time series foundation models are large forecasting models that are pretrained on massive datasets of, typically, both real and synthetic data, with tens of billions of points from different domains (Wang et al., 2026). They

are designed to provide universal, often zero-shot, probabilistic forecasts, and can be adapted to new tasks with little or no fine-tuning (Ansari et al., 2025). To the best of our knowledge, time series forecasters have not yet been applied to vessel trajectories. Recently published foundation models, such as Chronos-2 (Ansari et al., 2025) and TiRex (Auer et al., 2025) are exemplary for two distinct architectural paradigms: Chronos-2 is a Transformer-based model built on the T5 encoder architecture, a purely attention-based design adapted from LLMs. Ansari et al. extend it with a group-attention mechanism to support multivariate and covariate-informed forecasting. TiRex, by contrast, is based on the Extended LSTM (x-LSTM), and thus native to time series prediction. To the best of our knowledge, none of these pretrained foundation models have been applied directly to maritime or inland vessel trajectory prediction. Because these models are easy to set up and require no domain knowledge or underlying motion equations, they serve as highly attractive, low-effort baselines.

To address the limitations of existing studies to small sample sizes and narrow geographic constraints, this thesis provides a broader and more diverse evaluation basis. To achieve this, our work used a dataset of over four million sequences across multiple measurement stations for inland waterways with explicitly defined waterway domains. While incorporating multi-vessel interactions is highly relevant for autonomous surface vessels, it complicates the direct comparison between learned and kinematic models, as the latter cannot natively incorporate dependencies between vessels and would require a different input space. Within the modelling scope of this work, we therefore assume that predictions rely exclusively on the historical AIS data of a single vessel, meaning each target is predicted independently without relying on concurrent observations of neighbouring traffic. Finally, we incorporate the aforementioned foundation models into our evaluation framework to assess how their zero-shot capabilities compare against the other models under a unified optimisation and preprocessing pipeline.

2.2 Data Preprocessing and Context-Awareness

Raw AIS data contains errors such as physically implausible positions, inconsistent timestamps, and unreliable velocity readings (Zhao et al., 2018). Without systematic preprocessing, such artefacts propagate into learned motion representations and may corrupt model training. To address this, Zhao et al. (2018) proposed a foundational framework for AIS preprocessing. Their framework includes multiple steps. First, physical integrity checks are applied, where invalid data points violating fixed bounds are discarded and trajectories with fewer than 100 points are removed. Second, checks designed to enforce spatial integrity by removing physically implausible transitions are applied. Specifically, a trajectory is first partitioned wherever two consecutive points are more than 10 minutes apart. Sub-tracks resulting from this split that contain fewer than 100 points are discarded. Each surviving sub-track is then scanned for kinematic outliers and partitioned again if the difference between the recorded speed and the speed calculated from consecutive positions exceeds 15 knots. Following these partitions, the algorithm attempts to greedily stitch the resulting sub-tracks back together. Only if the kinematic transition between the end of one sub-track and the start of the next is physically plausible are they merged into a single track. Any reconstructed track that still contains fewer than 100 points is discarded. Third, time accuracy is restored by comparing the recorded timestamp of each point against the transmission timestamp generated by the AIS transponder itself. Where the deviation between the two is large and consistent across a track, the recorded times are shifted by that offset to align with the generated transmission times.

More recent work, such as Zaman et al. (2024), illustrates that a simpler pipeline remains the standard approach in many applied studies, rather than the elaborate preprocessing frameworks proposed by Zhao et al. (2018). To prepare their AIS dataset for trajectory prediction, Zaman et al. employ a sequential filtering algorithm. First, they filter the raw data by discarding points that fall outside specific geographical boundaries. Next, they remove basic outliers by dropping records with speeds exceeding 40 knots and invalid headings. The data are then grouped by individual ship identifiers, sampled down to regular one-hour intervals, and stripped of duplicate entries.

Finally, they apply interpolation to fill any temporal gaps in the hourly sequence and normalise the dataset for neural network training. The interpolation and normalisation techniques are not mentioned.

When more elaborate preprocessing is used, it is typically tailored to highly specific tasks rather than general tracking. For example, Capobianco et al. (2021) employ a preprocessing pipeline explicitly designed to extract and isolate known spatial patterns for sequence to sequence learning. Their procedure begins with standard filtering steps. They first apply a geographical bounding box and validate vessel IDs. Next, they restrict the dataset exclusively to tanker vessels to establish a uniform dataset. After grouping the remaining data by their IDs, they split the trajectories whenever the time gap between consecutive messages exceeds a maximum allowable threshold. They then apply linear interpolation to achieve a consistent 15 minute sampling interval. A defining characteristic of their data preparation is an explicit pattern selection step. The authors define specific Polygonal Geographical Areas over the region of interest and discard any trajectory that does not intersect a predefined origin and a predefined destination in a specific order. The resulting routes are then sliced into input and target sequences of a fixed length using a sliding window approach, and are annotated with a categorical destination label to condition their neural networks. While this intention aware approach successfully reduces displacement error by teaching the network to memorise specific predefined routes, it pursues a fundamentally different objective to generalised Multi-Target Tracking (MTT). By constraining the dataset to a few selected origin and destination pairs and explicitly feeding the destination into the model, the architecture becomes reliant on route memorisation rather than learning scalable and generalised vessel motion dynamics. The methodology proposed in our work explicitly seeks to avoid this limitation.

In summary, existing AIS preprocessing methods predominantly treat trajectories as purely kinematic sequences. Standard pipelines typically consist of geographic bounding boxes, physical integrity checks, time-gap splitting, and resampling. While our methodology builds upon these standard practices, we made adaptations to some of these steps for our work. For instance, we analytically derived the Rate of Turn (ROT) from the Course Over Ground (COG) and applied sliding windows and geometric augmentation. Furthermore, even when existing pipelines consider spatial geometry to extract common routes, the semantic context of the environment is generally overlooked. This lack of context is particularly limiting for inland navigation, where diverse waterway domains have distinct kinematic characteristics. For instance, the manoeuvring dynamics of a vessel approaching a lock differ fundamentally from its behaviour on an open river. Because standard pipelines process all trajectories identically regardless of their geographic context, the resulting models fail to leverage this underlying domain knowledge. While sufficiently large deep learning models might implicitly learn these spatial dynamics given large enough datasets, we hypothesise that explicitly encoding this domain knowledge is more sample-efficient and interpretable.

To address this gap, the main novelty of our preprocessing framework is the introduction of semantic domain labelling based on OpenStreetMap (OSM) data. Rather than relying purely on vessel kinematics, we assign categorical waterway labels (e.g., harbour, river, channel, lock) to each trajectory. As detailed in Chapter 3, we encode this domain knowledge through class-specific geographic buffers, priority mapping rules, and domain-aware dataset splitting and augmentation. This aligns directly with our first contribution: developing an automated pipeline that consolidates standard cleaning steps, extends them with semantic domain labels, and ultimately produces a curated, large-scale, and domain-aware dataset.

2.3 Hyperparameter Optimisation for Vessel Prediction

In existing maritime trajectory prediction studies, deep learning architectures are often evaluated with manually selected hyperparameters. Extensive HPO is usually left as an open challenge. For example, Donandt et al. (2022) manually fix their hidden layer size to 512, dropout to 0.1, and the number of encoder/decoder layers to 3, and explicitly acknowledge in their conclusion that

further hyperparameter optimisation needs to be conducted. Similarly, Yang et al. (2022) fix their Bi-LSTM hyperparameters without any systematic search procedure. Both studies are representative instances of a manual tuning practice. They result in sub-optimal configurations and make robust cross-study and fair comparisons between models difficult.

To overcome these limitations and ensure reproducible evaluations, Automated Machine Learning (AutoML) provides tools to automate HPO in a data driven way (Baratchi et al., 2024; Feurer et al., 2015). While AutoML has been successfully applied to some general time series forecasting tasks (Deng et al., 2023), its use in maritime motion prediction is very limited. Zhou et al. (2024) combine a Bi-LSTM model with the Optuna framework to tune a small set of hyperparameters, and demonstrate that automated HPO can improve accuracy over manually tuned baselines. However, they only apply this to a single architecture with a relatively small trial budget, and do not extend automated tuning to alternative model families. To the best of our knowledge, no prior work applies systematic HPO across multiple model families within a shared AIS preprocessing and evaluation pipeline. To address this, we apply systematic HPO methods, following the broader AutoML paradigm of automating model configuration, aiming to ensure a fair and reproducible benchmark across all evaluated model families.

Chapter 3

Data & Preprocessing

In this chapter, we detail the data preparation process, outlining the transition from raw Automatic Identification System (AIS) logs to a curated, domain-aware dataset. First, in Section 3.1, we introduce the publicly available AIS dataset used for our experiments. In Section 3.2, we outline the five-step preprocessing pipeline developed to extract, clean, segment, and normalise the raw trajectories. Finally, we summarise the key characteristics of the final dataset in Section 3.3.

3.1 Data Sources

In this work, we used publicly available AIS data from three different environments to ensure diversity in traffic patterns, waterway types, and vessel behaviours. The datasets cover measurement stations in three German cities, recorded by AIS receivers operated by the Federal Maritime and Hydrographic Agency (Bundesamt für Seeschifffahrt und Hydrographie (BSH)) as part of their ship emissions monitoring programme. These data are organised by measurement station and month, downloadable per day, and publicly accessible via GovData. They are provided under the *Data licence Germany – attribution – Version 2.0*¹. For each measurement station, we used all data available for the year 2025 to cover a complete annual cycle and avoid bias to seasonal characteristics (Bundesamt für Seeschifffahrt und Hydrographie (BSH), 2025a, 2025b, 2025c).

Raw Data Format. The AIS data are stored as semicolon-delimited log files with fields including navigation status, heading, Speed Over Ground (SOG), latitude/longitude, Course Over Ground (COG), true heading, and a UTC timestamp. The dataset assigns a unique identifier, the Maritime Mobile Service Identity (MMSI) to each vessel, allowing individual tracking.

3.2 Data Preprocessing

In this work, the raw AIS data underwent a five-step pipeline to produce the final samples: (1) Data extraction and cleaning, (2) geographic domain assignment, (3) resampling, (4) trajectory segmentation, and (5) feature engineering and normalisation. In this section, we explain these steps in detail. The resulting dataset features are presented in Table 3.1. A simplified visualisation of steps 1, 3 and 4 can be found in Figure A.2 in Appendix A.

¹<https://www.govdata.de/dl-de/by-2-0>

Field	Description
track_id	Unique identifier for each generated trajectory
role	Context or prediction
t_utc	Date and time of the recording (UTC)
x, y	Mean-centred UTM coordinates
sog	Speed over Ground in knots
cog	Course over Ground in degree
rot	Rate of Turn in degree / minute
context	geographic domain label
vessel_id	Hashed MMSI
dt	time distance to closest point
Normalised Features	
dx_norm, dy_norm	Relative movement between time steps
sog_norm	Normalised speed over ground
cog_sin_norm, cog_cos_norm	Course expressed as sine/cosine components
dt_norm	Normalised time distance to closest point
rot_norm	Normalised rate of turn

Table 3.1: Overview of Automatic Identification System (AIS) data fields available after preprocessing. The table distinguishes between the absolute fields used for evaluation and sample identification, and the normalised features used by the trajectory prediction models.

3.2.1 Data Extraction and Cleaning

The first stage of the pipeline focuses on extracting dynamic position information and removing erroneous records. Raw AIS log files contain multiple message types beyond vessel position reports, including voyage-related data, static data reports, base station reports, and aids-to-navigation reports (International Telecommunication Union, 2001). To isolate relevant vessel movements, navigation aids (buoys etc.) and aircraft were filtered out, and only message types carrying dynamic position information were retained. These provide SOG, COG, heading, and coordinates. All other messages were discarded prior to parsing. This initial parsing reduced the raw dataset by approximately 15%. During this extraction, known encoding errors were also corrected; specifically, we added a constant offset of 102.4 to correct SOG readings that were negative due to parsing errors, and converted the default heading values of 511 to missing data (NaN), as specified in International Telecommunication Union (2001). Additionally, duplicate records, which we define as two data points with the same timestamp and MMSI, were resolved. Duplicates can occur when multiple stations receive the same signals. Approximately two-thirds of the duplicates were exact matches and were directly discarded in the cleaning process; the one-third that were not exact matches were handled by retaining the row with the fewest zero entries as the first criterion and the oldest row as the second criterion, respectively.

Following the initial extraction, the dataset was filtered for spatial and kinematic outliers. First, the data were restricted to specific geographic bounding boxes for each target region, as detailed in Table 3.2, which functioned as a primary filter for large coordinate errors. Following the approach by Yang et al. (2022), within these boundaries, the dataset was further filtered by excluding any records with a reported SOG exceeding 30 knots as implausible sensor or transmission errors. In order to identify faulty Global Navigation Satellite System (GNSS) coordinates, we utilised the distance between consecutive points as follows: If the distance exceeded what is physically achievable based on the reported SOG of the vessel and the time that elapsed since the previous timestamp, the point was discarded. During this cleaning phase, no interpolation was performed to fill the resulting gaps. This prevented the introduction of synthetic data before the formal resampling stage, which was not necessary because of the large number of available data points. Our kinematic approach to filtering out implausible GNSS positions aligns with the methods proposed by Zhao et al. (2018), who filtered trajectories based on time gaps and physically implausible speed transitions.

Dataset	Bounding Box
Bremerhaven	lat 53.5°–54.0°N, lon 8.0°–8.8°E
Kiel	lat 53.5°–54.5°N, lon 9.2°–11.5°E
Wedel	lat 53.0°–54.0°N, lon 9.0°–10.5°E

Table 3.2: Geographic bounding boxes applied to the raw AIS data for each measurement station. Any recorded vessel positions falling outside these specified latitude and longitude ranges were discarded.

Domain	Bremerhaven	Kiel	Wedel
Harbour	800	1000	0
River	250	250	250
Lock	50	50	0
Channel	0	50	50

Table 3.3: Geometric buffer radii (in metres) per domain and harbour used to assign context to OpenStreetMap (OSM) coordinates. All points within those ranges of an OSM assignment were marked with the respective domain, according to the priority mapping.

Finally, because the Rate of Turn (ROT) is rarely transmitted by standard AIS transponders, it was analytically derived as the time derivative of the COG. To account for the circular nature of the COG feature, we calculated $ROT = \frac{((\Delta COG + 180) \bmod 360) - 180}{freq_s / 60}$ in degrees per minute. Positive values indicate a starboard turn and negative values a port turn. To minimise the impact of GNSS localisation noise, which causes fluctuations in COG and thus artificial spikes in ROT at low speeds, we applied two filters. The first filter excluded segments with an SOG below 0.5 knots. The second filter removed any remaining data points with a calculated absolute ROT exceeding $90^\circ / \text{min}$.

3.2.2 Geographic Domain Assignment

To enable the training of domain-aware models, each AIS message was assigned a waterway type, based on its coordinates, hereafter referred to as the geographic *domain*. We distinguish between `harbour`, `river`, `channel` and `lock` domain. To assign those labels, we extracted water bodies and maritime infrastructure from OpenStreetMap (OSM) via the OSMnx library (OpenStreetMap contributors, 2017), and matched vessel coordinates to these OSM polygons using a spatial join following geometric buffering. Because explicit harbour tags in OSM are often incomplete or unreliable, we employed a heuristic approach to classify waterway domains. First, we identified polygons tagged as locks, channels, and rivers (see Figure 3.1 for the full taxonomy of domain types). Any remaining valid water bodies within our bounding boxes that are not assigned to any of these types were defaulted to the `harbour` domain.

Furthermore, to account for GNSS inaccuracies and completely unmapped open port basins, we expanded each OSM polygon by a class- and dataset-specific radius before joining. For instance, we used a large buffer for harbours to capture vessels in unmapped bays and applied smaller buffers for channels and locks. In cases where these expanded polygons overlap, a priority mapping of `lock > river > channel > harbour` was applied to resolve conflicts (e.g., a lock is prioritised over a river). Points that fell outside the buffered water polygons are labelled as `unknown` and excluded from the training data. Given the nature of this assignment, the resulting assignments were verified through manual visual inspection and the buffers tuned accordingly to ensure geographic validity. A visualisation of the label definitions in Kiel can be seen in Figure 3.1, and the spatially assigned data applied to the Kiel and Bremerhaven datasets is illustrated in Figure 3.2. The buffers are available in Table 3.3.

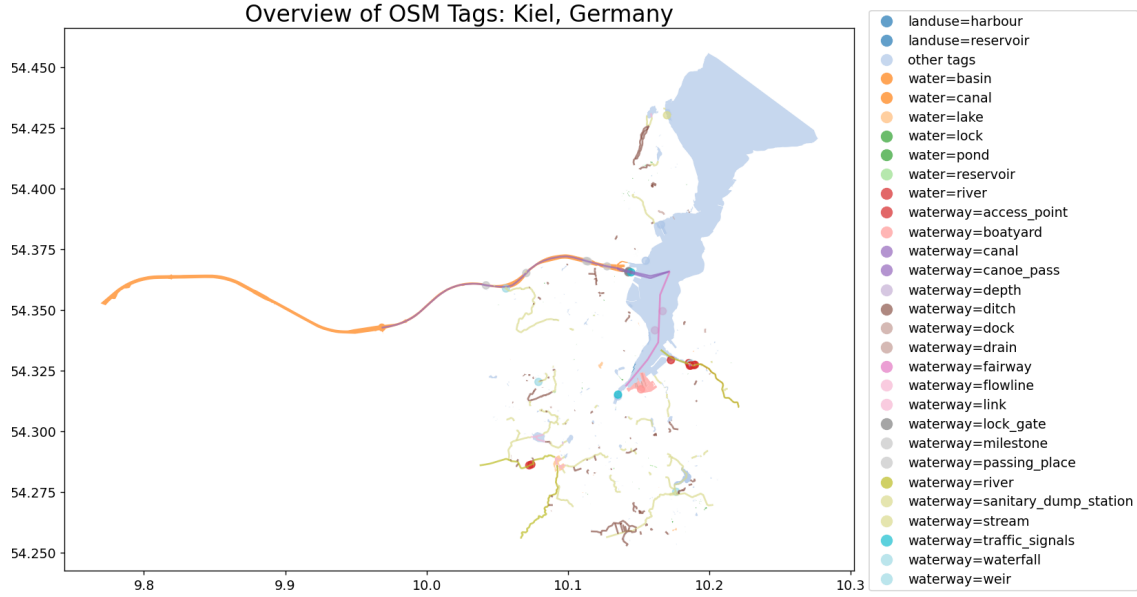


Figure 3.1: Spatial distribution of waterway types in the Kiel harbour area, extracted from OpenStreetMap contributors (2017). Distinct types are colour-coded across the map, with the legend on the right detailing the full taxonomy of available geographic domains. Axes denote longitude (x) and latitude (y).

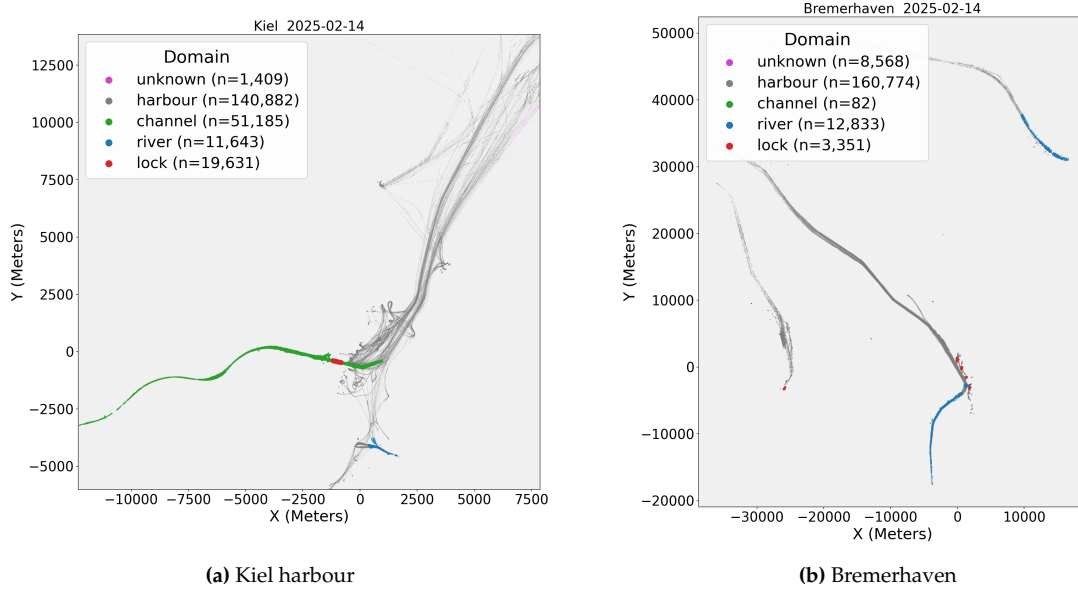


Figure 3.2: Visualisation of the geographic domain assignments for AIS trajectories in two measurement sites. The colour of each data point represents its corresponding waterway type: gray for harbour, green for channel, blue for river, red for lock, and purple for remaining exceptions. Axes denote longitude (x) and latitude (y).

Class A (cargo ships, tankers, etc.)		Class B (smaller/recreational)	
Condition	Interval	Speed	Interval
At anchor/moored, SOG ≤ 3 kn	3 min		
At anchor/moored, SOG > 3 kn	10 s	≤ 2 knots	3 min
0–14 knots	10 s	2–14 knots	30 s
0–14 knots + changing course	$3\frac{1}{3}$ s		
14–23 knots	6 s	14–23 knots	15 s
14–23 knots + changing course	2 s		
> 23 knots	2 s	> 23 knots	5 s

Table 3.4: Automatic Identification System (AIS) reporting intervals defined by the ITU-R M.1371-01 standard. Transmission frequencies are dynamically adjusted based on transponder class (Class A for large commercial vessels, Class B for smaller craft), vessel speed, and manoeuvring status.

3.2.3 Resampling

Accurate and consistent geometric distances are required for linear interpolation, for several motion models, and for our evaluation metrics. Therefore, geographic coordinates were projected to the Universal Transverse Mercator coordinate system (UTM), for which the zone was determined automatically from the median longitude of each dataset. The Cartesian representation ensures that linear interpolation produces geometrically correct intermediate positions, that normalisation operates on a uniform scale, and that Euclidean distance computations in loss functions are physically meaningful regardless of latitude. Further details regarding the coordinate projection can be found in Figure A.1 in Appendix A.

To address temporal characteristics, the pipeline must account for non-uniform AIS reporting frequencies, because transmission intervals are dynamically adjusted based on vessel class, speed, and manoeuvring status (International Telecommunication Union, 2001). Class A transponders are mandatory for large commercial vessels (typically > 300 GT international or > 500 GT domestic) and transmit significantly more often than the Class B transponders used by smaller or recreational vessels. The resulting variable intervals (summarised in Table 3.4) necessitate a standardised resampling approach. Therefore, the raw AIS data were resampled to a uniform 30s grid, which captures our target speed range of 2 to 14 knots and ensures that both Class A and Class B transponders remain viable for model training. The interpolation method varied according to the feature type. For most telemetry features, time linear interpolation was applied. Geographic coordinates were interpolated using Piecewise Cubic Hermite Interpolating Polynomial (PCHIP), which was preferred over natural cubic splines as the latter can produce oscillations near trajectory endpoints. PCHIP preserves monotonicity locally and yields smoother and more realistic trajectories than piecewise linear interpolation. Categorical attributes, such as the domain, cannot be interpolated mathematically and are instead inherited from the last valid AIS observation.

We acknowledge that interpolating data adds patterns to our dataset that may also be learned by the motion models. To quantify the reliability of interpolated values, we computed a temporal offset feature, denoted as Δt . This feature represents the absolute time difference between each resampled point and its nearest original observation: $\Delta t = |t_{\text{resampled}} - t_{\text{original,nearest}}|$. By including Δt as a feature in subsequent modelling steps, the system can account for interpolation uncertainty. A higher Δt indicates lower confidence in the precision of the interpolated state at a specific timestamp.

3.2.4 Trajectory Segmentation

In this stage of the pipeline, cleaned trajectories are transformed into samples suitable for training. It includes the following steps: (1) Stratified Train-Validation-Test Splitting, (2) Sliding Window

Segmentation, (3) Kinematic Filtering and (4) Data Augmentation. In the following, these steps are explained in detail.

Stratified Train-Validation-Test Splitting. In order to prevent data leakage, the dataset was partitioned at the vessel level rather than the sample level into a 70:15:15 train-validation-test split. Since every sliding-window segment generated from a given vessel is assigned entirely to the same split, no vessel-level information can leak between training and evaluation, regardless of how densely the sliding window is sampled within a split. If the data were instead split randomly by sample, the sliding window stride would cause the model to be trained and evaluated on overlapping temporal segments of the same vessel, which would constitute data leakage. However, a simple random split at the vessel level is also insufficient due to the high diversity in vessel behaviour and the resulting class imbalance. To ensure that each data split contains a representative distribution of motion dynamics, we used a stratified split based on k -means clustering. Prior to the split, a three-dimensional feature vector was generated for each unique vessel, made of its mean velocity, maximum absolute **ROT**, and primary geographic context. These features were standardised using Z-score normalisation and clustered into five groups. The optimal number of strata was estimated using the elbow method on a small subset of the dataset. Further details about this estimation can be found in Figure A.3 in Appendix A. Finally, the stratified trajectories were saved separated by evaluation split and geographic domain, to enable targeted training and evaluation of the models.

Sliding Window Segmentation. Next, trajectories were partitioned using a sliding window approach with a 5-minute context window and a 5-minute prediction horizon. This specific window size sits at the lower end of context lengths typically used in related inland literature (Donandt et al., 2022; You et al., 2020), and aligns with the collision risk alerting window considered practically relevant for inland navigation (Donandt et al., 2022). To increase data utility, a 1-minute stride was applied between consecutive samples. A temporal guard was implemented to discard segments where the elapsed time exceeded the theoretical duration ($T_{ctx} + T_{pred}$), to ensure that the models only learn from continuous motion sequences without gaps. The guard simultaneously handled any gaps in the original data and the gaps we introduced with filters in previous preprocessing steps. During this windowing process, generating completely homogeneous domain samples can be challenging due to **GNSS** noise causing rapid label alternations near polygon borders. To resolve this, a rolling majority vote over a 3-minute window (six samples at 30s frequency) was applied to smooth out label jitter. The trajectory was then split into sub-tracks when a persistent transition into a new geographic context occurred. Consequently, every sample includes only one domain. Since the domain is known at inference time, domain prediction was not required for this work.

Kinematic Filtering. To focus the model on active vessel dynamics rather than stationary behaviour, a two-tier speed filter was applied to every generated track. A sample was excluded if the maximum **SOG** within the segment did not exceed 0.5 knots or if more than 50% of the data points within the segment reported an **SOG** below 1.0 knots. While these filters removed a significant portion of the raw data, they were essential for isolating manoeuvring patterns and preventing bias towards stationary or moored vessels. Notably, more than 90% of points marked **lock** were filtered out in this step, resulting in a loss of more than 98% of the possible lock samples after further preprocessing, almost double the loss of all other domains at 50 to 60%. This aligned with our expectations, given that vessels spend a lot of time stationary in those locks. The dynamics we are interested in, entry and exit, were not filtered out.

Data Augmentation. The distribution of samples across geographic contexts is highly imbalanced, with lock passages being comparatively rare (Table 3.5). To ensure sufficient representation, data augmentation was applied to the **lock** segments within the training partition. Initially, the sliding window stride was reduced from 60 seconds to 10 seconds specifically for these segments. The tighter stride extracted a higher number of overlapping context-prediction samples from the same physical traversal, approximately doubling the number of original samples. Furthermore,

Table 3.5: Distribution of samples across geographic domains and measurement stations after preprocessing. Cell shading intensity corresponds to the relative volume of samples. The counts include augmented samples generated for the lock domain, which remains underrepresented compared to the heavily dominating harbour class.

Station	Harbour	Channel	Lock	River	Total
Kiel	2,397,340	470,044	51,534	748	2,919,666
Bremerhaven	1,863,012	23	22,023	70,787	1,955,845
Wedel	0	0	0	1,332,553	1,332,553
Total	4,260,352	470,067	73,557	1,404,088	6,208,064

each lock sample was augmented around its spatial centroid by rotating coordinates through 16 random angles and mirroring these rotated samples along their longitudinal axis. This added 32 augmented samples per genuine sample, which means that the latter make up 3.3% of the lock dataset. Through the combination of tighter striding and geometric transformations, the lock subset was significantly expanded to a total of 73,557 samples, as can be seen in the overview in Table 3.5. We avoided data leakage of synthetic duplicates of training samples into the held out validation and test sets completely by splitting the data first and applying the spatial augmentation exclusively to the training set. Despite this significant expansion of the lock class, the overall dataset remains inherently imbalanced, as the harbour dynamics still represent the vast majority of the data. We intentionally avoided downsampling of the majority classes at this stage to preserve the full scale of the captured traffic for the final dataset. The choice of how to strictly balance the classes is left to the model training phase or to future researchers using this data.

3.2.5 Feature Engineering and Normalisation

The final stage of the preprocessing pipeline generates the motion features used as model inputs and standardises them to ensure numerical stability during training. To ensure the deep learning models learn generalised motion patterns rather than memorising specific geographic locations, absolute GNSS positions and projected (x, y) coordinates were deliberately omitted from the feature set. This forces the networks to learn generalisable vessel dynamics rather than location-specific patterns. Instead, absolute UTM coordinates were converted into step-wise displacements (dx, dy) by calculating the difference between successive timestamps. Furthermore, to handle the $0^\circ/360^\circ$ wrap-around of the COG, the angle was represented using its sine and cosine components.

Following the feature transformation, Z-score normalisation was applied. To prevent data leakage, the normalisation parameters (mean μ and standard deviation σ) were strictly calculated from the training set. A single, global scaler was fitted this way and applied uniformly across all models and data subsets, including validation and test sets, without further modification. Using one shared scaler instead of a separate scaler per domain avoids introducing an additional source of variation between domain-specific models, but it also means that systematic scale differences between domains are not explicitly separated out at this stage.

The complete set of available features for model training is detailed in Table 3.1. The selection of which specific features to include for each model was handled using Automated Machine Learning (AutoML), as specified in section 5.2.4. After Normalisation, the data were parsed into Parquet files for faster loading times and more efficient memory usage.

Finally, to prepare the dataset for publication and ensure privacy, vessels were anonymised via SHA-256 hashing of the MMSI. We preserved positional privacy by expressing UTM coordinates relative to median position of the dataset. This removes any absolute geographic reference.

3.3 Characteristics of the Preprocessed Dataset

Following extraction and filtering, the 2025 dataset retained 40,249 unique vessels in Kiel, 26,908 in Bremerhaven, and 14,109 in Wedel. Preprocessing the Kiel dataset produced 2,919,666 samples across all contexts, as can be seen in Table 3.5. The figure shows the composition of the final dataset after running the full preprocessing pipeline, including augmentation for the lock datasets. The harbour context dominates across all stations, with Kiel contributing 2,397,340 samples, Bremerhaven 1,863,012, and Wedel exclusively providing 1,332,553 river samples.

Figure 3.3 shows the feature distributions of SOG and ROT as violin plots. The distributions confirm, that the applied filters produce a kinematically plausible set of trajectories which is consistent with the expected operating range of inland vessels and the speed limits in the respective areas. They also confirm that key characteristics differ between domains. The COG distribution can be better explained by looking at a polar histogram (Figure 3.4). The dominant headings reflect the routing geometry around each measurement station. Further qualitative comparisons of the generated samples for each domain are provided in Appendix A (Figures A.6, A.7, A.8, and A.9).

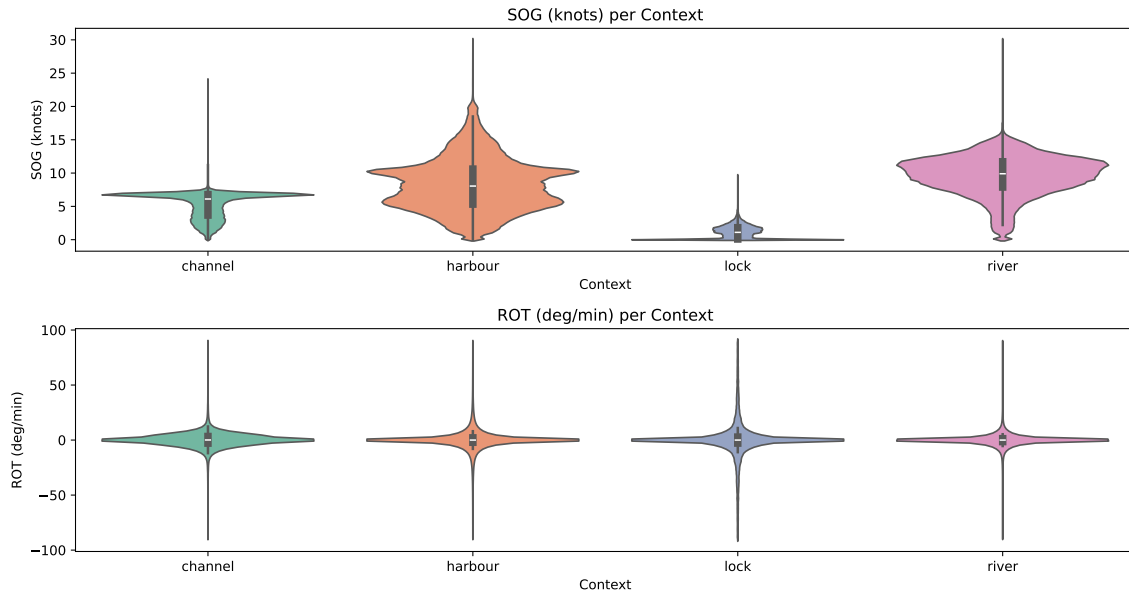


Figure 3.3: Kinematic characteristics of the preprocessed dataset. The violin plots display the distributions of Speed Over Ground (SOG) and Rate of Turn (ROT) grouped by geographic domain. These variations demonstrate that vessel behaviour differs significantly depending on the waterway environment.

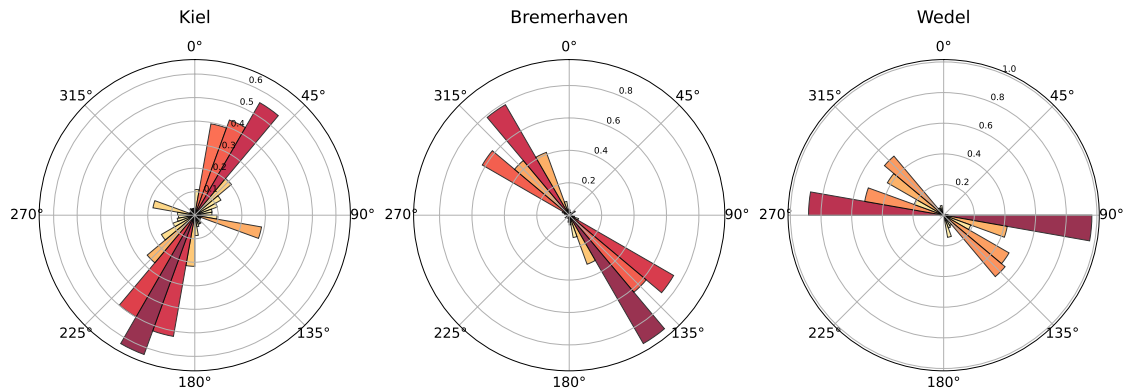


Figure 3.4: Polar Histogram of Vessel Course Over Ground (COG) after the Preprocessing Pipeline

Chapter 4

Methodology

In this chapter, we detail the methodological approach used to model vessel trajectories. We use Section 4.1 to establish the theoretical background required for sequence-to-sequence modelling and Section 4.2 for the principles of Automated Machine Learning (AutoML). In the subsequent sections 4.3 to 4.4, we describe the exact definitions, state representations, and architectural configurations of the implemented models. Implementation details for the learned models are laid out within the experimental setup in Section 5.3.

4.1 Sequence-to-Sequence Models

In the domain of Deep Learning, Sequence-to-Sequence (Seq2Seq) Models serve as a framework for tasks where both input and output take the form of ordered sequences, and the output sequence is conditioned on the input sequence as a whole. That is, each element of the output is not generated independently, but rather informed by the full context of the input (or previously generated outputs). While Seq2Seq is rooted in language processing tasks like translation (Sutskever et al., 2014), it can be applied to any kind of sequential data, ranging from numerical time series and sensor data to genomes and handwriting (R. M. Schmidt, 2019). Most Seq2Seq models are based on an Encoder-Decoder architecture. The Encoder processes the input sequence and compresses the information into a fixed-size context vector, typically called the hidden state. The Decoder uses this vector to generate the output sequence. Often, this regeneration is implemented autoregressively, meaning only one time step is predicted at a time, multiple times in a row with a shifting context window. To implement the encoder and decoder components within this Seq2Seq framework, an underlying architecture capable of processing ordered temporal data is required.

RNNs and LSTMs. The core architecture for sequence-based learning is the Recurrent Neural Network (RNN). The primary distinction between RNNs and conventional Feedforward Neural Networks is the direction of information flow. Instead of a strictly forward pass, an RNN utilises cycles to feed information back into the hidden state, which enables the incorporation of context from previous inputs. As the sequence length increases, standard RNNs suffer from the vanishing gradient problem, which limits their capability to learn long-term dependencies. To address this limitation, the Long Short-Term Memory (LSTM) architecture was introduced. The core innovation of the LSTM is a cell state regulated by three gates. The forget, input, and output gates allow relevant information to persist across many time steps. Furthermore, the architecture is mathematically equipped for multivariate sequence modelling. This allows the representation of non linear cross-channel correlations. When implementing LSTMs within Seq2Seq frameworks, several architectural configurations must be considered. Key design parameters include the number and direction of the hidden layers, as well as the choice of decoding strategy, which typically involves

either a direct linear mapping or an autoregressive approach. A more detailed mathematical description of the internal update processes for both *RNNs* and *LSTMs*, along with illustrations of the architectures can be found in Appendix C.1.

x-LSTM. Many variants and extensions of the *LSTM* have been proposed. Recently, Beck et al. (2024) introduced the Extended *LSTM* (*x-LSTM*) architecture to address bottlenecks of standard *LSTMs*. The primary goals of this architecture are to improve hardware scalability and to provide a larger memory capacity. To achieve this, two new *LSTM* layers were introduced: First, the Scalar *LSTM* (*s-LSTM*) introduces exponential gating, which allows the model to revise and update its memory more dynamically. Second, the Matrix *LSTM* (*m-LSTM*) replaces the traditional vector cell state and its element-wise updates with a matrix memory updated via matrix multiplication. This enables fully parallelised sequence processing during training. Using these layers, an *x-LSTM* can maintain the linear time complexity $\mathcal{O}(N)$ of conventional *RNNs* during inference, in contrast to the quadratic complexity of Transformers (Beck et al., 2024; Vaswani et al., 2017).

For our specific use case of predicting short 10-step samples, the expanded memory capacity offers limited benefits while introducing computational overhead. Furthermore, the architecture requires additional hyperparameters, such as the choice between *s-LSTM* and *m-LSTM* layers, as well as matrix dimensions. This expands the search space for Hyperparameter Optimisation (*HPO*) substantially. Nevertheless, because of its scaling capabilities, the *x-LSTM* builds the basis of the *TiRex* foundation model.

Attention and Transformers. While *LSTMs* successfully mitigate the aforementioned vanishing gradient problem, there remains a structural limitation for *Seq2Seq* architectures based on *RNNs* or *LSTMs*: The encoder must compress the entire input sequence into a single, fixed-size context vector. This compression creates an information bottleneck, causing the network to forget earlier parts of the sequence as the input length increases. To resolve this forgetting problem, Bahdanau et al. (2015) introduced the attention mechanism. Instead of relying on the final hidden state, attention allows the decoder to access all intermediate hidden states of the encoder. At each decoding step, the mechanism calculates an alignment score, which enables the model to assign higher weights to the timestamps most relevant to the current prediction. Building upon this, Vaswani et al. (2017) proposed the Transformer architecture, which completely discards recurrent layers (such as *RNN* or *LSTM*) in favour of Self-Attention. Self-Attention computes the relational importance of all data points within a sequence simultaneously. This enables the models to capture global dependencies. However, applying Transformers to time series, such as our Automatic Identification System (*AIS*) trajectories, presents new challenges. Unlike *LSTMs*, Transformers lack an inductive bias for sequential order and must rely on artificial positional encoding. A critique by Zeng et al. (2023) suggests that for short, physically continuous sequences where simple local dependencies dominate, Transformers can be unnecessarily complex, computationally expensive, and prone to overfitting compared to recurrent or linear models. Nonetheless, their capacity to parallelise efficiently has popularised Transformers as an architecture in large-scale predictive modelling.

Foundation Models. Because Transformers parallelise so effectively, they enabled the creation of foundation models. These are massive neural networks pretrained on vast, diverse datasets, across multiple domains such as weather, finance, and sensor data, to learn universal sequence patterns before being applied to a specific task. Once trained on very general data, they can be fine-tuned or adapted to specific tasks with relatively small amounts of task-specific data (Liang et al., 2024).

A prominent example of foundation models in time series forecasting is *Chronos* (Ansari et al., 2024), which tokenises time series values and processes them using a pure encoder architecture, or the more recent adaptation *Chronos-2* (Ansari et al., 2025), which uses group attention to maintain weights across channels and enable multivariate or covariate informed forecasting. For our vessel trajectory prediction, foundation models offer two application paradigms, of which we only employ the former due to a lack of pre-trained models:

- **Zero-Shot Forecasting:** The pretrained model generates predictions for AIS trajectories without ever having been trained on maritime data. Chronos in particular was developed to have comparable and occasionally superior zero-shot performance on new datasets, relative to methods that were trained specifically on them (Ansari et al., 2024).
- **Fine-Tuning (Warm Starting):** The model can be trained on our specific domain data. By initialising the training process with pretrained weights (a “warm start”) rather than random initialisation, the model requires significantly less domain data and computational time to adapt to the specific kinematics of inland vessels.

The **x-LSTM** architecture (Beck et al., 2024) addresses the lack of sequential bias of the Transformer while maintaining extreme scalability. Consequently, **x-LSTMs** provide the theoretical basis for **RNN**-based foundation models. The **TiRex** foundation model, which leverages the **x-LSTM**, was presented by Auer et al. (2025) and showed promising results in benchmarks against Chronos. However, the authors have noted the model does not perform well with multivariate time series data. As a consequence, applicability to AIS trajectories is limited, because the univariate model treats individual features as independent channels. Separating channels prevents the model from capturing cross-correlations. Since vessel dynamics are governed by physical constraints where Speed Over Ground (**SOG**) and Course Over Ground (**COG**) directly impact the trajectory, this approach may result in kinematically inconsistent predictions.

We benchmarked our self-trained **LSTM** model against Chronos-2 and TiRex to both investigate whether generalised, pretrained knowledge outweighs physically grounded, domain-specific recurrence, and to test the zero-shot performance of the **x-LSTM** based foundation model vs the Transformer based model.

4.2 Automated Machine Learning

The development of machine learning applications traditionally relies on human intervention and domain expertise. **AutoML** aims to reduce the manual effort and knowledge required by automating design choices. These choices include decisions regarding model architecture, evaluation, and optimisation (Hutter et al., 2019). The automated configuration of a machine learning model is fundamentally defined by three core components. First, a search space must be established to set the boundaries of all permissible model configurations. Second, a search strategy is required to systematically navigate this multidimensional space and identify promising candidates. Finally, we need to define performance estimation methods to evaluate the sampled configurations. In this work, **AutoML** is utilised specifically for **HPO**, as detailed in Section 5.2.4. The concepts of search spaces and search strategies are covered in the following subsections 4.2.1 and 4.2.2. The methodology for performance evaluation is addressed separately within Chapter 5.

4.2.1 The Search Space

The foundation of any **AutoML** problem is the definition of its search space. Using the search space, we set the boundaries of the automated system and define the set of all possible configurations that the optimiser is allowed to explore to find the configuration with the best performance. Depending on the intended level of automation, this search space can be formulated in multiple ways, scaling in complexity from tuning a single model to designing a full model architecture. A detailed mathematical formalisation of **HPO**, model selection, and the Combined Algorithm Selection and Hyperparameter Optimisation (**CASH**) problem is provided in Appendix C.3. Here, we give an overview:

Hyperparameter Optimisation. Hyperparameters control the learning process of a model, but they cannot be derived from training data via standard gradient-based optimisation. As a result, they must be set prior to training (Hutter et al., 2019). In **HPO**, the search space is restricted to the hyperparameters of a single, predefined learning algorithm. These parameters can be continuous, integer-valued, or categorical.

Model Selection. Model selection is applied at a higher level of abstraction than **HPO**. Instead of tuning the internal mechanics of a specific algorithm, the search space consists of a discrete set of different machine learning algorithms to identify the algorithm family that best models the underlying data distribution.

CASH Problem. In practice, model selection and **HPO** are intertwined, because the optimal hyperparameters depend entirely on the chosen algorithm. This interdependency is formulated as the **CASH** problem. When formulating **CASH** as a single, joint optimisation problem, the search space grows exponentially. The combination of discrete algorithm choices with their respective continuous, discrete, and categorical hyperparameters presents a massive computational challenge. Consequently, solving the **CASH** problem efficiently typically requires meta-learning or a warm start, where metadata and historical performance records from previously evaluated datasets are utilised (Baratchi et al., 2024). Because such extensive metadata is not available for the specific inland vessel trajectories utilised in this work, addressing the full **CASH** problem would require prohibitively long computation times. Thus, we could not address the **CASH** problem in this work.

NAS Problem. Neural Architecture Search (**NAS**) expands the search space even further by fully automating the design of the neural network architecture itself. Instead of selecting predefined algorithms, **NAS** optimises the arrangement of operations, layer connectivity, and network depth (Baratchi et al., 2024). Due to the immense size of the resulting search space, **NAS** requires infeasible computational resources and falls out of scope for this thesis. In practice, **NAS** is typically only applied once the fundamental type of layer (e.g., **LSTM** or Transformer) has already been established.

4.2.2 Search Strategies and Pruning

Once a search space is defined, the **AutoML** system requires a strategy to navigate it. Standard search strategies include Grid Search, which evaluates all predefined combinations, and Random Search, which samples configurations uniformly at random. While Random Search is generally more efficient in high-dimensional spaces, both methods are uninformed and do not utilise past evaluations to guide future selections (Hutter et al., 2019). Advanced, informed strategies, such as Bayesian Optimisation (**BO**), build a probabilistic surrogate model of the objective function to sample only the most promising regions of the search space. Traditional **BO** approaches relying on Gaussian Processes model the probability of the objective value given the hyperparameters directly (Hutter et al., 2019). This direct approach becomes computationally expensive as the number of evaluations grows and scales poorly in high-dimensional configuration spaces (Hutter et al., 2019). Moreover, Gaussian process based **BO** cannot natively handle mixed hyperparameter spaces and requires approximations to evaluate categorical variables (Baratchi et al., 2024). To address these limitations, we used the Tree-structured Parzen Estimator (**TPE**) algorithm (Bergstra et al., 2011) in this work. Rather than modelling the objective function directly, the **TPE** inverts the process by modelling the probability of the hyperparameters given the objective value, partitioning past observations into sets of good and bad configurations. This inversion provides a significant advantage for tuning complex sequence models, with the primary strength of the **TPE** lying in its robust handling of discrete and categorical hyperparameters (Baratchi et al., 2024). Furthermore, each iteration of the algorithm scales linearly with the number of hyperparameters. A more detailed technical description of **BO** and the **TPE** algorithm is provided in Appendix C.2.

Search strategies are frequently paired with pruning mechanisms. Through pruning, or early stopping, the training of unpromising configurations is terminated before the maximum allocated budget is consumed. In methods such as Successive Halving, a large number of configurations are evaluated for a few epochs, the worst-performing candidates are discarded, and more computational resources are allocated to the top performers.

4.3 Kinematic Models

In this section, we describe the kinematic models we used, and how they were implemented.

4.3.1 Constant Velocity

Despite not being the state-of-the-art, the Constant Velocity (**CV**) model is still applied in vessel motion tracking (Xiao et al., 2020). The model assumes that a vessel maintains a constant displacement per time step throughout the prediction horizon. Given the context sequence of projected positions $(x_1, y_1), \dots, (x_{T_c}, y_{T_c})$, the model computes displacements and averages the last n_{vel} of them to obtain a velocity estimate, as defined by Equation 4.1.

$$\overline{\Delta x} = \frac{1}{n_{\text{vel}}} \sum_{k=T_c-n_{\text{vel}}}^{T_c-1} (x_{k+1} - x_k), \quad \overline{\Delta y} = \frac{1}{n_{\text{vel}}} \sum_{k=T_c-n_{\text{vel}}}^{T_c-1} (y_{k+1} - y_k). \quad (4.1)$$

This estimate is then repeated for every prediction step and produces a straight-line trajectory, expressed in Equation 4.2:

$$\hat{x}_{T_c+i} = x_{T_c} + i \cdot \overline{\Delta x}, \quad \hat{y}_{T_c+i} = y_{T_c} + i \cdot \overline{\Delta y}, \quad i = 1, \dots, T_p. \quad (4.2)$$

The window size n_{vel} was treated as a hyperparameter and automatically optimised on the validation set. Small values make the estimate reactive to recent motion. Larger values smooth out short-term fluctuations at the cost of slower adaptation to changes in direction or velocity.

4.3.2 Constant Turn Rate and Velocity

The Constant Turn Rate and Velocity (**CTRV**) model relaxes the straight-line assumption by allowing the velocity vector to rotate at a constant rate. The displacement estimate $(\overline{\Delta x}, \overline{\Delta y})$ is computed identically to the **CV** model. We used the mean Rate of Turn (**ROT**) over the last $n_{\text{vel}} + 1$ context steps to find the heading change per step, resulting in Equation 4.3.

$$\Delta\psi = -\overline{\text{ROT}} \cdot \frac{\Delta t}{60}, \quad (4.3)$$

where $\overline{\text{ROT}}$ is given in degrees per minute. The sign is flipped because **ROT** is clockwise-positive in the nautical standard, opposite to the mathematical convention. Rather than treating the initial displacement as constant over the entire time step, the continuously rotating velocity vector is integrated exactly.

Substituting the observed position increments $\Delta x_i = \dot{x}_k \Delta t$ and $\Delta y_i = \dot{y}_k \Delta t$ into the exact constant-turn discretisation (Li & Jilkov, 2003, Eq. 62, p. 1346) yields Equations 4.4 and 4.5.

$$x_{i+1} = x_i + \frac{\Delta x_i \sin \Delta\psi + \Delta y_i (\cos \Delta\psi - 1)}{\Delta\psi}, \quad (4.4)$$

$$y_{i+1} = y_i + \frac{\Delta y_i \sin \Delta\psi - \Delta x_i (\cos \Delta\psi - 1)}{\Delta\psi}. \quad (4.5)$$

As $\Delta\psi \rightarrow 0$ both expressions recover the straight-line update. This linear approximation is substituted whenever $|\Delta\psi| < \varepsilon$ to avoid division by zero. The velocity vector is then rotated for the next step using Equation 4.6.

$$\begin{pmatrix} \Delta x_{i+1} \\ \Delta y_{i+1} \end{pmatrix} = \begin{pmatrix} \cos \Delta\psi & -\sin \Delta\psi \\ \sin \Delta\psi & \cos \Delta\psi \end{pmatrix} \begin{pmatrix} \Delta x_i \\ \Delta y_i \end{pmatrix} \quad (4.6)$$

This produces a circular arc trajectory. The single hyperparameter n_{vel} controls the averaging window for both the displacement and the ROT estimate and is optimised on the validation set.

In addition to this arc-based formulation, we also implemented a naive variant, which applies the displacement vector directly at each step as a straight-line, before rotating the velocity, skipping any arc corrections.

4.3.3 Hybrid Constant Velocity/Constant Turn Rate and Velocity

For each sample, the decision to apply CV or CTRV is based on the observed turning behaviour in the context window. The mean absolute ROT over the last n_{rot} context steps is computed, and if it exceeds a threshold τ_{rot} , the CTRV model is activated; otherwise CV is used, as summarised in Equation 4.7.

$$\text{model} = \begin{cases} \text{CTRV} & \text{if } \frac{1}{n_{\text{rot}}} \sum |\text{ROT}_k| \geq \tau_{\text{rot}}, \\ \text{CV} & \text{otherwise.} \end{cases} \quad (4.7)$$

The four hyperparameters $n_{\text{vel}}^{\text{CV}}$, $n_{\text{vel}}^{\text{CTRV}}$, n_{rot} , and τ_{rot} were jointly optimised on the validation set.

4.4 Stochastic Filters

Unlike purely kinematic models, which propagate a single deterministic state estimate, stochastic filters explicitly represent uncertainty by tracking both a state estimate and a measure of confidence in that estimate. Within this work, we investigated two stochastic filters:

4.4.1 Kalman Filter with Constant Velocity

The Kalman Filter (KF) combines model information and measurements to derive optimal filtering and to estimate the state of a dynamic system. Additionally, it contains information about the certainty of the estimation. It is frequently used in trajectory tracking and autonomous navigation (Urrea & Agramonte, 2021). The core operation of the KF, can be summarised as a recurring two-step algorithm of prediction and correction (Kalman, 1960). During prediction, the filter executes a pure forward simulation based on the system model and the previously estimated state. Dur-

ing correction, when a measurement becomes available, the measurement model is used to project the predicted state into a predicted measurement. The algorithm then determines the deviation between the actual measured signal and this predicted measurement. This residual, known as the innovation, is then used to correct the predicted state. This correction step functions as a least-squares problem, where a weighting matrix known as the Kalman Gain determines whether the filter should trust the model prediction or the sensor measurement more. The **KF** yields optimal state estimates under the assumption of linear system dynamics and Gaussian noise. For comparative purposes, we isolated the prediction phase in our implementation: during the context window, the filter runs normally with regular measurement corrections to stabilise its state estimate. Once the context is consumed, the filter performs predictions without corrections.

State and Motion Model. In this implementation we used the **CV** model as an underlying model. The filter uses a four-dimensional state vector $\hat{\mathbf{x}}_k = (x_k \ y_k \ \dot{x}_k \ \dot{y}_k)^\top$, encoding position and velocity in metres per time step. The constant velocity assumption is encoded in the state transition matrix **A**, derived from the kinematic equations $x_{k+1} = x_k + \dot{x}_k \cdot \Delta t$ and $\dot{x}_{k+1} = \dot{x}_k$ (and analogously for y). The observation matrix **C** extracts only the positional components of the state. Both matrices are formulated together in Equation 4.8.

$$\mathbf{A} = \begin{pmatrix} 1 & 0 & \Delta t & 0 \\ 0 & 1 & 0 & \Delta t \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \quad \mathbf{C} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix}. \quad (4.8)$$

Noise and Uncertainty Matrices. The diagonal covariance matrices were treated as hyperparameters. Using $\text{diag}(\cdot)$ to denote a diagonal matrix, they are defined as $\mathbf{Q} = \text{diag}(q_{\text{pos}}, q_{\text{pos}}, q_{\text{vel}}, q_{\text{vel}})$, $\mathbf{R} = \text{diag}(r_{\text{pos}}, r_{\text{pos}})$, and $\mathbf{P}_0^+ = \text{diag}(p_{0,\text{pos}}, p_{0,\text{pos}}, p_{0,\text{vel}}, p_{0,\text{vel}})$.

Q captures deviations from the ideal assumptions made by the **CV** model. Its diagonal structure assumes uncorrelated positional and velocity noise, neglecting the physical coupling that exists because velocities are derived from positional differences. **R** reflects the positional uncertainty of **AIS** reports, and $\mathbf{P}_0^+$ encodes the uncertainty of the filter prior to any observations. Together, **Q** and **R** control how strongly the filter weighs recent measurements against the motion model during the context phase, and thus determine the quality of the velocity estimate that seeds the prediction rollout.

4.4.2 Extended Kalman Filter with Constant Turn Rate and Velocity

The Extended Kalman Filter (**EKF**) (S. F. Schmidt, 1966) relaxes the strict linearity assumption of the standard Kalman Filter, by locally linearising the nonlinear system equations around the most recent state estimates. Specifically, the state transition model is linearised around the previously corrected state, while the measurement function is linearised around the newly predicted state. While the pure forward simulation of the state itself is executed using the actual nonlinear model, the filter uses these local linearisations to update the covariances and to compute the correction step. This approach allows the **EKF** to treat nonlinear systems consistently within the structure of the standard Kalman Filter.

The **EKF** we implemented uses the **CTRV** model and builds directly onto the arc integration introduced in Section 4.3.2. The key difference is that the turn rate was no longer estimated externally from **ROT** and held fixed, but instead treated as a latent state variable ω_k inferred by the filter during the context phase.

State and Motion Model. For the **EKF**, we define the state vector as $\hat{\mathbf{x}}_k = (x_k, y_k, \dot{x}_k, \dot{y}_k, \omega_k)^\top$. Denoting $\Delta\psi = \omega_k \Delta t$, the transition function f is formulated using the same arc integration as in the **CTRV** model with $\Delta\psi$ now depending on the current state rather than a fixed value, with the linear **CV** model substituted whenever $|\omega_k| < \varepsilon$. Because ω_k appears inside trigonometric functions, f is nonlinear in the state, and an **EKF** becomes necessary rather than the standard **KF**. The observation matrix $\mathbf{C}$ extracts only the position. Both the transition function f and the observation matrix $\mathbf{C}$ are formulated together in Equation 4.9.

$$f(\hat{\mathbf{x}}_k) = \begin{pmatrix} x_k + (\dot{x}_k \sin \Delta\psi + \dot{y}_k (\cos \Delta\psi - 1)) / \omega_k \\ y_k + (\dot{y}_k \sin \Delta\psi - \dot{x}_k (\cos \Delta\psi - 1)) / \omega_k \\ \dot{x}_k \cos \Delta\psi - \dot{y}_k \sin \Delta\psi \\ \dot{x}_k \sin \Delta\psi + \dot{y}_k \cos \Delta\psi \\ \omega_k \end{pmatrix}, \quad \mathbf{C} = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \end{pmatrix}. \quad (4.9)$$

Although **ROT** is available as an **AIS** field in principle, it is missing for a large proportion of messages in the dataset. It was therefore recomputed during preprocessing based on **COG** (cf. Section 3.2.1). Since **COG** is itself derived from successive Global Navigation Satellite System (**GNSS**) positions, the recomputed **ROT** is not an independent observation and is therefore excluded from the measurement.

Linearisation via Jacobian. The **EKF** updates the predicted covariance using the Jacobian $\mathbf{A}_k = \partial f / \partial \mathbf{x}|_{\hat{\mathbf{x}}_k^+}$, recomputed at each step. Following the standard **EKF** linearisation (Thrun et al., 2005), the full Jacobian is defined by Equation 4.10. The complete derivation can be found in Appendix C.4.

$$\mathbf{A}_k = \begin{pmatrix} 1 & 0 & \frac{\sin \Delta\psi}{\omega_k} & \frac{\cos \Delta\psi - 1}{\omega_k} & \frac{\Delta\psi \dot{x}_{k+1} - (\dot{x}_k \sin \Delta\psi + \dot{y}_k (\cos \Delta\psi - 1))}{\omega_k^2} \\ 0 & 1 & \frac{1 - \cos \Delta\psi}{\omega_k} & \frac{\sin \Delta\psi}{\omega_k} & \frac{\Delta\psi \dot{y}_{k+1} - (\dot{y}_k \sin \Delta\psi - \dot{x}_k (\cos \Delta\psi - 1))}{\omega_k^2} \\ 0 & 0 & \cos \Delta\psi & -\sin \Delta\psi & (-\dot{x}_k \sin \Delta\psi - \dot{y}_k \cos \Delta\psi) \Delta t \\ 0 & 0 & \sin \Delta\psi & \cos \Delta\psi & (\dot{x}_k \cos \Delta\psi - \dot{y}_k \sin \Delta\psi) \Delta t \\ 0 & 0 & 0 & 0 & 1 \end{pmatrix} \quad (4.10)$$

$\dot{x}_{k+1}$ and $\dot{y}_{k+1}$ denote the already rotated velocity components from f . The Jacobian $\mathbf{A}_k$ is used to predict the covariance according to $\mathbf{P}_{k+1}^- = \mathbf{A}_k \mathbf{P}_k^+ \mathbf{A}_k^\top + \mathbf{Q}$, while the state is predicted directly with the nonlinear function f .

Noise, Uncertainty, and Context Window. The covariance matrices extend those of the **KF** with the **CV** by one dimension, resulting in $\mathbf{Q} = \text{diag}(q_{\text{pos}}, q_{\text{pos}}, q_{\text{vel}}, q_{\text{vel}}, q_{\text{rot}})$, $\mathbf{R} = \text{diag}(r_{\text{pos}}, r_{\text{pos}})$, and $\mathbf{P}_0^+ = \text{diag}(p_{0,\text{pos}}, p_{0,\text{pos}}, p_{0,\text{vel}}, p_{0,\text{vel}}, p_{0,\text{rot}})$. The turn rate is initialised to $\omega_0 = 0$. The context phase is restricted to the most recent n_{ctx} observations, making the state estimate more responsive to recent manoeuvres. All eight parameters $q_{\text{pos}}, q_{\text{vel}}, q_{\text{rot}}, r_{\text{pos}}, p_{0,\text{pos}}, p_{0,\text{vel}}, p_{0,\text{rot}}$, and n_{ctx} are treated as hyperparameters.

Chapter 5

Setup of Experiments

In the following, we detail the experimental setup used to evaluate the performance of the motion models discussed in the preceding chapter. We begin by outlining the central research questions and the specific experiments conducted to address them in Section 5.1. Subsequently, we describe the unified evaluation framework in Section 5.2, including the data splits, metrics, and the optimisation and training procedure. Implementation details are described in the last Section 5.3.

5.1 Research Questions and Experiments

The goal of the thesis is to provide a fair comparison between deep learning models, kinematic models, and stochastic filters, and to investigate the impact of incorporating domain information into model training and architecture. In particular, the following research questions should be answered:

- **RQ1:** Given a preprocessed dataset and a systematic hyperparameter optimisation procedure, how do deep learning models, both task-specific and pretrained, kinematic baselines, and stochastic filters compare in short-term trajectory prediction of inland vessels?
- **RQ2:** Does domain-specific training improve short-term vessel trajectory prediction performance compared to mixed-domain training?
- **RQ3:** Does adding explicit domain information as one-hot encoded input improve short-term vessel trajectory prediction performance?

To answer the research questions, we designed three experiments, which differ in the train-test partitioning, the use of domain-specific training, and the use of domain encoding.

- **Exp I: Baseline Comparison of Motion Models:** Models were trained and evaluated on the full dataset. This experiment served as the overall comparison across models and model families and aims to answer RQ1.
- **Exp II: Effect of Domain-Specific Training on Predictive Performance:** Models were trained once separately per domain and once on the mixed dataset. They were then evaluated separately within each domain. This experiment tested whether domain-specific training improves predictive performance compared to mixed-domain training and aims to answer RQ2.
- **Exp III: Ablation of Explicit Domain Encoding for LSTMs:**

This study investigated whether explicitly providing domain information as one-hot encoded input improves LSTM performance. To answer RQ3, three LSTM variants were compared under the same data split and evaluation protocol:

- **LSTM**: baseline model without domain information.
- **LSTM+Domain**: augments the input with one-hot encoded domain information.
- **LSTM+Domain***: uses the same architecture as LSTM+Domain but the hyperparameter configuration of the baseline LSTM.

To assess robustness with respect to random initialisation and non deterministic CUDA training, we additionally trained the baseline LSTM and LSTM + Domain* ten times each with different random seeds, and evaluated the results on the overall test set.

5.2 Evaluation Framework

To systematically execute the experiments outlined above and ensure a fair comparison of prediction quality across all model families, a unified evaluation framework was required. This section details the shared pipeline we used to perform the comparison. All experiments utilised the full one-year dataset without downsampling. The data for all experiments were downloaded and preprocessed as described in Chapter 3. A fixed random seed was used across all models and experiments to support reproducibility (see Table B.1 in Appendix B). Note, however, that this seed does not cover the data augmentation applied during preprocessing, so all experiments must be run on the same, already-generated Parquet dataset rather than regenerating preprocessing independently. Each model received the normalised feature set as a context window and output displacement increments ($\Delta x, \Delta y$). The kinematic models additionally received x and y coordinates as an input, an input we omitted for the deep learning models as per Section 3.2.5. The predicted displacements were cumulatively summed and offset by the last known position to reconstruct absolute trajectories before any metric was applied. Hyperparameter Optimisation (HPO) was performed exclusively on the validation set. The final model and parameters were then evaluated on the test split without retraining.

5.2.1 Train-Val-Test Split

While formal definitions of the HPO and Combined Algorithm Selection and Hyperparameter Optimisation (CASH) problem typically use k -fold cross-validation to guide the search (Baratchi et al., 2024; Feurer et al., 2015), this is mainly designed for traditional machine learning on smaller datasets. For deep learning on our large Automatic Identification System (AIS) trajectory datasets, we instead used a fixed split between training, validation, and test dataset. In particular, we used a stratified 70:15:15 split, as specified in Section 3.2.4. At the scale of our AIS dataset, which can easily be extended with additional recordings of the past years, a single validation set is sufficient to guide hyperparameter search without multiplying training time by k . Additionally, we avoided the risk of data leakage by randomly splitting overlapping time series data completely.

5.2.2 Metrics

All trajectory-based metrics were computed from the Euclidean distance between predicted and ground-truth positions. The evaluation was performed over all files in the considered evaluation split and included all predicted time steps, not only the final one, which is separately reflected by the Final Displacement Error (FDE). Consistent with common trajectory prediction practice, we additionally reported the Average Displacement Error (ADE) over all predicted time steps and samples (Donandt et al., 2022; Yang et al., 2022).

In **HPO**, we optimised the Root Mean Square Error (**RMSE**) of the predicted x and y coordinates. **RMSE** is a standard metric for regression-based prediction tasks and is also used in vessel trajectory prediction by Yang et al. (2022) and Donandt et al. (2022) (called *ATE* in the latter). It was chosen because, due to the squared error term, it penalises large deviations more strongly than absolute-error-based metrics. This is desirable in our setting, where occasional large prediction errors are more problematic than many small ones. In addition to the aggregated metrics, per-file metrics were computed and stored. They can serve for further analysis and, in our case, were used for post-hoc statistical tests and to investigate seasonal trends.

Finally, because the foundation models (TiRex and Chronos-2) generate probabilistic quantile forecasts, we measured calibration metrics for their evaluation. Specifically, we assessed the quality of their uncertainty estimates using mean interval width, prediction coverage, and the Winkler score evaluated at the 80% confidence interval.

5.2.3 Statistical tests

To compare the candidate models statistically, we used the Friedman test followed by a Nemenyi post-hoc test. The Friedman test is a nonparametric alternative to repeated-measures ANOVA and is suitable for comparing multiple models on the same evaluation blocks without assuming normally distributed scores. In our setting, each file served as one block and each model as one treatment. For every file, the models were ranked according to their **RMSE**, with rank 1 assigned to the lowest error. The Friedman test then assessed whether the models differ significantly ($p < \alpha = 0.05$) in their average ranks across blocks.

5.2.4 Optimisation and Training

Stochastic filters and deep learning architectures depend on hyperparameter configurations that cannot be easily derived from training data and are impractical to tune manually at scale. Manual tuning risks a biased evaluation, where a model might underperform simply due to a suboptimal configuration rather than an inherent lack of predictive capability. To guarantee a fair comparison, each model was optimised independently using Automated Machine Learning (**AutoML**) in the form of Hyperparameter Optimisation (**HPO**). The scope was deliberately restricted to **HPO** rather than the full **CASH** formulation, since domain-specific meta-learning data for a warm start was unavailable. The set of candidate model classes (kinematic models, Kalman Filters (**KFs**), Long Short-Term Memory (**LSTM**) networks, and foundation models) was chosen manually, while their respective hyperparameters were explored automatically.

We centralised the **HPO** for all models. We used the Optuna framework (Akiba et al., 2019), which supports flexible definition of search spaces, including continuous, integer, and categorical parameters. Optuna was selected because it is easy to set up locally on both Windows and Linux systems, and can be expanded to be used on a cluster. For the Deep Learning models, the hyperparameter search was driven by the Tree-structured Parzen Estimator (**TPE**). To optimise computational resources, we employed an early-stopping strategy: an **RMSE**-based callback stopped the search if no improvement greater than a small threshold was observed within a predefined patience window. Also, all tuning experiments were parallelised, with the exception of the Hybrid CV/CTRV, which used the Constant Velocity (**CV**) and Constant Turn Rate and Velocity (**CTRV**) models after those were tuned. To further speed up convergence, we initialised the hyperparameters of the stochastic filters with values found via **HPO** on a small subset of the data.

For learned models (**LSTMs**), the training procedure during and after **HPO** was as follows. We applied pruning using successive halving to reduce total compute: trials were evaluated at fixed epoch checkpoints, where only the top $1/\eta$ fraction of trials ($\eta = 2$) were allowed to continue, while the rest were terminated early. With `min_resource = 8` and `min_early_stopping_rate = 0`, the schedule evaluated configurations at epochs 8, 16, and 32. Within each surviving trial, an inde-

pendent early-stopping criterion halted training if the validation [RMSE](#) did not improve by more than `min_delta` within a fixed patience window. To illustrate these two mechanisms, [Figure 5.1](#) depicts the intermediate validation errors during the [HPO](#) process of [LSTM](#) training. It demonstrates how underperforming trials are terminated early at these specific epoch checkpoints and how epochs with little improvement over time are additionally pruned. The best-found configu-

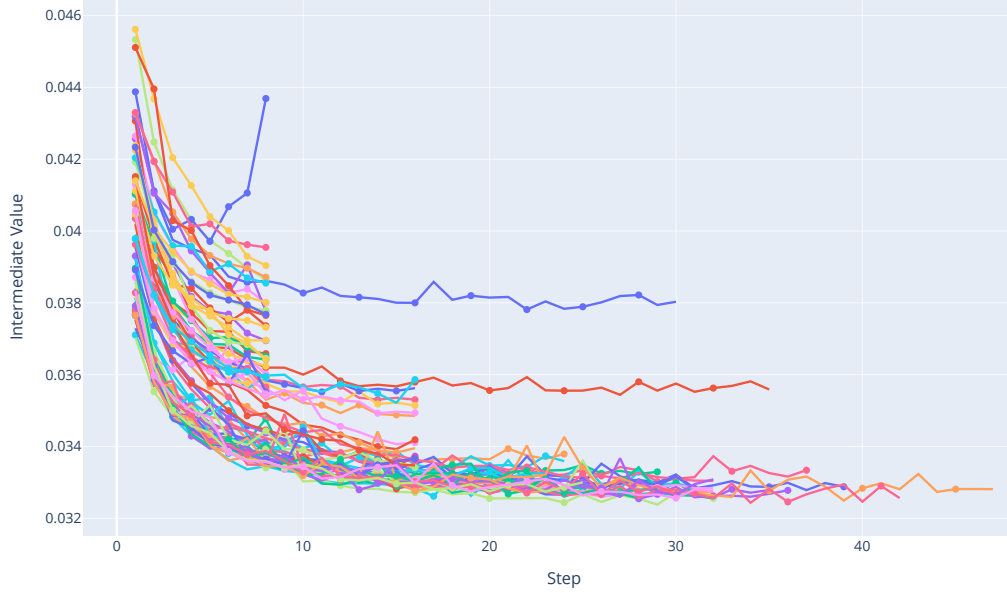


Figure 5.1: Intermediate Value plot of Long Short-Term Memory ([LSTM](#)) Hyperparameter Optimisation ([HPO](#)) in Optuna with early stopping enabled. Successive Halving was applied to all trials, except the initial 16. The Drop-offs at step 8, 16 and 32 indicate points, where the current batch was evaluated and the bottom-performing configurations were terminated.

ration was retrained from scratch on the full training set for up to `max_epochs` epochs using the same early-stopping criterion, and the resulting checkpoint was used for final evaluation on the test split. It should be noted that parameter importance scores derived from a pruned study are biased towards values that are important early on in the convergence, rather than final validation loss, since most trials do not run to full completion.

Optuna keeps track of intermediate results, enabling us to analyse the learning dynamics after an optimisation run. A complete specification of the hyperparameter optimisation methodology and the configurations for all experiments is detailed in [Table B.1](#) in [Appendix B](#). Furthermore, the corresponding hyperparameter search spaces for each model are defined within the respective methodology sections ([4.3.1](#) to [4.4.2](#)) and are additionally summarised in [Table B.2](#) in [Appendix B](#).

Search Spaces and Strategies. The optimisation strategy varies according to the complexity of the model:

- **Kinematic Models:** These models contain small, 1-Dimensional search spaces. We employed a full grid search over all valid configurations to ensure the global optimum for these parameters was found. [TPE](#) with $n \text{ trials} \gg \text{valid configurations}$ can also find the global optimum, but comes with a computational overhead we wanted to avoid. For fairness, we only used the validation split for this optimisation, ignoring the training split.
- **Filters:** We mapped the Covariance and Uncertainty matrices to a multi-dimensional continuous search space using log-uniform distributions, as the optimal values typically span several orders of magnitude. The search was then guided using [TPE](#).

Hyperparameter	Type	Values / Range
hidden_size	Categorical	{64, 128, 256}
num_layers	Fixed	2
dropout	Continuous	[0.1, 0.4] (uniform)
learning_rate	Continuous	$[10^{-4}, 10^{-2}]$ (log-uniform)
batch_size	Categorical	{128, 256, 512}
feature_set	Categorical	$\{\{dx_{\text{norm}}, dy_{\text{norm}}\} \cup S \mid S \in \mathcal{P}(\mathcal{A})\}$

Note: $\mathcal{A} = \{\{\sin(\text{cog})_{\text{norm}}, \cos(\text{cog})_{\text{norm}}\}, \text{sog}_{\text{norm}}, \text{rot}_{\text{norm}}, \text{dt}_{\text{norm}}\}$

Table 5.1: Hyperparameter search space used for the [LSTM](#) model optimisation. Continuous parameters are sampled from uniform or log-uniform distributions, categorical parameters are selected from predefined sets. For the `feature_set`, the search space consists of all feature combinations formed by the base input $\{dx_{\text{norm}}, dy_{\text{norm}}\}$ united with any subset S drawn from the power set $\mathcal{P}(\mathcal{A})$. The sin and cos components of Course Over Ground ([COG](#)) ($\sin(\text{cog})_{\text{norm}}$ and $\cos(\text{cog})_{\text{norm}}$) are treated as an indivisible pair within $\mathcal{A}$.

- **Foundation Models:** Chronos is designed as a pure Zero-Shot model and requires no [HPO](#). For TiREX, we optimised the size of the input window using the same 1-D space as the kinematic models. The optimisation is required, because the TiREX does not implement any attention mechanisms that can weight specific timestamps.
- **Learned Sequence Models:** These models involve higher-dimensional parameter spaces. We used [TPE](#) to navigate the search space.

Feature Selection. Within the optimisation process of our models, we treated the input feature subset as a categorical hyperparameter. Rather than relying on manual feature ablation, the selection of which [AIS](#) signals to include was integrated directly into the search space. This reflects prior research showing that optimal feature subsets and model hyperparameters are interdependent, as the best configuration for one inherently depends on the other (Feurer et al., 2015). The automated selection was not used for univariate models (TiREX), where other channels cannot influence performance, and models that implement attention across channels (Chronos-2), since those already autonomously identify the most relevant contributions during training and make explicit feature search redundant. To make sure that all 16 candidate feature sets were covered at least once in model training of the [LSTM](#), one trial was enqueued to the study for each feature set candidate. Those trials were identical, except for the feature set. After that, 10 additional startup trials were sampled uniformly at random before the optimisation began. The full search space for the [LSTMs](#) is listed in Table 5.1.

5.3 Implementation Details

In this section, we describe the sequence models we have selected for vessel trajectory prediction and how they were implemented. Two categories were evaluated: foundation models applied zero-shot, and [LSTM](#) models trained on the full dataset.

5.3.1 Foundation Models

TiRex. Auer et al. (2025) designed TiRex as a Univariate Zero-Shot Forecaster that leverages the new scaling capabilities of the Extended [LSTM](#) ([x-LSTM](#)). Our implementation is characterised by:

1. Univariate channels: To predict dx, dy , we fed the model with the normalised dx_{norm} and dy_{norm} channels, which the model treats independently.

2. Zero-Shot Evaluation: We used the model as is. Because no attention mechanism is implemented, we optimised only the input length. Due to the univariate character, feature selection was not a factor in our optimisation.

Chronos-2. In addition to the univariate TiRex, we implemented Chronos-2 (Ansari et al., 2025) as a zero-shot transformer for time series forecasting. Unlike the [LSTM](#) based model, it lacks the inductive bias for time series. Instead, it supports co-variate informed forecasting. The model was designed to estimate a probability distribution over future trajectories rather than computing a deterministic path. Our implementation is characterised by:

1. Multivariate Covariates: To predict dx, dy , we fed the model with all available normalised features: *sog_norm*, *cog_sin_norm*, *cog_cos_norm*, *dt_norm*, *rot_norm* as covariates, in addition to the *dx_norm*, *dy_norm* channels.
2. Zero-Shot Evaluation: We used the Model as is, without doing any [HPO](#) or finetuning. To avoid overhead, we deliberately did not add the feature selection to our [HPO](#), since we can assume, that the cross-channel attention of Chronos-2 already gives low weights to irrelevant features. For a similar reason, the context length was set to the maximum 10 steps available after preprocessing, as the self-attention implemented in the transformer can recognise irrelevant timestamps.

5.3.2 LSTM

The [LSTM](#) represents learned sequence models in our comparison. In contrast to the kinematic models and filters, which encode an explicit motion assumption, the [LSTM](#) can learn temporal dynamics directly from the data. Unlike the foundation models, it was trained from scratch on the [AIS](#) dataset, giving full control over architecture, feature set, and training procedure.

Architecture. The model follows a Sequence-to-Sequence ([Seq2Seq](#)) encoder–decoder design (Figure 5.2): a stacked [LSTM](#) encoder processes the context window and compresses it into a hidden state, which seeds the decoder. The decoder then predicts all future displacements simultaneously. We picked the linear prediction instead of an autoregressive decoder, due to the faster compute time. The number of layers was fixed at 2 to keep the search space reasonable. Dropout was applied between layers for regularisation. More details about the feature selection can be found in Section 5.2.4.

Domain Encoding. In the [LSTM](#) + *domain* variant, the domain is represented as a 4-dimensional one-hot encoded vector, with one dimension per class. Instead of appending the domain vector as an input feature at every timestep, it is added to the initial hidden and cell states of both LSTM layers. Specifically, the last 4 nodes of the initial hidden state h_0 and cell state c_0 are set to the one-hot vector. The remaining nodes are initialised to zero, as is the default on the baseline model. This keeps the total hidden size identical between models and ensures a fair comparison. Encoding the domain only in the initial states is acceptable here, since we focus on short term predictions and only need to retain the contextual information for 10 timesteps. We considered adding additional groupings for domains with similar expected behaviours or constraints, but decided against this in order to avoid introducing potentially incorrect assumptions about the domain structure.

A learned embedding layer was considered as an alternative. However, for only four domain classes, one-hot encoding is preferable as it is parameter-free, exact, and directly interpretable. In contrast, learned embeddings introduce additional trainable parameters and reduce interpretability, since the resulting coordinates no longer correspond directly to individual domain classes. This trade-off is usually more attractive for higher-cardinality categorical variables; if the number

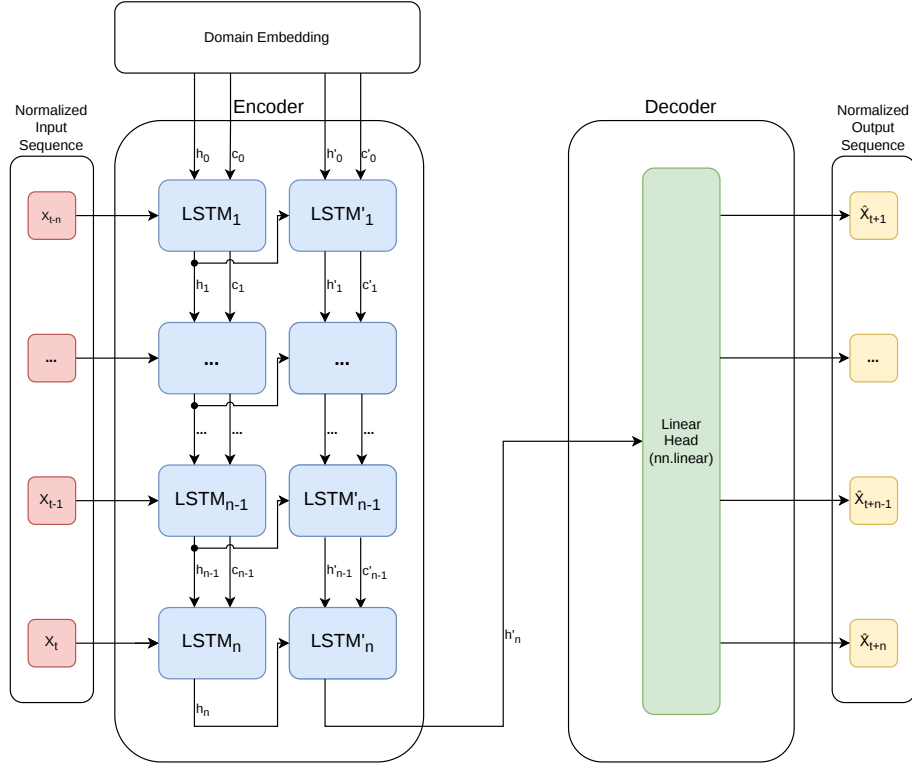


Figure 5.2: The Long Short-Term Memory (LSTM) Architecture implemented in this work: Two LSTM Encoding Layers feed into a Linear Decoder. The Initial Cell and Hidden state are initialised with the Embedded Domain. The figure is self-made.

of domains were to grow substantially in future work, an embedding-based representation could become worth it.

5.4 Reproducibility and Code Availability

The preprocessed dataset is provided at Hugging Face¹. The dataset can be regenerated with the pipeline provided at RWTH GitLab² or the public GitHub mirror³. The repository also provides the code to train and optimise all models. For details on the pipeline, refer to the readme files and Appendix D for a general overview of the codebase architecture.

¹<https://huggingface.co/datasets/kawihss/ais-inland-vessel-trajectories>

²<https://git-ce.rwth-aachen.de/g-nav-mob-irt/projects/galileonautic2plus/multi-object-tracking/motion-models/-/tree/handedIn>

³<https://github.com/kawihss/Data-based-vessel-motion-modelling-Thesis/tree/handedIn>

Chapter 6

Results and Discussion

In this chapter, we present the findings from the experimental evaluation, and address the core research questions systematically. The overall goal of these experiments was to provide a fair comparison of deep learning models, kinematic models, and stochastic filters, as well as to investigate the impact of incorporating geographic domain information into model training and architecture. To evaluate the three research questions outlined in Section 5.1, we conducted a total of three experiments: a baseline comparison of all motion models on the full dataset (Exp I), an assessment of the effect of domain-specific training on predictive performance (Exp II), and an ablation study of explicit domain encoding for LSTMs (Exp III). The remainder of this chapter is structured around the questions, for each of which we present the quantitative results first. A more detailed discussion of the results follows, along with the answers to our research questions.

RQ 1: *Given a preprocessed dataset and a systematic hyperparameter optimisation procedure, how do deep learning models, both task-specific and pretrained, kinematic baselines, and stochastic filters compare in short-term trajectory prediction of inland vessels?*

We set up Exp. I to answer this question and to serve as an overall baseline comparison across all evaluated motion models and model families on the full dataset.

Results. The optimised hyperparameter configurations for all evaluated models are listed in Table 6.1, and the corresponding test set error metrics are detailed in Table 6.2. Specifically, the standard Long Short-Term Memory (LSTM) achieves an Average Displacement Error (ADE) of 54.78 m and a Root Mean Square Error (RMSE) of 111.36 m, reducing the error of the Constant Velocity (CV) baseline (ADE 78.39 m, RMSE 155.73 m) by 30.1% and 28.5%, respectively. To assess the overall statistical significance of these results, we conducted a Friedman test combined with Nemenyi post-hoc comparison. The significant differences are illustrated as a Critical Difference (CD) diagram in Figure 6.1. The diagram reveals a clear ranking structure: the two LSTM-based models occupy the top positions, with average ranks of 1.5 and 2 respectively, followed by the Extended Kalman Filter (EKF) (4.7) and Chronos-2 (5.4), while the remaining models form a middle group with mostly non-significant pairwise differences (6.4 – 7). The CTRV-based models rank last (8.1).

Figure 6.2 depicts the development of the ADE for all models over the prediction horizon. While the error for the CV model grows approximately linearly, the errors for CTRV and CTRV Arc diverge heavily at longer horizons. The LSTM-based models maintain the lowest ADE across all ten prediction steps. The ranking order of all models stays consistent over the full horizon.

Regarding the foundation models, Table 6.3 shows the probabilistic calibration metrics. TiRex produces extremely narrow prediction intervals (Mean Interval Width of 6.41) and achieves a coverage of only 0.059. In contrast, Chronos-2 produces wider intervals (Mean Interval Width of 145.37) but achieves a near-ideal coverage of 0.769. Figure 6.3 visualises the error distribution shapes as violin

Table 6.1: Optimised hyperparameter (HP) configurations for all models. Discussed parameters are shaded.

Model	HP	Value
CV	n_{vel}	1
CTRV	n_{vel}	2
CTRV Arc	n_{vel}	3
Hybrid	$n_{\text{vel}}^{\text{CV}}$	1
	$n_{\text{vel}}^{\text{CTRV}}$	2
	n_{rot}	1
	τ_{rot}	29.28°/min
KF (CV)	q_{pos}	1.578×10^{-3}
	q_{vel}	5.932
	r_{pos}	1.077×10^{-2}
	$p_{0,\text{pos}}$	7.737×10^{-2}
	$p_{0,\text{vel}}$	1.190×10^{-1}
EKF (CTRV)	q_{pos}	15.957
	q_{vel}	4.290×10^{-2}
	q_{rot}	1.002×10^{-7}
	r_{pos}	1.792×10^{-2}
	$p_{0,\text{pos}}$	4.046×10^{-1}
	$p_{0,\text{vel}}$	1.845×10^{-3}
	$p_{0,\text{rot}}$	2.076×10^{-6}
	n_{ctx}	2
Model	HP	Value
Chronos-2	<i>no HPO</i>	
TiRex	context length	1
LSTM	hidden size	256
	num. layers	2
	dropout	0.290
	learning rate	5.530×10^{-4}
	batch size	128
	feature set	no rot
LSTM+Domain	hidden size	256
	num. layers	2
	dropout	0.300
	learning rate	4.000×10^{-4}
	batch size	256
	feature set	all

plots, showing that TiRex and the CV model share nearly identical error distributions, while the LSTM distribution is distinctly shifted toward lower errors.

Furthermore, an evaluation of the error distributions for the kinematic baselines (CV and Constant Turn Rate and Velocity (CTRV)) reveals a stark deviation from a normal distribution, with coefficients of determination (R^2) ranging from 0.6138 to 0.6428.

Finally, the hyperparameter optimisation process for the LSTM is visualised in Figure 6.4. It presents a parallel coordinate plot of the parameter search with a clear band around the optimal configuration. The plot indicates that the optimisation selected a feature set that explicitly excluded the Rate of Turn (ROT). Furthermore, Figure 6.5 plots the average RMSE per month over the full dataset, with consistent performance trends across all models.

Table 6.2: Test set error metrics for all models

Model	ADE	FDE	RMSE
CV	78.39	175.82	155.73
CTRV	82.92	195.18	164.51
CTRV Arc	85.21	196.78	168.29
Hybrid CV/CTRV	78.49	176.45	155.49
KF (CV)	78.39	175.82	155.73
EKF (CTRV)	78.94	174.88	151.50
Chronos-2	80.25	179.80	152.86
TiRex	78.37	175.82	155.71
LSTM	54.78	121.18	111.36
LSTM + Domain	53.80	119.02	109.95

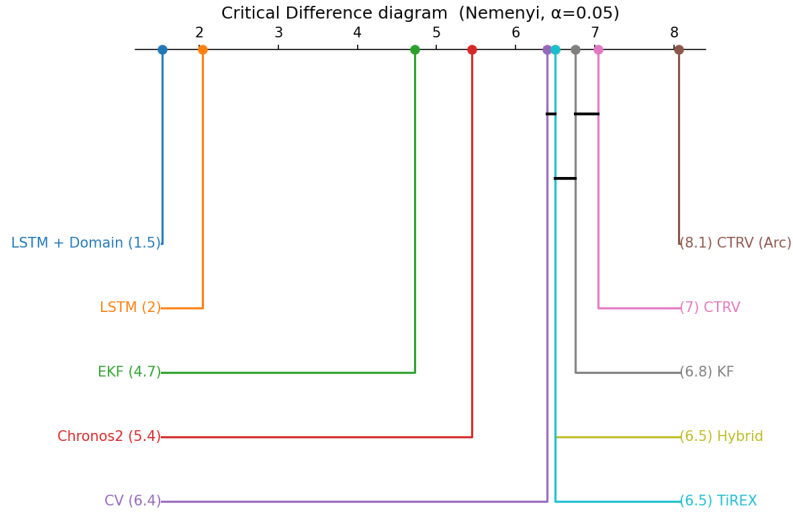


Figure 6.1: Critical Difference (CD) diagram based on the Friedman test and Nemenyi post-hoc comparison. Models connected by a horizontal bar are not significantly different at $\alpha = 0.05$. The shown numbers are the mean rank per model in the evaluation, 1 being the best possible result.

Table 6.3: Exp. I Probabilistic Metrics for Foundation Models

Model	MIW	Coverage	Winkler80
Chronos-2 (zero-shot)	145.37	0.769	114.65
TiRex	6.41	0.059	112.20

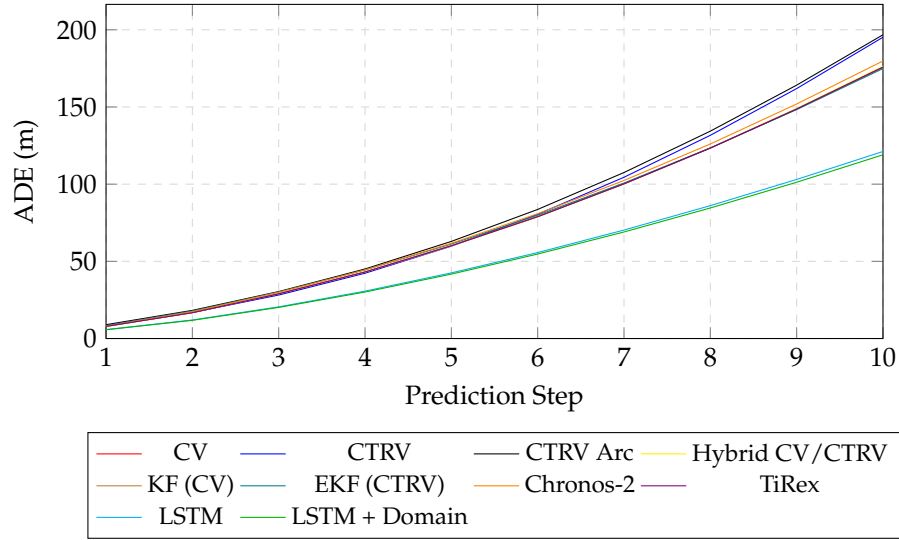


Figure 6.2: Progression of Average Displacement Error (ADE) over time, showing the divergence of error across all ten prediction steps. The plot demonstrates that Long Short-Term Memory (LSTM)-based architectures achieve the lowest errors, while kinematic models exhibit higher trajectory divergence, particularly as the prediction horizon increases.

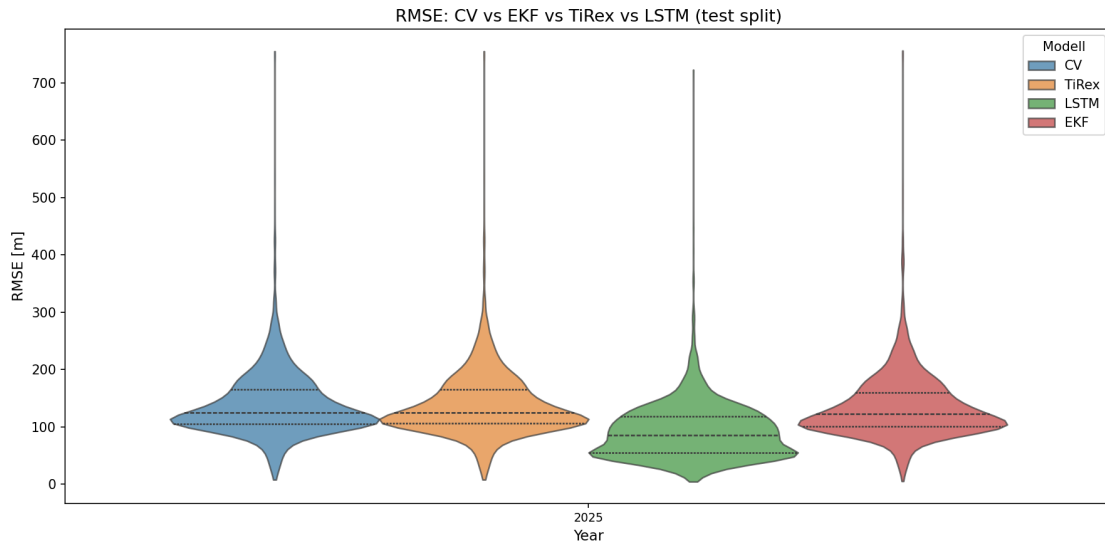


Figure 6.3: Violin plots visualising the distribution of Root Mean Square Error (RMSE) for the Constant Velocity (CV), Extended Kalman Filter (EKF), TiRex, and Long Short-Term Memory (LSTM) models on the test set. The LSTM model demonstrates a distinct shift toward lower error values compared to the other models, whereas the Constant Velocity (CV) and TiRex models exhibit nearly identical error distributions.

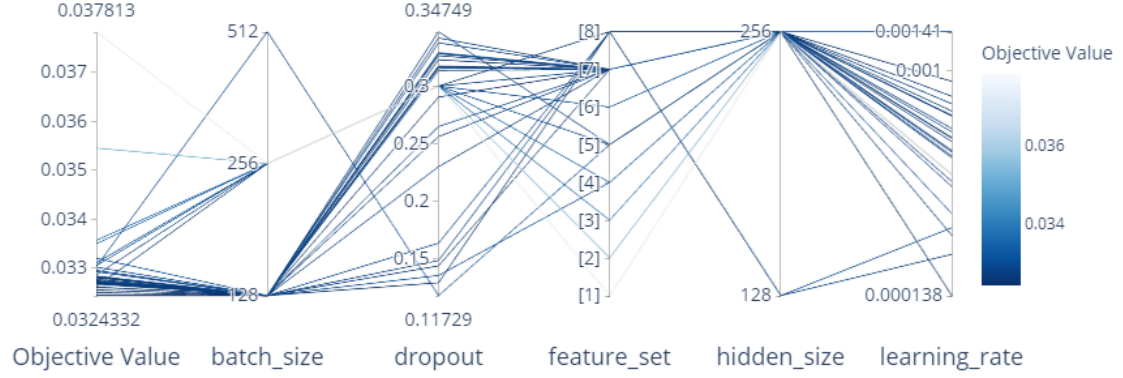


Figure 6.4: Parallel Coordinate Plot of Long Short-Term Memory (LSTM) Hyperparameter selection, showing a clear band around the selected hyperparameter configuration. Feature set mappings: [1] dx_norm, dy_norm; [2] dx_norm, dy_norm, sog_norm; [3] dx_norm, dy_norm, sog_norm, rot_norm; [4] dx_norm, dy_norm, sog_norm, cog_sin_norm, cog_cos_norm; [5] dx_norm, dy_norm, sog_norm, rot_norm, cog_sin_norm, cog_cos_norm; [6] dx_norm, dy_norm, sog_norm, rot_norm, dt_norm; [7] dx_norm, dy_norm, sog_norm, cog_sin_norm, cog_cos_norm, dt_norm; [8] dx_norm, dy_norm, sog_norm, rot_norm, cog_sin_norm, cog_cos_norm, dt_norm.

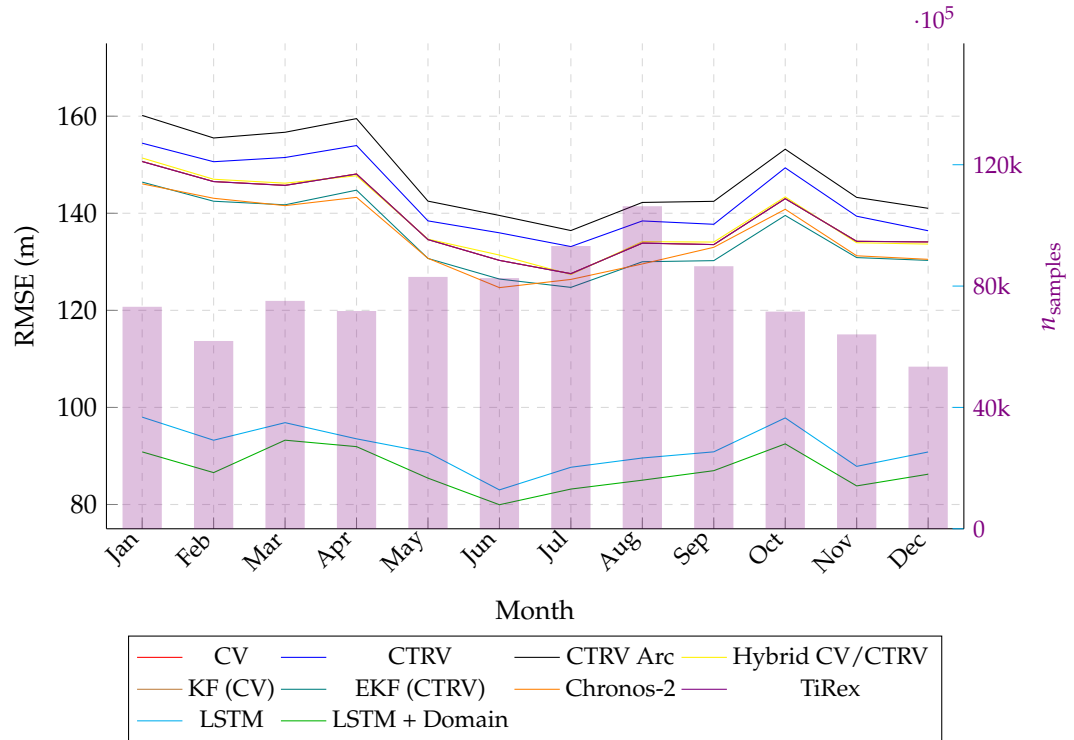


Figure 6.5: Comparison of Root Mean Square Error (RMSE) by month, demonstrating consistent performance trends across all models. Violet bars indicate the monthly volume of samples.

RQ 2: *Does domain-specific training improve short-term vessel trajectory prediction performance compared to mixed-domain training?*

We set up Exp. II to evaluate the extent of these performance differences and how they are influenced by both the specific waterway domain and the underlying model complexity.

Results. For the **CV** model, no change in performance (0.00 m difference) is observed between mixed and domain-specific training. Since the **CV** search space consists of the single hyperparameter n_{vel} , and the optimiser converged to $n_{\text{vel}} = 1$ in every domain as well as in the mixed setting, the resulting models are identical across all training regimes. The **CTRV** variants exhibit deviations of up to 12.55 m in **RMSE**. For the **EKF** and **LSTM**, however, substantial performance drops occur under mixed training, particularly in the channel and lock domains (Table 6.5, shaded in orange). Specifically, in the lock domain, the **ADE** of the **LSTM** degrades by 13.36 m and the **RMSE** by 13.11 m when evaluated with the mixed training model. This performance difference is not uniform across domains: in the harbour and river domains, only minor deviations are observed, generally remaining below a 5.6 m difference in **RMSE**. Finally, it can be observed that the harbour domain is the only environment where domain-specific training fails to produce a statistically significant improvement for the **LSTM**. Further details about the statistical significance across domains can be found in Figure A.10 in Appendix A.

In Table 6.5, the corresponding test set error metrics evaluated per domain are presented on the left, alongside the performance delta when evaluating the mixed training model on the right. Positive deltas indicate better performance for the models optimised on domain-specific data. The lowest absolute errors are achieved in the channel domain; notably, the **LSTM** attains an **ADE** of 32.09 m and an **RMSE** of 55.85 m (shaded in cyan). Conversely, the highest errors are consistently produced in the harbour domain (e.g., an **LSTM ADE** of 61.07 m), making it comparatively difficult to predict. When comparing model families, the largest relative improvements are observed in the lock and channel domains when moving from kinematic models to learned models. Changes in the harbour domain are inconsistent, with only minor improvements for the **LSTM** and measurable degradation for the **EKF** (a delta of -1.47 m **RMSE**).

In Table 6.4, the optimised hyperparameter configurations for both mixed and domain-specific training setups are listed. Notably, the optimised hyperparameters vary substantially across the domains. This applies not only to the learned models but also to the Kalman filters. The optimised values for measurement noise, process noise, and initial covariance differ considerably. The measurement noise r_{pos} covers nearly five orders of magnitude across models and domains, ranging from approximately 1.0×10^{-2} to 7.3×10^2 . In particular, highly distinct parameter selections are yielded by the lock class for all models. For example, very distinct hyperparameters can be observed for the **LSTM** (such as a reduced hidden size of 64 and an increased batch size of 512). The **EKF** applied to this domain is assigned a larger measurement noise r_{pos} coupled with a very small process noise q_{pos} (both shaded in green). Similarly, for **TiRex** and **EKF**, a longer context length is selected specifically for the channel domain ($n_{\text{ctx}} = 9$ and 6, respectively), while for kinematic baselines, the context length is reduced (shaded in cyan). For the lock domain, in contrast, the **EKF**'s context length is reduced to $n_{\text{ctx}} = 1$. With only a single observation available before prediction begins, the filter has little opportunity to correct its $\omega_0 = 0$ initialisation if the vessel is already turning. Combined with the low process noise q_{pos} and high measurement noise r_{pos} discussed above, the filter is effectively optimised to trust its motion assumption over new measurements, which likely contributes to the disproportionately large performance gap between the **EKF** and the **LSTM** specifically in this domain.

Table 6.4: Optimised hyperparameter configurations for per-domain training and mixed training. Discussed parameters are shaded in.

Model	Hyperparameter	Mixed	Harbour	River	Channel	Lock
CV	n_{vel}	1	1	1	1	1
CTRV	n_{vel}	2	2	2	1	1
CTRV Arc	n_{vel}	3	3	3	2	1
Hybrid CV/CTRV	n_{vel} (CV)	1	1	1	1	1
	n_{vel} (CTRV)	2	2	2	1	9
	n_{rot}	1	1	1	1	9
	τ_{rot}	29.28	29.28	29.28	1.00	72.76
KF (CV) EKF (CTRV)	q_{pos}	1.578×10^{-3}	1.578×10^{-3}	2.434×10^{-3}	2.483×10^{-3}	1.578×10^{-3}
	q_{vel}	5.932	5.932	5.115	9.930	5.932
	r_{pos}	1.077×10^{-2}	1.077×10^{-2}	3.035×10^1	1.005×10^{-2}	1.077×10^{-2}
	$p_{0,\text{pos}}$	7.737×10^{-2}	7.737×10^{-2}	2.941	1.637×10^{-2}	7.737×10^{-2}
	$p_{0,\text{vel}}$	1.190×10^{-1}	1.190×10^{-1}	1.556×10^{-3}	3.510×10^{-1}	1.190×10^{-1}
	q_{pos}	15.957	2.067×10^{-2}	2.645×10^1	1.140×10^{-1}	5.667×10^{-2}
	q_{vel}	4.290×10^{-2}	3.573×10^{-1}	1.646×10^{-3}	2.197×10^{-1}	3.625×10^{-1}
	q_{rot}	1.002×10^{-7}	2.206×10^{-5}	1.378	1.295×10^{-6}	6.197×10^{-7}
	r_{pos}	1.792×10^{-2}	4.662×10^{-2}	9.167×10^{-1}	2.378×10^1	7.339×10^2
	$p_{0,\text{pos}}$	4.046×10^{-1}	4.862×10^2	4.471×10^{-1}	4.161×10^2	2.951×10^3
	$p_{0,\text{vel}}$	1.845×10^{-3}	1.713×10^{-2}	3.479×10^{-4}	6.428×10^1	1.348×10^{-4}
	$p_{0,\text{rot}}$	2.076×10^{-6}	3.404×10^{-1}	1.896×10^{-6}	3.569×10^{-6}	7.577×10^{-5}
	n_{ctx}	2	1	1	6	1
TiRex	context length	1	1	1	9	1
LSTM	hidden size	256	256	256	128	64
	dropout	0.290	0.232	0.377	0.165	0.378
	learning rate	5.530×10^{-4}	4.852×10^{-4}	1.935×10^{-4}	9.003×10^{-4}	3.049×10^{-3}
	batch size	128	128	256	128	512
	feature set	no rot	all	all	all	no sog, no dt

Table 6.5: Test set error metrics for mixed training and separated training, evaluated per domain. Per-domain results are represented as a delta to the mixed training results. Discussed results are shaded in.

Model	Metric	Mixed Training				Δ (Mixed Training – Per-domain)			
		Harbour	River	Channel	Lock	Harbour	River	Channel	Lock
Constant Velocity	ADE	81.76	69.84	73.45	73.47	0	0	0	0
	FDE	182.44	157.16	171.98	167.58	0	0	0	0
	RMSE	166.25	134.29	110.43	100.73	0	0	0	0
CTRV	ADE	89.94	68.47	62.48	86.55	0	0	+0.98	+6.23
	FDE	212.11	160.64	145.04	197.34	0	0	-1.36	+9.93
	RMSE	178.60	135.47	96.58	116.72	0	0	+0.07	+6.10
CTRV Arc	ADE	92.44	69.83	65.58	94.17	0	0	+3.33	+12.46
	FDE	213.77	160.82	150.38	210.11	0	0	+3.27	+20.13
	RMSE	182.62	138.41	100.85	125.02	0	0	+3.16	+12.55
Hybrid CV/CTRV	ADE	81.89	69.86	73.56	74.80	0	0	+12.09	+1.33
	FDE	183.25	157.30	172.28	170.91	0	0	+26.05	+3.33
	RMSE	166.02	133.90	110.61	103.15	0	0	+14.03	+2.42
KF (CV)	ADE	81.76	69.84	73.45	73.47	0	+0.13	0	0
	FDE	182.44	157.16	171.98	167.58	0	+0.18	0	0
	RMSE	166.25	134.29	110.43	100.73	0	+0.02	0	0
EKF (CTRV)	ADE	82.79	70.84	68.35	76.53	-1.43	+0.50	+6.43	+3.06
	FDE	182.93	155.95	158.69	173.23	-1.42	-1.56	+13.67	+5.65
	RMSE	162.19	130.09	103.00	103.89	-1.47	-3.46	+7.05	+3.16
Chronos-2	ADE	84.88	72.47	61.62	64.92	0	0	0	0
	FDE	189.21	162.80	145.44	154.50	0	0	0	0
	RMSE	163.56	133.37	97.09	94.87	0	0	0	0
TiRex	ADE	81.76	69.78	73.50	73.47	0	0	+4.39	0
	FDE	182.44	157.14	172.00	167.58	0	0	+7.21	0
	RMSE	166.24	134.22	110.46	100.73	0	0	+3.11	0
LSTM	ADE	61.07	43.29	32.09	45.41	+0.39	+3.01	+4.81	+13.36
	FDE	135.73	94.14	70.23	93.17	+1.12	+7.26	+11.09	+26.54
	RMSE	121.45	91.59	55.85	68.59	+0.50	+5.51	+7.36	+13.11

RQ3: Does adding explicit domain information as one-hot encoded input improve short-term vessel trajectory prediction performance? To answer this, we set up Exp. III.

Results. In Table 6.6, we recall the performance metrics of the standard **LSTM** (first introduced in Table 6.2 and 6.5) as a benchmark for comparison, alongside the metrics of two domain-aware variants: **LSTM + Domain**, which was independently tuned using a separate hyperparameter optimisation study, and **LSTM + Domain***, a variant using the exact same hyperparameter configuration as the baseline architecture. Both domain-aware models achieve lower overall error metrics than the standard **LSTM**. For example, the **LSTM + Domain*** variant reduces the overall **RMSE** from 111.36 m to 110.20 m, representing a decrease of 1.04%. This overall figure is dominated by the harbour domain, which accounts for the majority of samples and shows almost no improvement (121.45 m to 121.38 m **RMSE**), mirroring the finding from Exp. II. The channel and lock domains, by contrast, show substantially larger relative gains (shaded in cyan). For instance, the **ADE** decreases from 32.09 m to 29.22 m in the channel domain, and from 45.41 m to 31.43 m in the lock domain. To ensure that the observed performance gap is robust, the baseline **LSTM** and the **LSTM + Domain*** model were trained ten times with different random seeds. Across all ten runs, strictly lower test errors in **ADE**, **FDE**, and **RMSE** were recorded for the **LSTM + Domain*** model. In Figure 6.6, the corresponding performance distributions of both models are presented. Finally, the combined critical difference diagram confirming this statistical significance is depicted in Figure 6.7, with the **LSTM + Domain*** model ranking 1.71 on average, **LSTM + Domain** achieving an average rank of 1.85 and the default **LSTM** ranking last at 2.44.

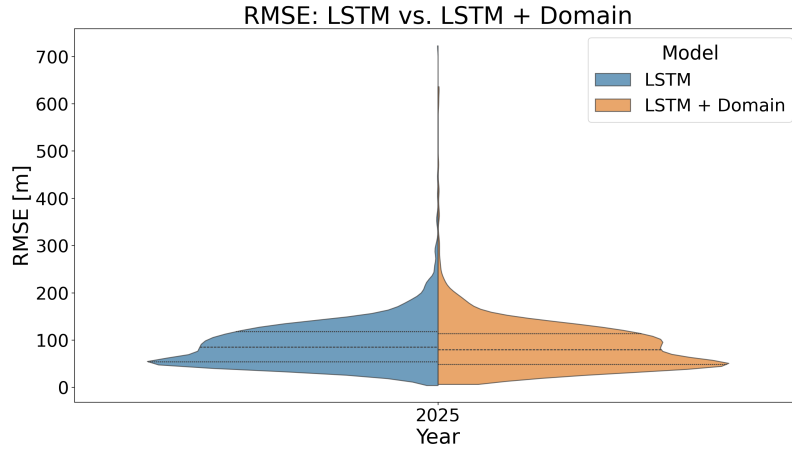


Figure 6.6: **RMSE** violin plot comparing the standard Long Short-Term Memory (**LSTM**) and the **LSTM + Domain*** models on the full test set.

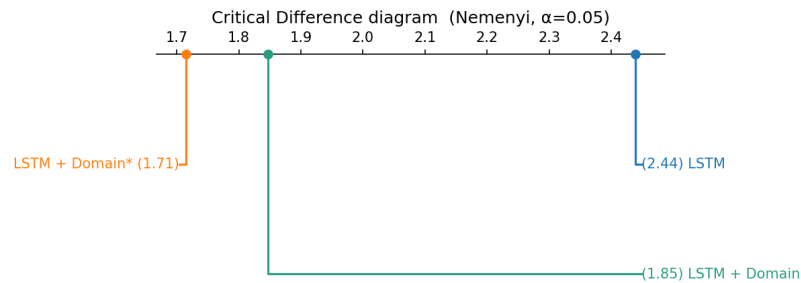


Figure 6.7: Critical Difference diagram comparing the Long Short-Term Memory (**LSTM**) variants with and without Domain specific training. **LSTM +Domain*** refers to the **LSTM + Domain** model that was run using the hyperparameters of the **LSTM** without any optimisation.

Table 6.6: Test set error metrics comparing the LSTM variants with and without Domain specific training. LSTM + Domain* refers to the LSTM + Domain model that was run using the hyperparameters of the LSTM without any optimisation. Discussed performance improvements are shaded in.

Model	Metric	Overall	Harbour	River	Channel	Lock
LSTM	ADE	54.78	61.07	43.29	32.09	45.41
	FDE	121.18	135.73	94.14	70.23	93.17
	RMSE	111.36	121.45	91.59	55.85	68.59
LSTM + Domain	ADE	53.80	60.79	40.79	29.53	34.94
	FDE	119.02	135.29	88.16	64.00	74.15
	RMSE	109.95	121.01	86.77	51.07	55.27
LSTM + Domain*	ADE	53.80	60.70	41.12	29.22	31.43
	FDE	118.62	134.73	88.44	63.05	65.02
	RMSE	110.20	121.38	86.72	50.23	50.77

Further Analysis

In what follows, we discuss the mechanisms and implications behind these results, starting with a comparison of predictive capabilities across all evaluated model families, followed by an investigation of the impact of geographic domains and their encodings to model performance, and ending with a brief section on monthly trends.

Predictive Capabilities of Motion Models

The results provide a clear answer to the first research question: learned sequence models, particularly task-specific LSTMs, significantly outperform both classical kinematic models and stochastic filters in predicting short-term inland vessel trajectories. Following, we investigate each model family in more detail.

Kinematic Models. The performance of the kinematic models suggests that simple arc estimations are insufficient to outperform straight-line motion prediction. The CV model learned to only use the velocity of the most recent timestep (highlighted in purple in Table 6.1). This is an assumption that is also applied in most CV definitions, which was relaxed for these experiments by parametrising context length. An analysis of the error distributions for the CV and CTRV models reveals that the resulting residuals are heavy-tailed rather than normally distributed. This deviation is reflected by low coefficient of determination (R^2) values of approximately 0.6138 to 0.6428 when compared against a theoretical normal distribution. Further analysis of this error distribution can be found in Figure A.4 in Appendix A. This observation leads us to a likely explanation for the optimisation result: vessel displacement over our samples could be dominated by brief, discrete regime changes rather than smooth, Gaussian drift. Averaging the displacement would therefore mix in an increasingly stale velocity estimate rather than reduce noise, so the most recent single step remains the least biased predictor of the immediate future. With increasing model complexity, the best context size increases. The Hybrid Model learns to switch to CV at a relatively small τ_{rot} of $29.28^\circ/\text{min}$. This indicates that straight-line motion is more prevalent than motion with a single arc in our dataset. Furthermore, Figure 6.2 shows the error progression over time. While the ADE of the CV model does not grow perfectly linearly, it has a much more linear trend compared to the CTRV models, which diverge heavily at longer horizons. One cause for this difference is that the deviation of a circular arc generated by an unsuitable turn rate in CTRV grows superlinearly with the number of prediction steps. In contrast, a wrong velocity estimate in CV produces a constant

lateral offset that accumulates linearly. For an intuitive visual comparison of this dynamic, refer to the upper rows of Figures A.6 and A.8 in Appendix A.

Stochastic Filters. In the Kalman Filter (KF), position changes are a linear function of the velocity state, so positional errors remain small as long as the velocity estimate is reasonable. Consequently, the optimisation selected a relatively small process noise q_{pos} compared to the measurement noise r_{pos} (highlighted in cyan in Table 6.1). The different orders of magnitude for r_{pos} in different domains, and the respective value differences between KF and EKF allow us to conclude that fixing the measurement noise to values commonly found in literature would not have been sufficient. Despite the optimised tuning, the KF fails to surpass the simple CV model. This points to two primary conclusions regarding its limitations. First, the bottleneck of the prediction lies in the underlying constant-velocity assumption rather than the state estimation process itself. Second, the KF is unsuitable for this task due to its strict reliance on Gaussian noise assumptions. In the previous section we have established that the error does not follow a Gaussian distribution. A further methodological caveat is that the diagonal $\mathbf{Q}$ assumes uncorrelated positional and velocity noise, despite their physical coupling. The extent to which this simplification also contributes to the observed gap remains open. Since the CV model was optimised to only consider the last step, the Markov assumption of the KF is not the cause for the inability of the KF to outperform the CV model. With the noise ratio established above and the ten correction steps available within the context window, the filter’s prior is effectively forgotten by the time prediction begins, so its velocity estimate converges to the same estimate as the CV model’s last-step displacement, rather than to a genuinely smoothed estimate.

In the EKF, position depends nonlinearly on the turn rate ω_k through trigonometric terms. This means that even a small error in the estimated rotation propagates into a drastically larger positional error over time. The optimisation process adapted to this instability by selecting a substantially larger process noise q_{pos} and a near-zero q_{rot} (highlighted in green in Table 6.1), which indicates that the filter learned position predictions are highly unreliable when rotation is involved. ω_0 is initialised to 0, and the minimum context length to $n_{\text{ctx}} = 2$. Together, these parameters effectively restrict the underlying CTRV model to straight-line motion with negligible curvature to avoid compounding nonlinear errors. Despite this, the EKF outperforms the KF significantly (Figure 6.1). However, this performance gap must be interpreted with caution. Although both filters belong to the same family, their predefined search spaces differed: the context window length (n_{ctx}) was treated as an optimisable hyperparameter for the EKF (converging to 2), whereas it was kept fixed at the full ten steps for the standard KF. Given that the baseline CV model also performed best using only the single most recent step, forcing the KF to smooth over the entire 10-step context window likely disadvantaged it. Consequently, the observed gap might partially stem from this discrepancy in the search space design rather than purely from the ability of the EKF to handle nonlinear kinematics. Future evaluations should align the search spaces to allow for a consistent comparison. Nevertheless, looking at the error distributions, the EKF still manages to suppress extreme outliers slightly better than the standard KF, which is reflected in the more pronounced gap in the RMSE metric. Since RMSE heavily penalises large deviations, this difference demonstrates that the state estimation of the EKF still manages to suppress extreme outliers slightly better than the standard KF. However, the violin plots in Figure 6.3 confirm that while the EKF error distribution is shifted towards slightly lower values compared to CV and TiRex, it still has a similar spread and long upper tails. Therefore, while the EKF provides a measurable baseline improvement, its fundamental handling of sudden manoeuvres and outliers is not drastically improved, consistent with the still prevailing non-Gaussian error distribution.

Foundation Models. TiRex runs best with a context length of 1. The model does not implement any attention mechanisms to selectively weight individual timestamps and has no mechanism to filter out the added noise from a longer context. It is thus prone to the same bias as the CV model, leading to the present configuration. With only a single input point and no learned domain adaptation, TiRex has little basis to do more than propagate it forward, which is consistent with the statistical indifference between TiRex and CV across all point-forecast metrics. Not only the perfor-

mance, but also the generated trajectories are similar to those of the **CV** models (Figures A.6, A.7, A.8, and A.9 in Appendix A), indicating the foundation model predicts straight-line kinematics. A distributional comparison of per-sample **RMSE** (Figure 6.3) reveals that **CV** and TiRex have nearly identical violin shapes with similar medians, interquartile ranges, and tail behaviour. They confirm that TiRex effectively learns **CV** kinematics without any domain-specific training.

If we consider Table 6.3, we notice that TiRex produces extremely narrow prediction intervals compared to Chronos-2. Narrow intervals can indicate either high accuracy or poor calibration. Inspecting the coverage reveals the latter: only 5.9% of true future positions fall within the nominal 80% prediction intervals of TiRex, far below the ideal coverage of 0.80. Chronos-2, by contrast, achieves a near-ideal coverage of 0.769. The Winkler score of TiRex is lower than that of Chronos-2 because the score rewards narrow intervals when they happen to contain the true value, and penalises misses in proportion to interval width. Since the intervals of TiRex are so narrow, each miss only causes a small absolute penalty. It follows that if we want to use a foundation model and also have an accurate quantification of uncertainty, TiRex is unsuitable despite its better performance in ADE and FDE.

LSTM. The **LSTMs** show the strongest forecasting performance among all evaluated models by a large margin (Table 6.2 and Figure 6.2). Compared to the kinematics based models, they reduce the prediction error substantially. This outperformance grows heavily as the prediction horizon extends. This widening gap is expected: kinematic filters extrapolate purely from the last observed state without any memory of prior context, so their assumptions become increasingly inaccurate the further ahead they are extrapolated. The **LSTMs**, in contrast, have access to the full context window, which allows them to remain comparatively accurate even at longer horizons within the tested window.

A visual inspection of the generated trajectories (Figures A.6, A.7, A.8, and A.9 in Appendix A) explains another key advantage: the **LSTMs** particularly excel at estimating the correct velocity during acceleration and deceleration phases. This capability makes the biggest difference in the lock domain, where most samples include abrupt speed changes to either start or stop vessel movement. Overall, the network learns motion patterns that generalise far better than the handcrafted assumptions encoded in **CV**, **CTRV**, and the stochastic filters. The **RMSE** violin plots in Figure 6.3 support this, showing that the error distribution of the **LSTM** is shifted toward lower values than all other models. The network performs better on the vast majority of trajectories, while a smaller number of difficult cases still produce comparatively high errors. The **LSTM** therefore achieves not only better average accuracy but also a more favourable distribution of prediction errors. To achieve this performance, the training process relied on the Hyperparameter Optimisation (**HPO**) framework.

Beyond the pruning strategy, the hyperparameter importance analysis demonstrates that the input feature set is the most influential parameter for model success. In contrast, the analysis shows that the hidden size of the network has almost no impact on the final performance. This indicates that the bottleneck for this forecasting task lies primarily in the provided data representation rather than the overall capacity of the model. Details about hyperparameter importances can be found in Figure A.5 in Appendix A. Interestingly, the best configuration does not use the full feature set but excludes `rot` (orange highlights in Table 6.1). While one might expect that adding more input features should always improve performance, as we can confirm for all other values by looking at the gradient, this result indicates that **ROT** does not contribute useful information in this setting. This is consistent with the **EKF** results and suggests that the **ROT** feature we derived is either noisy or only weakly informative for the motion patterns considered here. Future work should revisit our choice to derive this feature manually and consider using the *heading* field instead. A deeper comparison between this plain **LSTM** and a domain-aware variant follows in Exp III.

Figure 6.1 provides an overall summary comparison of the evaluated model families. From this perspective, learned motion models are clearly best suited to capture the non-trivial vessel dynamics in the evaluated AIS data. At the same time, the results show that stochastic filters remain com-

petitive and can outperform foundation models when the underlying motion assumptions match the task reasonably well. In contrast, increasing kinematic complexity alone does not necessarily improve performance, as the CTRV variants consistently remain among the weakest approaches.

The Impact of the Geographic Domain

Domain Comparison. The evaluation across different waterway types demonstrates that domains are not equivalent in terms of difficulty. They all represent distinct motion patterns and uncertainty characteristics. While a detailed analysis of parameter variations is beyond our scope, extreme deviations such as the [EKF](#) learning to heavily distrust incoming positional measurements to compensate for noise serve as concrete evidence. The highly distinct hyperparameter configuration of the lock domain is consistent with the expectation that the data augmentation applied exclusively to this class alters the effective training distribution. Future work should investigate whether sampling and data augmentation should be applied more uniformly across all classes to reduce such asymmetries.

The low errors observed in the channel domain suggest a homogeneous data distribution. By contrast, the comparative difficulty of predicting the harbour domain likely reflects a more diverse set of manoeuvres, with a broader range of trajectory patterns. This is consistent with the open navigable space available in harbour areas compared to the strictly confined boundaries of the other classes (Figure 3.2a). Furthermore, the massive performance leap observed in the lock and channel domains when transitioning from classical models to learned architectures has clear physical causes. The classical [CTRV](#) and [CV](#) models fail to estimate combinations of consecutive corners, typical for navigating channels. This dynamic could be learned by the [LSTM](#). Similarly, the performance leap in the lock domain is physically driven by start-and-stop accelerations that the motion equations cannot represent.

Comparison to Mixed Training. As the results indicate, the benefit of per-domain training increases with model complexity. This varying benefit of per-domain training shows that high-capacity models are sensitive to domain heterogeneity. They either require explicit domain conditioning or separate, domain-specific models to fully realise their potential. The lack of statistical significance in the performance drop of the harbour domain is most likely due to the similarity of the mixed and harbour dataset distributions, with the harbour domain dominating the dataset (Table 3.5). Isolating the harbour data actively degrades the performance of the [EKF](#). This anomaly likely occurs because the pure harbour environment contains a higher density of outliers, which violates the Gaussian noise assumptions of the filter even more than the mixed dataset. The fact that the channel and lock domains benefit heavily from specialised models indicates that they represent the most distinct motion regimes within the overall dataset. Meanwhile, the strict separation of harbour and river data is less critical for model performance in this specific setup. When extending the model architecture to more waterways, this varying sensitivity suggests that new training domains should be defined by discovering naturally distinct motion regimes within the data. These findings support the hypothesis that domain-specific training benefits high-capacity models, addressing the second research question, albeit under the limitations discussed in chapter 7.

Advantages of Explicit Domain Encoding

Interestingly, the LSTM + Domain* model that reuses the hyperparameters of the baseline LSTM outperforms the LSTM + Domain model that was tuned in a separate Optuna study with statistical significance (Figure 6.7). During the hyperparameter search for the standard LSTM + Domain model, the optimisation algorithm failed to find a better configuration than the initial 16 trials before early stopping terminated the study. This indicates that the search likely stopped prematurely

and that the domain-aware architecture could benefit from a more exhaustive tuning procedure. Future work should revisit this experiment with increased patience or a larger hyperparameter optimisation budget to fully exploit the potential of the explicit domain encoding.

The consistent outperformance of the LSTM + Domain models against the standard LSTMs in Table 6.6 and Figure 6.6 indicates a clear and robust dominance of the domain-aware model over the baseline. These results provide strong evidence that explicitly encoding domain information into the model architecture consistently improves trajectory prediction performance in all considered domains. The domain-aware model has a lower median error and lower first and third quartiles, indicating consistently better typical performance. The interquartile ranges of both models are comparable, but the baseline LSTM shows more extreme outliers. This suggests that the embeddings help in avoiding occasional large errors. Eliminating these outliers is exactly the primary objective when deploying trajectory forecasting models in safety-critical environments. While the best-case RMSE of the baseline is slightly lower in this particular run, this effect was not consistent across the additional ten training runs and does not affect the overall dominance of the domain-aware model.

We have thus answered the third research question and confirmed the impact of explicit domain encoding. It should be highlighted that this experiment used a relatively naive domain encoding strategy across only a small number of distinct classes. Even more substantial performance improvements can be expected in future work by exploring more sophisticated encoding techniques or incorporating a broader variety of waterway environments.

Monthly Trends

The monthly trends in Figure 6.5 are consistent across all models, which suggests that they are not caused by the underlying motion models, but rather reflect properties of the dataset itself. Without data from multiple years, we cannot determine whether this constitutes a seasonal trend. Possible causes could be, but are not limited to: large vessels being easier to predict and being more represented in summer, or different traffic characteristics resulting in more evasive manoeuvres in winter. The simplest explanation is higher data availability in the summer months combined with hyperparameter optimisation and model training being fitted to those months. However, this does not explain why the same trend is visible for Chronos-2, which was neither trained nor tuned on this data. We therefore take this simply as a confirmation that using a full year of data was the right decision, and note that the evaluation timeframe has a significant impact on reported results. Future work should investigate this further by incorporating older data.

Chapter 7

Conclusions

In this thesis, we systematically evaluated motion models for inland vessel trajectory prediction across a large-scale dataset of over four million samples. The overarching goal was to benchmark kinematic, filter-based, and deep sequence models under a unified framework and to quantify the impact of incorporating geographic domain knowledge.

Our experiments demonstrate that the evaluated learned sequence models, specifically the Long Short-Term Memory (**LSTM**) networks, clearly outperform kinematic models and stochastic filters. The baseline **LSTM** reduced the overall test Root Mean Square Error (**RMSE**) to 111.36 m, achieving a roughly 28% error reduction compared to the standard Constant Velocity (**CV**) model (155.73 m). Furthermore, while the zero-shot foundation model TiRex reaches kinematic-level accuracy out of the box (RMSE of 155.71 m), it suffers from severe miscalibration, capturing only 5.9% of true future positions within its 80% confidence intervals. In contrast, the foundation model Chronos-2 achieves a near-ideal coverage of 0.769, highlighting the varying calibration qualities among pretrained models.

We also found that the benefit of per-domain training is dependent on model complexity. While simple kinematic models remain largely unaffected, high-capacity models like the **LSTM** suffer significant performance drops when trained on mixed-domain datasets rather than domain-specific subsets. This was particularly evident in domains with distinct kinematic characteristics, such as channels and locks; for instance, mixed training degraded the **LSTM** Final Displacement Error (**FDE**) in the lock domain by 26.54 m compared to the domain-specific model.

Finally, we demonstrated that explicitly providing domain information as a simple one-hot encoded input yields robust and statistically significant performance gains. While the overall dataset average improved only modestly, the domain-aware architecture (**LSTM** + Domain*) achieved large error reductions specifically in the channel and lock domains, lowering the **RMSE** by approximately 10% in channels (from 55.85 m to 50.23 m) and by over 26% in locks (from 68.59 m to 50.77 m). Encoding domain information directly into a single **LSTM** via one-hot features closely matches the accuracy of separately trained per-domain models, offering a practical alternative that avoids maintaining multiple domain-specific models in deployment.

Taken together, these findings suggest that future multi-target tracking systems for inland navigation that focus on prediction accuracy should prioritise deep sequence models for trajectory prediction. Furthermore, embedding basic environmental context into the model architecture is an effective strategy to improve prediction accuracy in specific waterway domains.

Limitations & Future Work

In this section, we discuss limitations of this work, together with ideas on how these limitations could be approached in future work, sorted by Dataset, Methods and specific model limitations.

Dataset

Data Availability and AIS Biases. The results in this thesis are based on Automatic Identification System (AIS) logs from three German measurement stations and a single calendar year. While the datasets cover a large number of trajectories, they still reflect the traffic patterns of a limited set of waterways and regulatory conditions. For the observation period of the year 2025, data availability varies slightly across the three measurement sites (e.g., Bremerhaven lacks data for June 2025). In AIS networks, data loss can occur for diverse reasons, such as hardware maintenance. However, since our deep learning models operate on short trajectory windows rather than continuous long-term sequences, these gaps do not impact the training quality, as the available days still yield a sufficiently large and representative number of independent samples. Extending the analysis to additional years and measurement sites would be necessary to assess whether the observed error distributions and monthly trends generalise well.

Furthermore, the data primarily reflects the movement patterns of large commercial vessels, which are mandated to carry Class A transponders under the International Convention for the Safety of Life at Sea (SOLAS) (International Maritime Organization, 2014). This includes cargo vessels exceeding 300GT on international voyages and all tankers. Consequently, smaller recreational craft and fishing vessels with Class B transponders are significantly underrepresented. Because Class B devices transmit at lower power and with less frequent update rates (as detailed in Table 3.4), the resulting models may be biased to the more predictable dynamics of large commercial ships.

Finally, for live deployment, requiring geographic domain labels means the tracking system must maintain a stable internet connection or store local offline maps.

Station Domain Overlap. Another limitation is that samples for each waterway-type domain are usually dominated by a single measurement station (Table 3.5). The correlation between domain and measurement station means that models could potentially learn station characteristics instead of domain characteristics. To test if this is an issue, it is possible to use our One-Hot-Encoded LSTM from Exp III with the measurement station as a feature instead of the domain. If it is an issue, targeted data selection and augmentation, as well as the addition of more measurement stations can mitigate it in future work.

Data Efficiency and Scaling. Future work should also investigate the relationship between dataset volume and model predictive performance to determine the point of diminishing returns. Given the high computational cost of training on the full dataset, identifying the threshold at which additional data yields no significant improvements in accuracy would improve the scalability and efficiency of the training pipeline.

Augmentation and Class Imbalance. While the geometric augmentation (Section 3.2.4) mitigates class imbalance for lock manoeuvres, it introduces physical simplifications. Because spatial coordinates are transformed while velocity and temporal spacing remain unchanged, the model observes identical acceleration profiles from multiple angles. This increases the risk of memorizing kinematic profiles rather than learning general behaviour. Furthermore, random rotation and mirroring assume spatial invariance and perfect lateral symmetry, neglecting direction-specific environmental disturbances like local currents, wind, and propeller walk.

Because only data within the lock domain were augmented, hyperparameters are distinct and hard to compare to the other domains. Future work should explore alternative strategies for handling rare domains, such as downsampling of other classes, and quantify how strongly these design choices affect the error distribution and the robustness of the models.

A related, though non-synthetic, form of repetition arises from the overlapping sliding window used throughout the entire dataset. Because consecutive windows share most of their underlying trajectory, each additional overlapping sample contributes less new information than a fully independent trajectory would. This autocorrelation also violates the independence assumption underlying the statistical significance tests used in this work, since the effective number of independent observations is lower than the raw sample count, which may make the reported significance levels overly optimistic. We nonetheless consider it preferable to keep these overlapping samples in the dataset, as they still provide additional, if diminishing, training data at negligible cost. Should the dataset volume be reduced as part of the data efficiency investigation proposed above, overlapping samples (e.g., by increasing the stride towards the window length) should be the first candidates for removal, before discarding entire, independent trajectories.

Filter Configuration. This work evaluated a fixed configuration of preprocessing filters, without assessing the impact of filtering on the compared models. Future work should conduct an ablation study to investigate different filter configurations, in order to quantify their influence on model performance and ranking. It remains unclear whether the learned models require all filters to perform well, or whether they could instead learn to handle noisy, near-stationary data directly.

Derived Kinematic Features. A limitation in preprocessing is that the Rate of Turn (ROT) was analytically derived from the Course Over Ground (COG), as it is rarely provided in raw AIS data. Because COG relies on consecutive Global Navigation Satellite System (GNSS) position fixes, it becomes highly unreliable at near-stationary speeds where localisation noise exceeds the actual distance travelled, resulting in artificial ROT spikes that do not reflect actual vessel movement. While the true heading channel of the vessel offers a more stable orientation metric at low speeds, we did not use it as an alternative feature for two primary reasons. First, as per International Telecommunication Union (2001), true heading is an optional AIS field and is frequently missing, particularly among smaller inland vessels equipped with Class B transponders. Second, heading only indicates the direction the bow of the vessel is pointing, whereas COG represents the actual path of motion over the earth. Because inland waterway vessels can be subjected to lateral drift from river currents and wind, COG was used for predictions, despite the limitations at near-zero velocities. Since both the Extended Kalman Filter (EKF) and the LSTM optimisation indicate that the derived rate-of-turn signal does not contribute much predictive value, this choice should be revisited in future works.

Integration of Geographic Constraints. As this work already accesses OpenStreetMap (OSM) data to define geographic domain boundaries, a natural extension is to utilise these polygons as explicit geometric constraints. Currently, the motion models predict trajectories based solely on learned dynamics, which can occasionally result in physically impossible paths that overlap with land. Integrating the OSM water masks directly into the prediction pipeline, possibly through a geometric projection layer that snaps predictions to the nearest water body, or by incorporating a penalty term in the loss function for land-collisions, could significantly improve the performance and safety of the tracking system.

Stratification. For the stratification prior to the dataset split, the primary domain, was encoded using label encoding, mapping each domain to a distinct integer. However, the domains are nominal categorical variables with no inherent ranking. Label encoding nevertheless imposes an artificial order on the domains, which distorts the distances used in clustering. Future work should

therefore consider the use of one-hot encoding instead, as it represents each domain as an independent binary dimension and avoids introducing any artificial order or magnitude relationship.

Methods

Comparability of Optimised Model Variants. For both the domain-specific vs. mixed-domain comparison and the domain-encoding ablation (LSTM vs. LSTM + Domain), each model variant was independently optimised via Hyperparameter Optimisation (HPO) within the same search space, rather than evaluated under one fixed hyperparameter configuration. This choice reflects how these models would be used in practice, where hyperparameters are always tuned for the given data and architecture, but it also means that measured performance differences cannot be attributed solely to the intended manipulation (domain-specific vs. mixed data, or presence vs. absence of the domain embeddings), since they are confounded with differences in the resulting hyperparameter configurations. As HPO offers no guarantee of converging to a global optimum, these comparisons should be read as benchmarks of practically achievable performance rather than controlled, isolated measurements of a single effect.

A further issue specific to the domain-specific vs. mixed-domain comparison is that the training splits contain different numbers of samples, as domain-specific models are trained on strictly smaller subsets of the data. This difference in sample count is itself a change alongside the intended change in data composition, so part of any observed effect may stem from the amount of available training data rather than domain specificity alone.

A separate consideration is the number of training epochs. The maximum number of training epochs was fixed across all models as a boundary condition, which, if reached, could have introduced an unrelated difference between models trained on splits of different sizes. In practice, early stopping halted all variants well before this limit was reached, ruling out training time as a factor.

Hyperparameter Optimisation Bias. We initialised the Optuna search for the LSTM with a manually selected subset of starting configurations (16 out of 26 seeding trials). This ensures that all feature sets are evaluated at least once, but it also introduces an implicit prior over the search space, because the Tree-structured Parzen Estimator (TPE) sampler conditions its density estimates on these seedings. Future work should therefore compare this seeded strategy against a larger number of fully random initial trials or split up the search into two stages, to separate feature selection from the remaining hyperparameters, and quantify the difference to our implementation.

Statistical Analysis. The statistical analysis in Chapter 5.2.3 is based on file-level aggregated metrics, with one file representing one domain on one day on one measurement station. This is used as a fallback to per-sample evaluation, since only the aggregated errors were available at evaluation time. As a result, all files contribute equally as blocks, even though they may contain different numbers of samples. Because the rank of a file does not reflect how many samples it contains, models with near-identical aggregated error metrics can still receive noticeably different average ranks and appear separated in the critical difference diagram, as has happened with the CV and Kalman Filter (KF) in Table 6.2 and Figure 6.1. Such artifacts are expected under this block definition and are not evidence of a computational error. Future iterations of this work should save the per-sample evaluation for more accurate metrics.

Metrics. Our evaluation focused exclusively on predictive accuracy and calibration; we did not measure inference time or training time for any of the compared models. This is a relevant omission in practice, since kinematic filters such as the KF and EKF are frequently preferred in deployed Multi-Target Tracking (MTT) systems precisely because of their fast, closed-form inference and low computational cost, making them well-suited for resource-constrained hardware, unlike larger learned models. Future work should therefore report inference and training time alongside

accuracy metrics, to enable a more complete assessment of the practical trade-offs between model families.

A further limitation concerns the choice of [RMSE](#) as the [HPO](#) objective. We chose [RMSE](#) because it penalises large deviations more strongly than absolute-error-based metrics. This same property likely amplifies the effect of the heavy-tailed, non-Gaussian residual distribution established in Chapter 6, since outliers receive disproportionately high weight in the loss. This plausibly strengthens the bias towards short context lengths. Additionally, [RMSE](#) aggregates error across both the longitudinal and lateral directions into a single scalar. Future work should also report or optimise for along-track and cross-track deviation separately, which would make it easier to distinguish outliers caused by incorrect speed estimation from those caused by incorrect heading or turn-rate estimation.

Domain Labels. In this work, we rely on a four-class domain labelling scheme (harbour, river, channel, lock) obtained from [OSM](#). It should be noted that most [OSM](#) data are labelled by hand by volunteer contributors. We could not verify the correctness of the annotations for this study beyond plausibility checks with satellite images, and we cannot guarantee that no noise was introduced due to mislabelled waterways. This limitation is inherent to a label-based approach. A natural next step would be to refine the domain representation, for example by introducing additional domain types, or by clustering waterway segments based on kinematic key characteristics to obtain a more fine-grained context. It would also be interesting to investigate whether representing the domain as a sequence over time, instead of a single static vector per sample, further improves performance for longer context windows.

Hyperparameter Optimisation Budget. Because of the pruning and early stopping mechanisms we used, hyperparameter search can terminate before the true optimum is explored. The observation that the domain-aware [LSTM](#) with reused hyperparameters outperforms the separately tuned variant suggests that at least one Optuna study stopped prematurely. Future iterations of this work should consider larger budgets or more conservative pruning schedules for the most promising architectures.

Models

Model Architectures. The [LSTM](#) implemented in this thesis uses a stacked encoder with a linear decoder that predicts the full horizon in a single step. This choice matches the short 5-minute prediction horizon, but it also restricts the flexibility of the architecture. Autoregressive decoders, where each predicted step is fed back into the model as additional context, are the standard for [LSTMs](#). In the current setup, both context and prediction length are fixed and identical across all models, so it remains unclear how sensitive the observed ranking between models is to these design choices. Future work should therefore investigate autoregressive decoders and variable context and prediction windows.

Additionally, a bidirectional encoder should be investigated, as it has improved model performance in other scenarios (Yang et al., 2022). Other gains could be found by introducing attention mechanisms that give higher weights to relevant timestamps of the context window. In addition to time-wise attention, covariate-informed forecasting with attention across feature channels is particularly relevant. The comparison between Chronos-2 and TiRex suggests that cross-channel attention can provide an advantage over univariate or channel-independent sequence models. At the moment, TiRex is limited to univariate inputs, and the [LSTM](#) uses only a simple concatenation of features without explicit channel weighting. After the completion of the model training and evaluation within this work, TiRex-2 was published (Podest et al., 2026). It tackles the core weaknesses of TiRex relative to the Chronos-2 model by generalising to a multivariate forecaster and covariate informed modelling, while maintaining the inductive bias of an [LSTM](#) for time series data. Incorporating TiRex-2 into the benchmark provides an opportunity to revisit this comparison.

Beyond recurrent architectures, Transformer models for time series have shown promising results in other forecasting domains, mainly due to their ability to model long-range dependencies. Training such models from scratch on the AIS data was out of scope for this thesis because of the associated runtime and tuning cost, but remains an interesting direction for future work.

Uncertainty Quantification for Learned Models. For applications in multi-target tracking and autonomous navigation, a well-calibrated estimate of predictive uncertainty is important. A natural extension would therefore be to equip the LSTM with probabilistic outputs. That would make it possible to evaluate the calibration of LSTM-based motion models in the same way as for the probabilistic baselines and to study how uncertainty behaviour differs between domains.

Multiple Model Frameworks. In this work, we evaluated isolated stochastic filters (KF, EKF) and a heuristic, threshold-based Hybrid CV/CTRV model. Practical MTT systems often rely on probabilistic frameworks to switch between filters. A natural extension to the current filters that future work should investigate is an Interacting Multiple Model (IMM) filter, which dynamically mixes multiple kinematic states (e.g., CV and CTRV) based on Markov transition probabilities. In the context of machine learning, an approach analogous to the IMM filter is the Mixture of Experts (MoE) architecture. Future work could explore MoE to dynamically weight multiple LSTM variants, or to construct hybrid systems that directly combine the predictions of classical filters with learned sequence models.

Acknowledgements

OpenStreetMap. I thank the OSM community and the operators of Overpass and mirror services for providing the infrastructure and data used in this work. The map data is copyrighted by OpenStreetMap contributors and available at <https://www.openstreetmap.org>.

Supervision. I want to sincerely thank my two daily supervisors for their detailed reviews, helpful meetings, and continuous support throughout my thesis. Their guidance helped keep the scope focused and made the collaboration between our institutes possible. I am especially grateful that they enabled me to approach this work from both a practical and a scientific perspective.

AI Usage. Generative AI usage for the thesis text was limited to rephrasing and formatting assistance. Additionally, due to an injury that restricted my ability to type, I used Google Gemini to draft initial slides for the kickoff presentation based strictly on my thesis text drafts at the time. These slides were edited and finalised manually.

For the codebase, generative AI (Perplexity with Claude Sonnet Thinking) was used as a reference tool. It was used to provide overviews of suitable Python libraries for specific tasks. Once a library was selected, AI assisted with specific syntax formatting (for example, utilising the `osmnx` library for spatial labels or handling `DataFrame` geometries). Due to my injury, I also used GitHub Copilot in combination with Windows voice assistant to assist with coding tasks, including structure and implementation of the evaluation framework and models. All code that was generated in this way was reviewed and validated by me, and I take full responsibility for its correctness. In addition, AI was used to write auxiliary scripts responsible for generating plots and debugging. These scripts are not necessary to run the main pipeline and should not be considered for grading. I have provided a version of the repository without these scripts in the branch `handedIn` at RWTH GitLab¹.

¹<https://git-ce.rwth-aachen.de/g-nav-mob-irt/projects/galileonautic2plus/multi-object-tracking/motion-models/-/tree/handedIn>

References

- Akiba, T., Sano, S., Yanase, T., Ohta, T., & Koyama, M. (2019). Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the Conference on Knowledge Discovery and Data Mining (SIGKDD)*.
- Ansari, A. F., Shchur, O., Küken, J., Auer, A., Han, B., Mercado, P., Rangapuram, S. S., Shen, H., Stella, L., Zhang, X., Goswami, M., Kapoor, S., Maddix, D. C., Guerron, P., Hu, T., Yin, J., Erickson, N., Desai, P. M., Wang, H., ... Bohlke-Schneider, M. (2025). Chronos-2: From univariate to universal forecasting [arXiv, v1]. <https://doi.org/10.48550/arXiv.2510.15821>
- Ansari, A. F., Stella, L., Turkmen, A. C., Zhang, X., Mercado, P., Shen, H., Shchur, O., Rangapuram, S. S., Arango, S. P., Kapoor, S., Zschiegner, J., Maddix, D. C., Wang, H., Mahoney, M. W., Torkkola, K., Wilson, A. G., Bohlke-Schneider, M., & Wang, B. (2024). Chronos: Learning the language of time series. *Transactions on Machine Learning Research*. <https://openreview.net/forum?id=gerNCVqqtR>
- Auer, A., Podest, P., Klotz, D., Böck, S., Klambauer, G., & Hochreiter, S. (2025). TiRex: Zero-shot forecasting across long and short horizons with enhanced in-context learning. In *proceedings of the Annual Conference on Neural Information Processing Systems (NeurIPS)*. <https://doi.org/10.48550/arXiv.2505.23719>
- Bahdanau, D., Cho, K., & Bengio, Y. (2015). Neural machine translation by jointly learning to align and translate. In *Proceedings of the International Conference on Learning Representations (ICLR)*. <https://doi.org/10.48550/arXiv.1409.0473>
- Baratchi, M., Wang, C., Limmer, S., van Rijn, J. N., Hoos, H., Bäck, T., & Olhofer, M. (2024). Automated machine learning: Past, present and future. *Artificial Intelligence Review*, 57(5), 122. <https://doi.org/10.1007/s10462-024-10726-1>
- Beck, M., Pöppel, K., Spanring, M., Auer, A., Prudnikova, O., Kopp, M., Klambauer, G., Brandstetter, J., & Hochreiter, S. (2024). xLSTM: Extended long short-term memory. In *Proceedings of the Annual Conference on Neural Information Processing Systems (NeurIPS)*, 37, 107547–107603. <https://doi.org/10.52202/079017-3417>
- Bedoya-Maya, F., Shobayo, P., Beckers, J., & van Hassel, E. (2024). The impact of critical water levels on container inland waterway transport. *Transportation Research Part D: Transport and Environment*, 131. <https://doi.org/10.1016/j.trd.2024.104190>
- Bengio, Y., Simard, P., & Frasconi, P. (1994). Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks*, 5(2), 157–166. <https://doi.org/10.1109/72.279181>

- Bergstra, J. S., Bardenet, R., Bengio, Y., & Kégl, B. (2011). Algorithms for hyper-parameter optimization. In *Proceedings of the Annual Conference on Neural Information Processing Systems (NeurIPS)*, 24.
- Bundesamt für Seeschifffahrt und Hydrographie (BSH). (2025a). AIS data for ship emission measurement on the measurement site Bremerhaven 2025 – series. <https://gdi.bsh.de/de/feed/AIS-data-for-ship-emission-measurement-on-the-measurementsite-Bremerhaven-2025-series.xml>
- Bundesamt für Seeschifffahrt und Hydrographie (BSH). (2025b). AIS data for ship emission measurement on the measurement site Kiel 2025 – series. <https://gdi.bsh.de/de/feed/AIS-data-for-ship-emission-measurement-on-the-measurementsite-Kiel-2025-series.xml>
- Bundesamt für Seeschifffahrt und Hydrographie (BSH). (2025c). AIS data for ship emission measurement on the measurement site Wedel 2025 – series. <https://gdi.bsh.de/de/feed/AIS-data-for-ship-emission-measurement-on-the-measurementsite-Wedel-2025-series.xml>
- Bundesstelle für Seeunfalluntersuchung. (2025, June). *Jahresbericht 2024* (tech. rep.). Bundesstelle für Seeunfalluntersuchung. https://www.bsu-bund.de/SharedDocs/pdf/DE/Jahresstatistik/Jahresbericht_2024.pdf
- Capobianco, S., Millefiori, L. M., Forti, N., Braca, P., & Willett, P. (2021). Deep learning methods for vessel trajectory prediction based on recurrent neural networks. *IEEE Transactions on Aerospace and Electronic Systems*, 57(6), 4329–4346. <https://doi.org/10.1109/taes.2021.3096873>
- de Vos, J., Hekkenberg, R. G., & Valdez Banda, O. A. (2021). The impact of autonomous ships on safety at sea – a statistical analysis. *Reliability Engineering & System Safety*, 210, 107558. <https://doi.org/10.1016/j.ress.2021.107558>
- Deligiannidis, L., Dimitoglou, G., & Arabnia, H. R. (Eds.). (2024). *Artificial intelligence: Machine learning, convolutional neural networks and large language models*. De Gruyter. <https://doi.org/10.1515/9783111344126>
- Deng, D., Karl, F., Hutter, F., Bischl, B., & Lindauer, M. (2023). Efficient automated deep learning for time series forecasting. In *Proceedings of the European Conference on Machine Learning and Principles and Practice of Knowledge Discovery in Databases (ECML PKDD)*, 664–680.
- DiBiase, D. (2015). Geog 160: Mapping our changing world. <https://courses.ems.psu.edu/geog160/node/1914>
- Donandt, K., Böttger, K., & Söffker, D. (2022). Short-term inland vessel trajectory prediction with encoder-decoder models. In *Proceedings of the IEEE International Conference on Intelligent Transportation Systems (ITSC)*, 974–979. <https://doi.org/10.1109/ITSC55140.2022.9922148>
- Eisenblätter, N., Rehbronn, T., Kurowski, M., Benz, D., Vallery, H., & Jeinsch, T. (2025). Automated maneuvering of networked vessels in confined waters. *Journal of Physics: Conference Series*, 3123(1), 012020. <https://doi.org/10.1088/1742-6596/3123/1/012020>
- Eurostat. (2024). Freight transport statistics – modal split [Data extracted March 2024, accessed March 2026]. https://ec.europa.eu/eurostat/statistics-explained/index.php/Freight_transport_statistics_-_modal_split
- Feurer, M., Klein, A., Eggenberger, K., Springenberg, J., Blum, M., & Hutter, F. (2015). Efficient and robust automated machine learning. In *Proceedings of the Annual Conference on Neural Information Processing Systems (NeurIPS)*, 28.
- Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780. <https://doi.org/10.1162/neco.1997.9.8.1735>

- Hopfield, J. J. (1982). Neural networks and physical systems with emergent collective computational abilities. *Proceedings of the National Academy of Sciences*, 79(8), 2554–2558. <https://doi.org/10.1073/pnas.79.8.2554>
- Hutter, F., Kotthoff, L., & Vanschoren, J. (Eds.). (2019). *Automated machine learning: Methods, systems, challenges*. Springer. <https://doi.org/10.1007/978-3-030-05318-5>
- International Maritime Organization. (2014). International convention for the safety of life at sea (SOLAS), 1974 (2014 Consolidated Edition). <https://doi.org/10.25678/BC2M-YC67>
- International Telecommunication Union. (2001). *Technical characteristics for an automatic identification system using time division multiple access in the VHF maritime mobile frequency band* (tech. rep. No. ITU-R M.1371-1) (Incorporated by reference in 47 CFR). International Telecommunication Union Radiocommunication Sector. <https://law.resource.org/pub/us/cfr/ibr/004/itu-r.M-1371-1.2001.pdf>
- Jones, D. R., Schonlau, M., & Welch, W. J. (1998). Efficient global optimization of expensive black-box functions. *Journal of Global Optimization*, 13(4), 455–492.
- Jwo, D.-J., & Cho, T.-S. (2010). Critical remarks on the linearised and extended Kalman Filters with geodetic navigation examples. *Measurement*, 43(9), 1077–1089. <https://doi.org/10.1016/j.measurement.2010.05.008>
- Kalman, R. E. (1960). A new approach to linear filtering and prediction problems. *Transactions of the ASME – Journal of Basic Engineering*, 82(1), 35–45. <https://doi.org/10.1115/1.3662552>
- Legel, T., Söffker, D., Schätzle, R., & Donandt, K. (2025). Towards explainable deep learning for ship trajectory prediction in inland waterways. In *Proceedings of the European Safety and Reliability Conference (ESREL) and the Society for Risk Analysis Europe Conference (SRA-E)*, 1111–1117. https://doi.org/10.3850/978-981-94-3281-3_esrel-sra-e2025-p1370-cd
- Li, X. R., & Jilkov, V. P. (2003). Survey of maneuvering target tracking. part i: Dynamic models. *IEEE Transactions on Aerospace and Electronic Systems*, 39(4), 1333–1364. <https://doi.org/10.1109/TAES.2003.1261132>
- Liang, Y., Wen, H., Nie, Y., Jiang, Y., Jin, M., Song, D., Pan, S., & Wen, Q. (2024). Foundation models for time series analysis: A tutorial and survey. In *Proceedings of the Conference on Knowledge Discovery and Data Mining (SIGKDD)*, 6555–6565. <https://doi.org/10.1145/3637528.3671451>
- NOAA Marine Cadastre. (2025, September). Automatic identification system (AIS) data dictionary [Broadcast Points Data Dictionaries (2018–2024 and 2025–present)]. <https://coast.noaa.gov/data/marinecadastre/ais/data-dictionary.pdf>
- OpenStreetMap contributors. (2017). Planet dump retrieved from <https://planet.osm.org>. <https://www.openstreetmap.org>
- Podest, P., Pichler, M., Bürger, E., Zólyomi, L., Voggenberger, B., Berghammer, W., Klotz, D., Böck, S., Klambauer, G., & Hochreiter, S. (2026). Tirez-2: Generalizing tirex to multivariate data and streaming [arXiv, v1]. <https://doi.org/10.48550/arXiv.2607.01204>
- Schmidt, R. M. (2019). Recurrent neural networks (RNNs): A gentle introduction and overview [arXiv, v1]. <https://doi.org/10.48550/arXiv.1912.05911>
- Schmidt, S. F. (1966). Application of state-space methods to navigation problems. In *Advances in control systems* (pp. 293–340, Vol. 3). Elsevier. <https://doi.org/10.1016/B978-1-4831-9716-6.50011-4>
- Sekhon, J., & Fleming, C. (2020). A spatially and temporally attentive joint trajectory prediction framework for modeling vessel intent. In *Proceedings of the Conference on Learning for Dy-*

- namics and Control (L4DC)*, 120, 318–327. <https://proceedings.mlr.press/v120/sekhon20a.html>
- Statistisches Bundesamt (Destatis). (2026). *Goods transport – transport, performance*. Retrieved August 20, 2026, from <https://www.destatis.de/EN/Themes/Economic-Sectors-Enterprises/Transport/Goods-Transport/Tables/goods-transport-lr.html>
- Sutskever, I., Vinyals, O., & Le, Q. V. (2014). Sequence to sequence learning with neural networks. *In Proceedings of the Annual Conference on Neural Information Processing Systems (NeurIPS)*, 27. https://proceedings.neurips.cc/paper_files/paper/2014/file/5a18e133cbf9f257297f410bb7eca942-Paper.pdf
- Thrun, S., Burgard, W., & Fox, D. (2005). *Probabilistic robotics*. MIT Press.
- Urrea, C., & Agramonte, R. (2021). Kalman filter: Historical overview and review of its use in robotics 60 years after its creation. *Journal of Sensors*, 2021(1), 9674015. <https://doi.org/10.1155/2021/9674015>
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *In Proceedings of the Annual Conference on Neural Information Processing Systems (NeurIPS)*, 6000–6010.
- Wang, Y., Wu, H., Dong, J., Liu, Y., Wang, C., Long, M., & Wang, J. (2026). Deep time series models: A comprehensive survey and benchmark. *IEEE Transactions on Pattern Analysis and Machine Intelligence*. <https://doi.org/10.1109/TPAMI.2026.3690845>
- Xiao, Z., Fu, X., Zhang, L., & Goh, R. S. M. (2020). Traffic pattern mining and forecasting technologies in maritime traffic service networks: A comprehensive survey. *IEEE Transactions on Intelligent Transportation Systems*, 21(5), 1796–1825. <https://doi.org/10.1109/TITS.2019.2908191>
- Yang, C.-H., Wu, C.-H., Shao, J.-C., Wang, Y.-C., & Hsieh, C.-M. (2022). AIS-based intelligent vessel trajectory prediction using Bi-LSTM. *IEEE Access*, 10, 24302–24315. <https://doi.org/10.1109/ACCESS.2022.3154812>
- You, L., Xiao, S., Peng, Q., Claramunt, C., Han, X., Guan, Z., & Zhang, J. (2020). ST-Seq2Seq: A spatio-temporal feature-optimized Seq2Seq model for short-term vessel trajectory prediction. *IEEE Access*, 8, 218565–218574. <https://doi.org/10.1109/ACCESS.2020.3041762>
- Zaman, U., Khan, J., Lee, E., Balobaid, A. S., Aburasain, R. Y., & Kim, K. (2024). Deep learning innovations in South Korean maritime navigation: Enhancing vessel trajectories prediction with AIS data. *PLOS ONE*, 19(10), e0310385. <https://doi.org/10.1371/journal.pone.0310385>
- Zeng, A., Chen, M., Zhang, L., & Xu, Q. (2023). Are transformers effective for time series forecasting? *In Proceedings of the AAAI Conference on Artificial Intelligence*. <https://doi.org/10.1609/aaai.v37i9.26317>
- Zhang, X., Fu, X., Xiao, Z., Xu, H., & Qin, Z. (2022). Vessel trajectory prediction in maritime transportation: Current approaches and beyond. *IEEE Transactions on Intelligent Transportation Systems*, 23(11), 19980–19998. <https://doi.org/10.1109/TITS.2022.3192574>
- Zhao, L., Shi, G., & Yang, J. (2018). Ship trajectories pre-processing based on AIS data. *Journal of Navigation*, 71(5), 1210–1230. <https://doi.org/10.1017/S0373463318000188>
- Zhou, Y., Dong, Z., & Bao, X. (2024). A ship trajectory prediction method based on an Optuna-BiLSTM model. *Applied Sciences*, 14(9), 3719. <https://doi.org/10.3390/app14093719>

Appendix A

Supplementary Figures

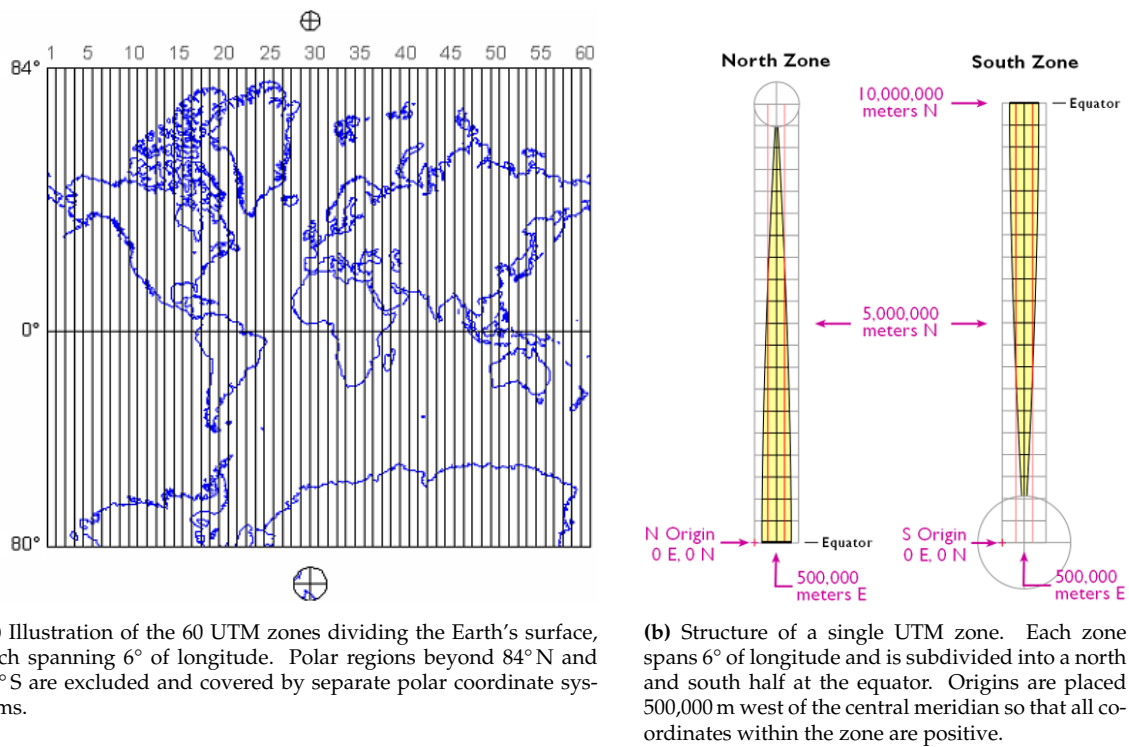


Figure A.1: The Universal Transverse Mercator coordinate system (UTM) projects geographic (spherical) coordinates onto a planar Cartesian grid. Figures from DiBiase (2015), licensed under CC BY-NC-SA 4.0.

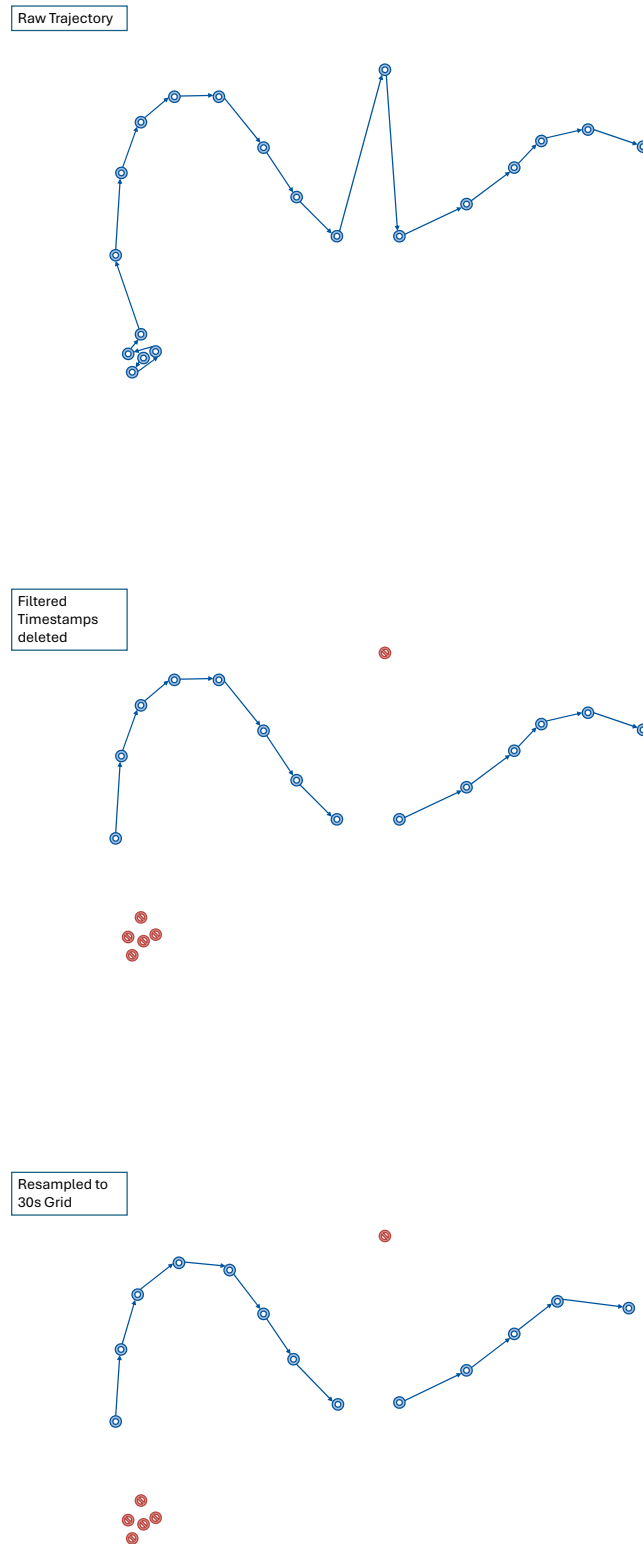


Figure A.2: Preprocessing visualisation. First, implausible messages are removed. The remaining trajectories are then resampled to a fixed 30 s grid. After resampling, a sliding window is applied to extract the final samples. (1/2)

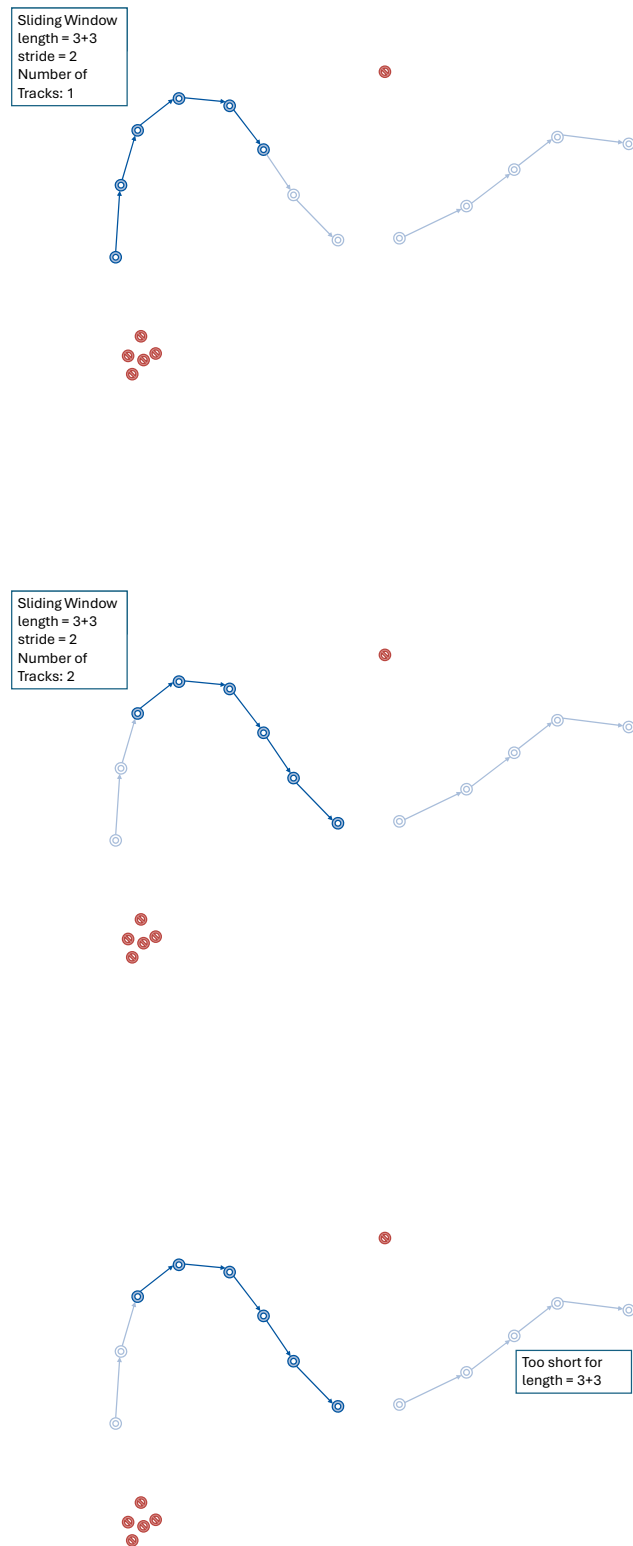


Figure A.2: Preprocessing visualisation. First, implausible messages are removed. The remaining trajectories are then resampled to a fixed 30 s grid. After resampling, a sliding window is applied to to extract the final samples. (2/2)

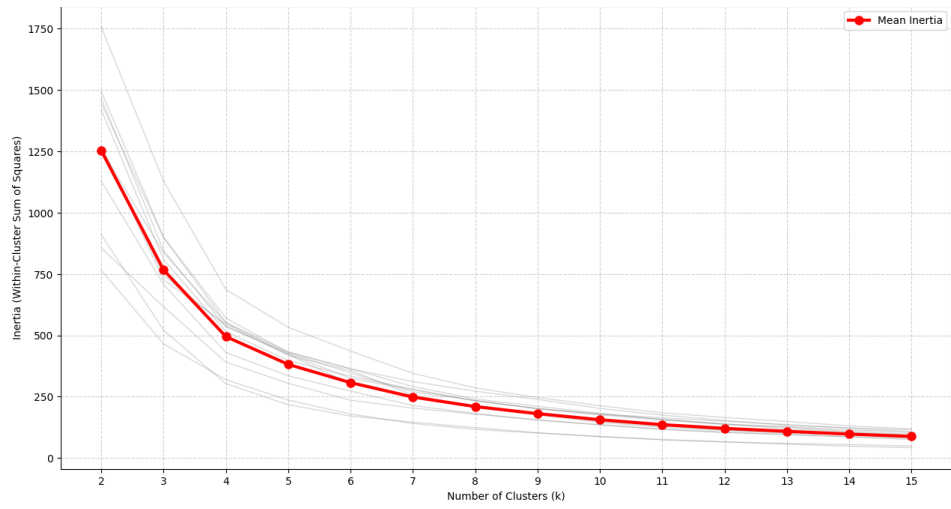


Figure A.3: Elbow-Plot: Inertia vs. Number of Clusters $k \in [2, 15]$ used to estimate the optimal number of strata. For each file in the sampled dataset, vessel profiles were aggregated by computing mean Speed Over Ground (SOG), maximum absolute Rate of Turn (ROT), and the context label. Features were standardised prior to clustering. KMeans ($n_{init} = 10$) was fitted independently per file; grey lines represent per-file inertia curves, the red line indicates the mean across all files. The elbow between $k = 4$ and $k = 5$ justifies the choice of five strata to capture the kinematic diversity of the dataset.

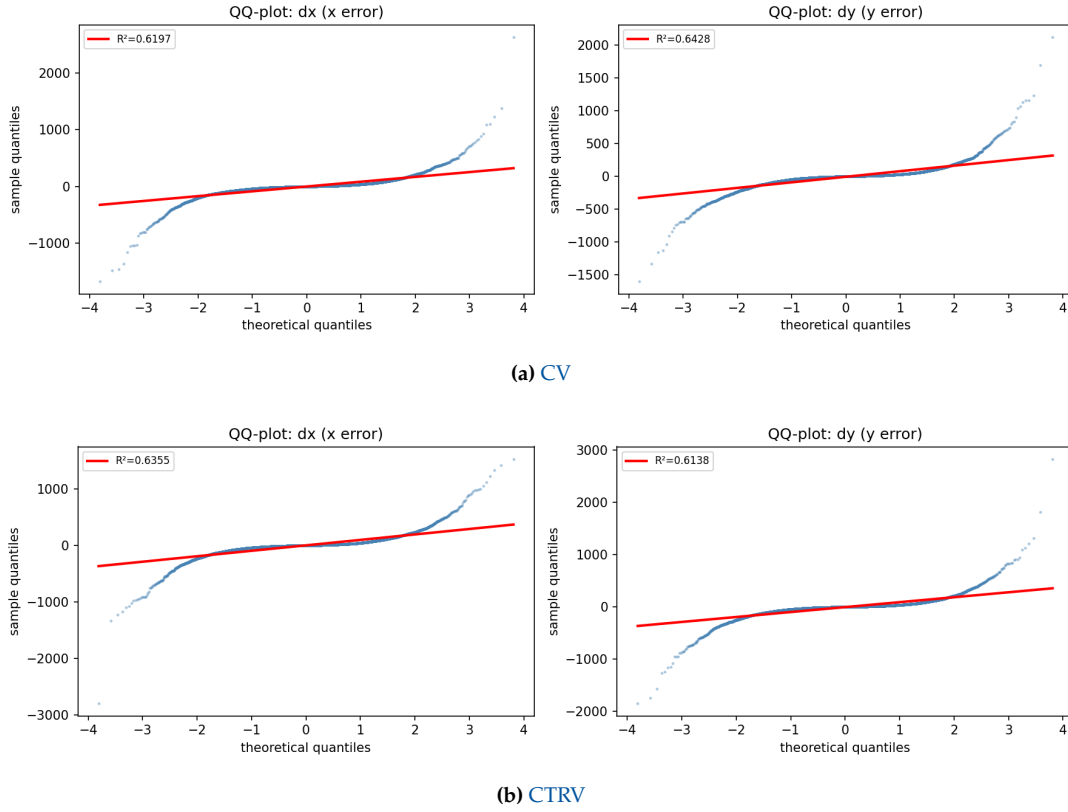


Figure A.4: Quantile-Quantile (QQ) plots of the positional prediction errors (dx and dy) for Constant Velocity (CV) and Constant Turn Rate and Velocity (CTRV) models. The errors of these pure kinematic models represent the real process noise of the **KFs**' underlying motion models, which they internally model as a Gaussian distribution. The solid red lines represent the theoretical normal distribution; the significant deviation of the sample quantiles at the extremes indicates heavy tails and extreme outliers. This is supported by low coefficient of determination (R^2) values ranging from 0.6138 to 0.6428, demonstrating a clear violation of the Gaussian noise assumption.

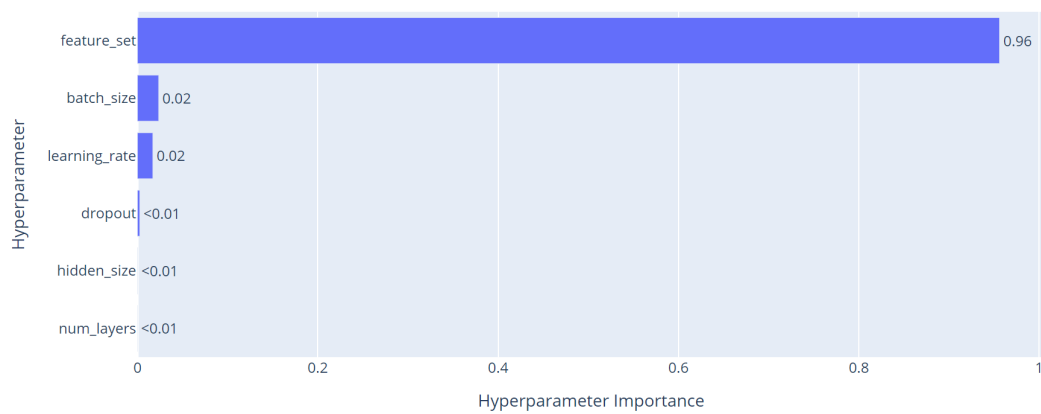


Figure A.5: Quantitative hyperparameter importances for the Long Short-Term Memory (LSTM), calculated by Optuna using the functional Analysis of Variance (fANOVA) framework. fANOVA isolates the marginal contribution of each parameter to the overall variance of the objective function, and thus provides quantitative metrics. The `feature_set` dominates convergence success of the model by a significantly large margin. This high score must be interpreted with caution, as it is amplified by the evaluation bias detailed in Section 7. Conversely, `hidden_size` registers an extremely low importance score; this matches with the observations from the parallel coordinate plot (Figure 6.4), which demonstrated that exploring a wide range of hidden sizes had little to no impact on the final performance. Notably, `num_layers` yields an importance of zero strictly because it was held constant and not defined as an active variable in this tuning space.

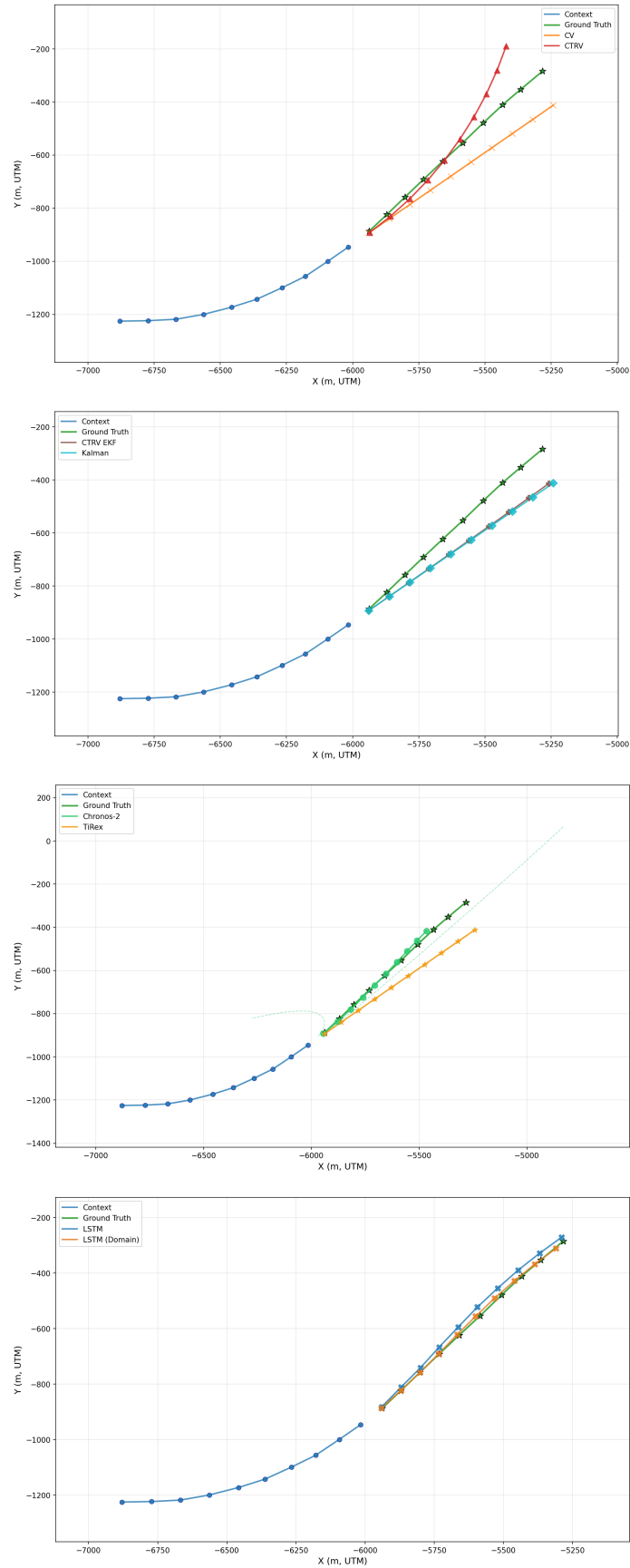


Figure A.6: Forecasting examples for the Channel domain (Kiel). These plots provide a qualitative visual comparison of the predicted paths generated by the evaluated models. The observed context window is displayed in blue, while the actual ground truth trajectory that follows is marked with green stars. The various model predictions are shown in the remaining colours. For Chronos 2, the 80% certainty band is displayed using thin green dots. Both axes represent relative **UTM** coordinates measured in meters.

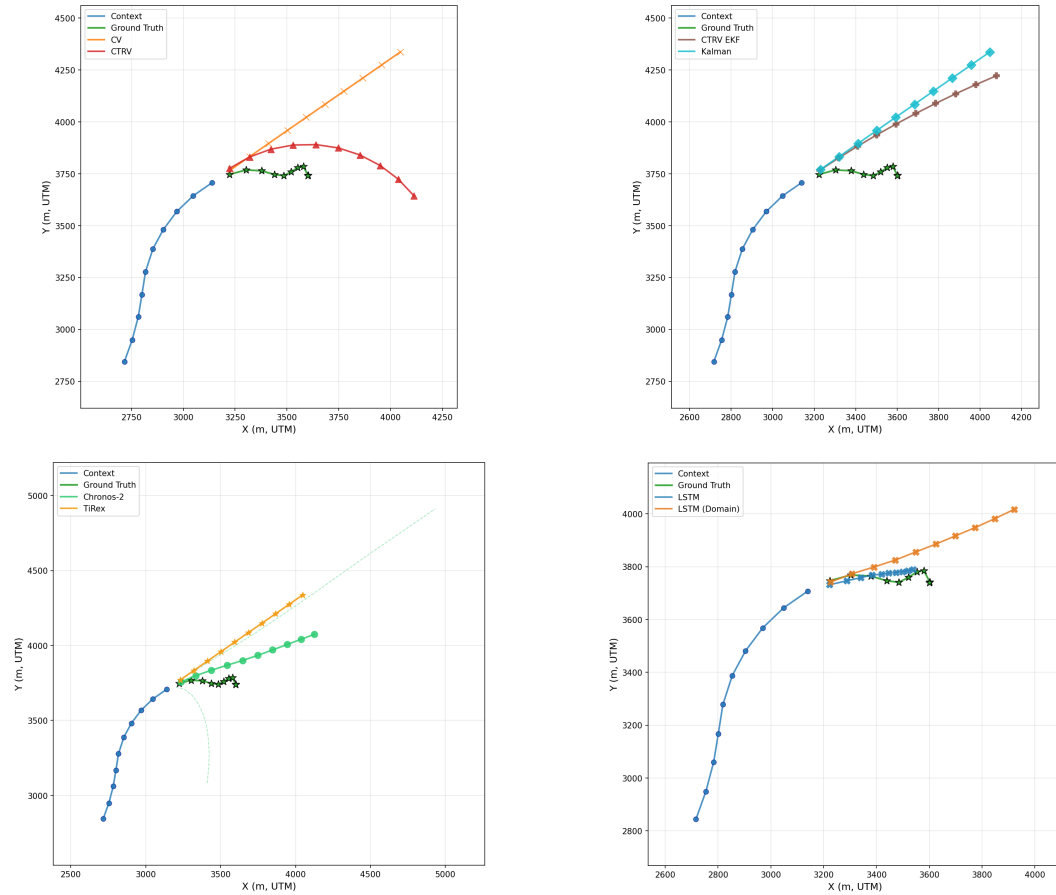


Figure A.7: Forecasting examples for the Harbour domain (Kiel). These plots provide a qualitative visual comparison of the predicted paths generated by the evaluated models. The observed context window is displayed in blue, while the actual ground truth trajectory that follows is marked with green stars. The various model predictions are shown in the remaining colours. For Chronos 2, the 80% certainty band is displayed using thin green dots. Both axes represent relative **UTM** coordinates measured in meters.

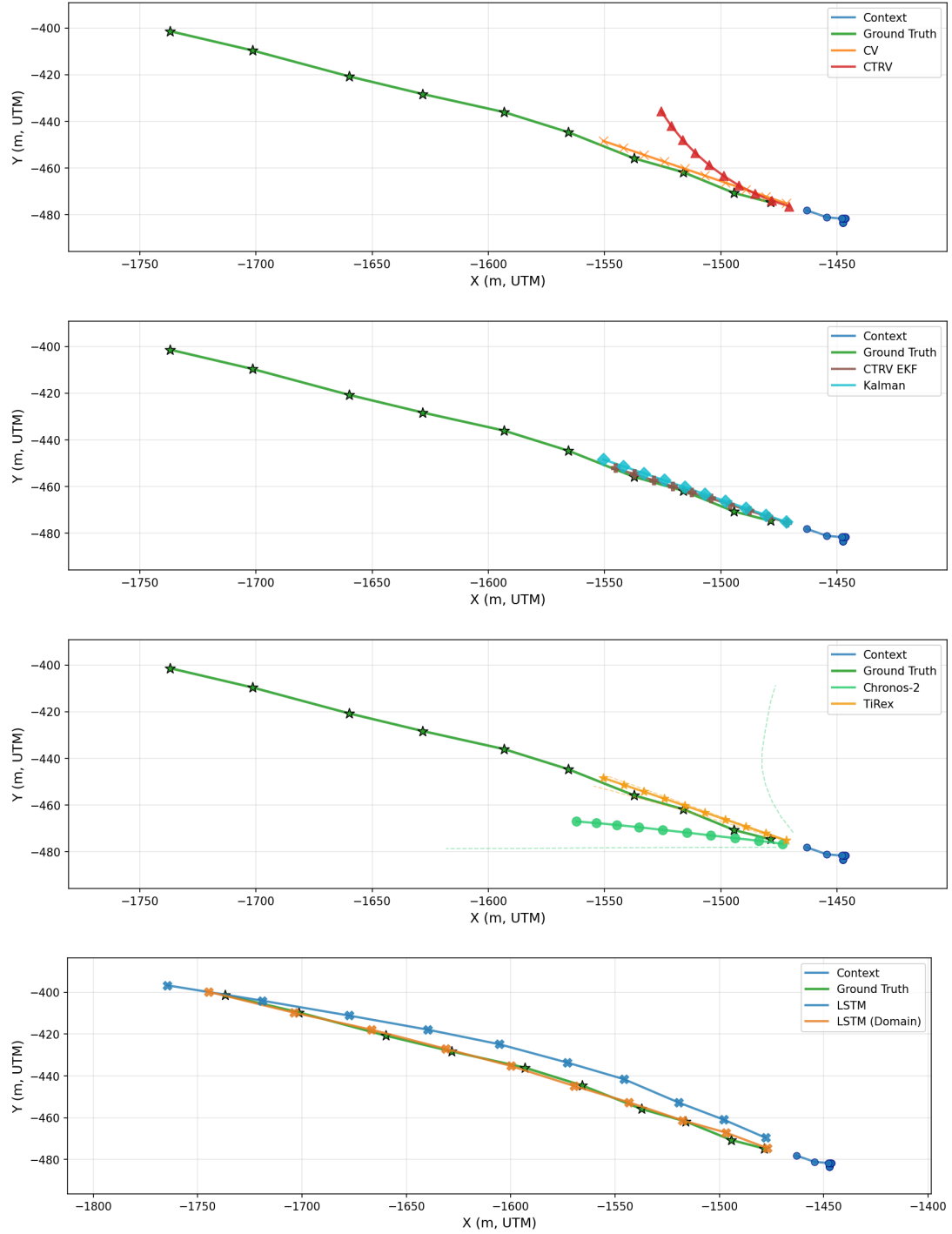


Figure A.8: Forecasting examples for the Lock domain (Kiel). These plots provide a qualitative visual comparison of the predicted paths generated by the evaluated models. The observed context window is displayed in blue, while the actual ground truth trajectory that follows is marked with green stars. The various model predictions are shown in the remaining colours. For Chronos 2, the 80% certainty band is displayed using thin green dots. Both axes represent relative UTM coordinates measured in meters.

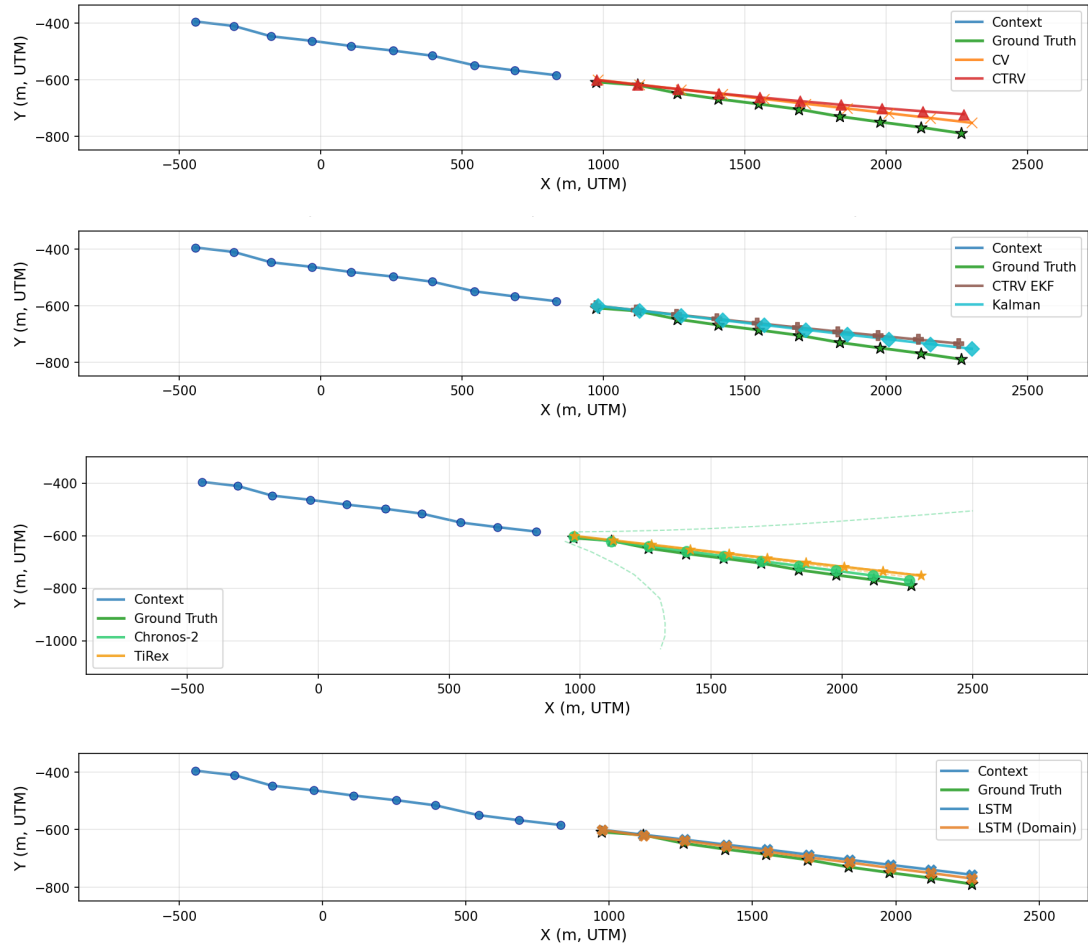


Figure A.9: Forecasting examples for the River domain (Wedel). These plots provide a qualitative visual comparison of the predicted paths generated by the evaluated models. The observed context window is displayed in blue, while the actual ground truth trajectory that follows is marked with green stars. The various model predictions are shown in the remaining colours. For Chronos 2, the 80% certainty band is displayed using thin green dots. Both axes represent relative **UTM** coordinates measured in meters.

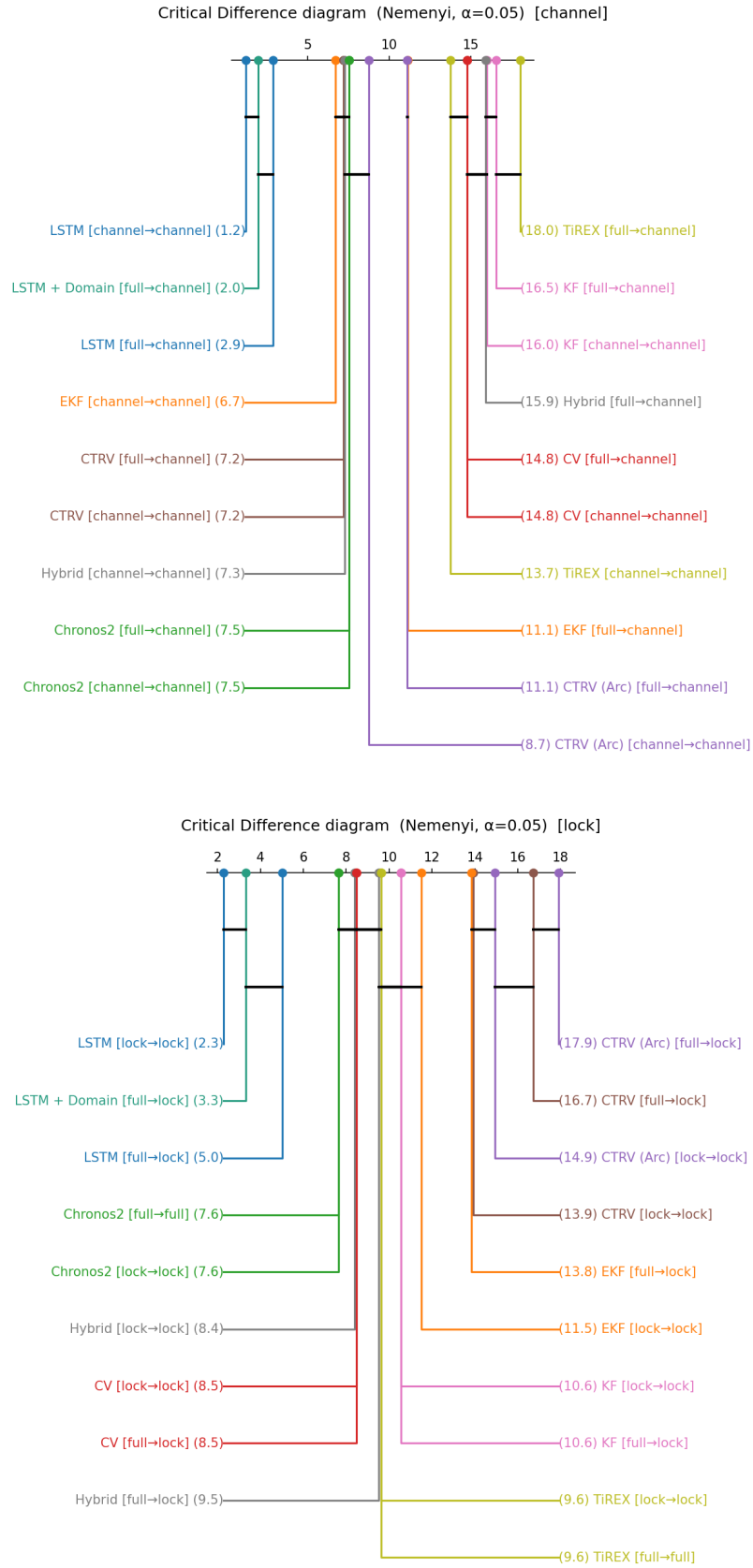


Figure A.10: Critical Difference (CD) diagram based on the Friedman test and Nemenyi post-hoc comparison for mixed and domain specific training, evaluated per domain. Models connected by a horizontal bar are not significantly different at $\alpha = 0.05$. The shown numbers are the mean rank per model in the evaluation, 1 being the best possible result. It can be observed that the harbour domain is the only environment where domain-specific training fails to produce a statistically significant improvement for the LSTM (1/2).

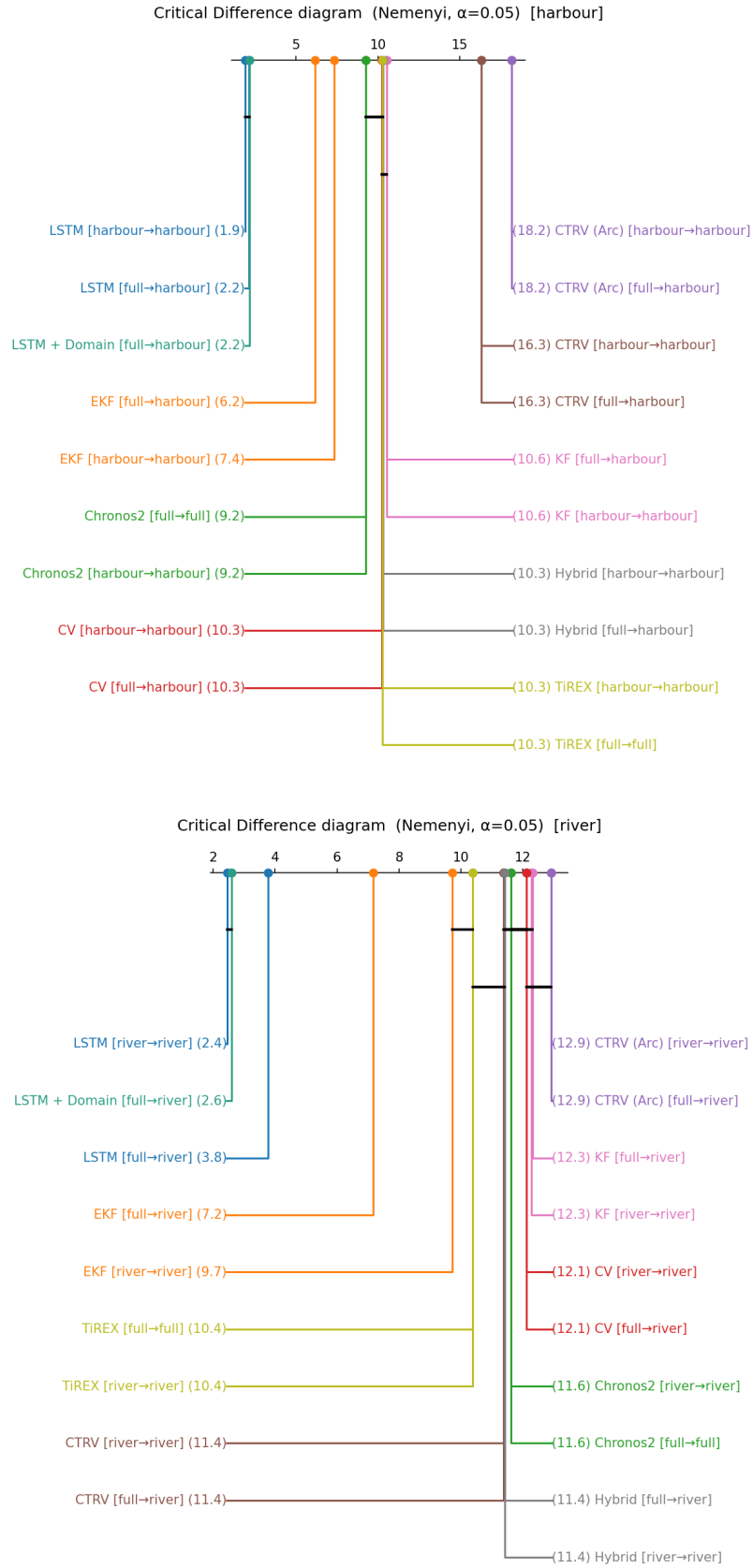


Figure A.10: Critical Difference (CD) diagram based on the Friedman test and Nemenyi post-hoc comparison for mixed and domain specific training, evaluated per domain. Models connected by a horizontal bar are not significantly different at $\alpha = 0.05$. The shown numbers are the mean rank per model in the evaluation, 1 being the best possible result. It can be observed that the harbour domain is the only environment where domain-specific training fails to produce a statistically significant improvement for the LSTM (2/2).

Appendix B

Supplementary Tables

Table B.1: Configuration parameters for the Optuna hyperparameter optimisation. The table details the specific search algorithms, trial counts, seeding strategies, and early stopping criteria applied to each model type, ranging from kinematic models to the LSTM.

Parameter	Value
<i>General</i>	
HPO Framework	Optuna
Device	CUDA (2 x RTX 4090)
TPE Multivariate	false
Early Stopping Patience	50 trials
Early Stopping Min. Delta	10^{-4}
Random Seed	42
<i>CV / CTRV / CTRV Arc</i>	
Method	Grid Search (1D)
Search Space	[1, max]
<i>Hybrid CV/CTRV</i>	
Method	TPE
Number of Trials	100
TPE Startup Trials	20
Seeded Initial Trial	CV/CTRV best <code>velocity_steps</code>
<i>Kalman Filter & EKF</i>	
Method	TPE
Number of Trials	100
TPE Startup Trials	20
<i>LSTM</i>	
Method	TPE + Successive Halving Pruner
Number of Trials	100
TPE Startup Trials	26 (16 feature sets + 10)
Seeded Initial Trials	16 (one per feature set)
Pruner Min. Resource	8 epochs
Pruner Reduction Factor	2
Pruner Min. Early Stopping Rate	0
Max. Epochs	100
Training Patience	8 epochs
Training Min. Delta	10^{-4}

Table B.2: Model-specific hyperparameters used in the experiments.

Model	Hyperparameters
CV	n_{vel}
CTRV	n_{vel}
CTRV Arc	n_{vel}
Hybrid CV/CTRV	$n_{\text{vel}}^{\text{CV}}, n_{\text{vel}}^{\text{CTRV}}, n_{\text{rot}}, \tau_{\text{rot}}$
KF (CV)	$q_{\text{pos}}, q_{\text{vel}}, r_{\text{pos}}, p_{0,\text{pos}}, p_{0,\text{vel}}$
EKF (CTRV)	$q_{\text{pos}}, q_{\text{vel}}, q_{\text{rot}}, r_{\text{pos}}, p_{0,\text{pos}}, p_{0,\text{vel}}, p_{0,\text{rot}}, n_{\text{ctx}}$
Chronos-2	—
TiREX	context length
LSTM	hidden size, number of layers, dropout, learning rate, batch size, feature set
LSTM + Domain	hidden size, number of layers, dropout, learning rate, batch size, feature set

Appendix C

Methodological Details

C.1 Mathematical Foundations of Sequence Models

In this appendix, we present more details on the mathematical formalisations underlying the sequence models discussed in the main text.

Recurrent Neural Network (RNN)

Building on the first popular model to use recurrence, the Hopfield Network introduced by Hopfield (1982), the foundational concept for sequence-based learning is the RNN. The primary distinction between RNNs and conventional Feedforward Neural Networks, such as Multi-Layer Perceptrons (MLPs), is the direction of information flow. While the latter are acyclic, RNNs use cycles to feed information back into the hidden state. With this architecture, the model incorporates context from previous inputs $X_{0:t-1}$ when processing the current input X_t , instead of processing each input in isolation (R. M. Schmidt, 2019). The basic structure of this recurrent process, configured as an Encoder-Decoder architecture, is illustrated in Figure C.1. Consequently, the network can make decisions based on the patterns it recognises (Deligiannidis et al., 2024). The recurrence can be expressed as Equation C.1,

$$H_t = \phi_h(X_t W_{xh} + H_{t-1} W_{hh} + b_h) \quad (\text{C.1})$$

where W represents weight matrices and ϕ an activation function. Despite their theoretical capability to handle sequences, RNNs suffer from the *vanishing gradient problem*, as well as the *exploding gradient problem*, as established by Bengio et al. (1994). As the sequence length increases, the gradients used to update weights during backpropagation decrease exponentially. This exponential decrease prevents the model from learning long-term dependencies.

Long Short-Term Memory (LSTM)

To address the limitations of standard RNNs to capture long-term dependencies, Hochreiter and Schmidhuber (1997) introduced the LSTM architecture. The corresponding architecture of the LSTM-based Encoder-Decoder model is depicted in Figure C.2. The core innovation of the LSTM is the introduction of a cell state C_t . Through this state, information persists across many time steps. The flow of information in and out of these cells is regulated by three gates:

- **Forget Gate** F_t : Determines which information from the previous cell state C_{t-1} is no longer needed and should be discarded.
- **Input Gate** I_t : Decides which new information from the current input X_t and previous hidden state H_{t-1} is relevant to be stored in the cell state.
- **Output Gate** O_t : Controls which parts of the current cell state C_t are used to compute the new hidden state H_t .

Following the formalisation by R. M. Schmidt (2019), the internal update process can be defined by Equation C.2:

$$\begin{aligned}
F_t &= \sigma(X_t W_{xf} + H_{t-1} W_{hf} + b_f), \\
I_t &= \sigma(X_t W_{xi} + H_{t-1} W_{hi} + b_i), \\
\tilde{C}_t &= \tanh(X_t W_{xc} + H_{t-1} W_{hc} + b_c), \\
C_t &= F_t \odot C_{t-1} + I_t \odot \tilde{C}_t, \\
O_t &= \sigma(X_t W_{xo} + H_{t-1} W_{ho} + b_o), \\
H_t &= O_t \odot \tanh(C_t).
\end{aligned} \tag{C.2}$$

In these equations, $\tilde{C}_t$ represents a candidate cell state, σ is the sigmoid activation function $\sigma(x) = \frac{1}{1+e^{-x}}$, and $\odot$ denotes the element-wise (Hadamard) product. W and b are the respective weights and biases. This update procedure for the cell state C_t prevents the gradient from vanishing as quickly as in standard RNNs.

As indicated by the multi-dimensional input vector X_t and the associated weight matrices, the LSTM architecture is mathematically equipped for multivariate sequence modelling. The model has the capacity to represent nonlinear cross-channel correlations, because the internal gating mechanisms integrate information from all input features simultaneously. This integration models their interdependencies within a shared hidden representation. Consequently, this design is highly suitable for the present use case.

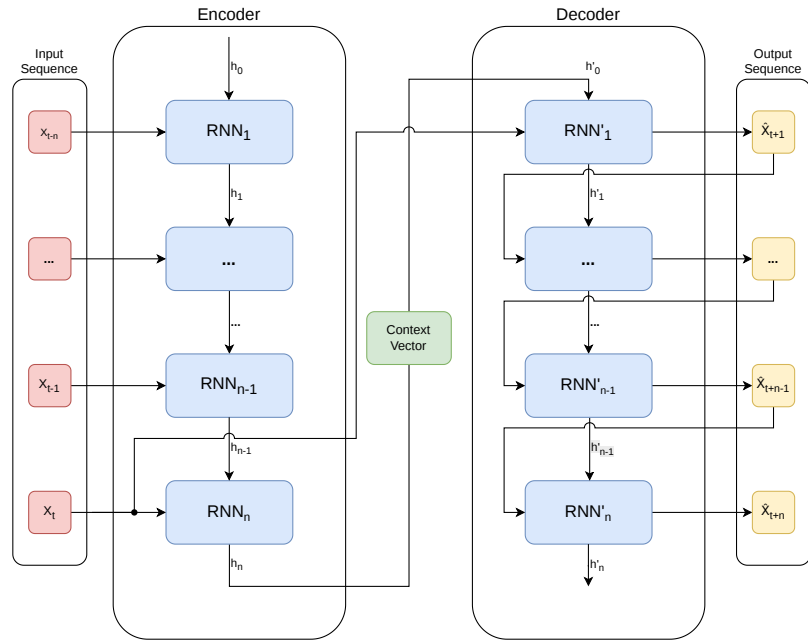


Figure C.1: Architecture of the standard Recurrent Neural Network ([RNN](#)) Encoder-Decoder model (1 Layer).

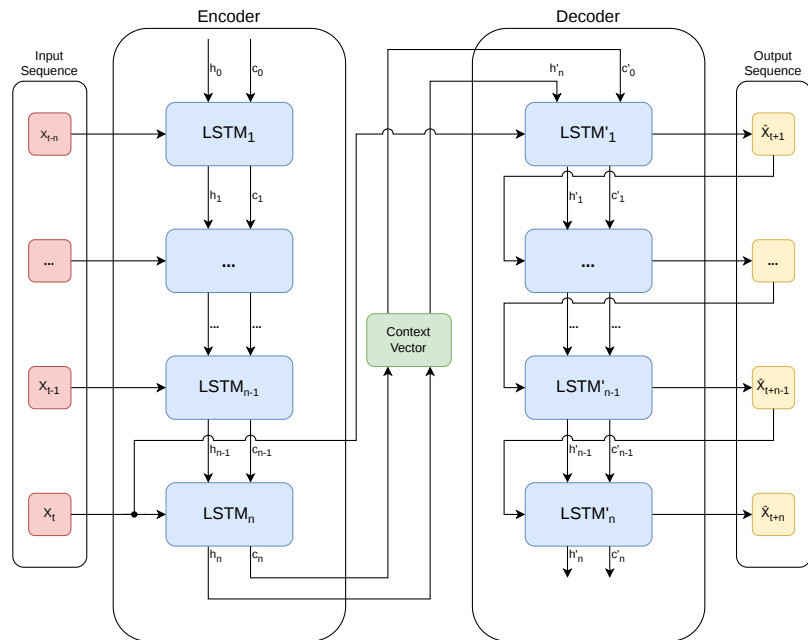


Figure C.2: Architecture of the Long Short-Term Memory ([LSTM](#)) Encoder-Decoder model (1 Layer).

C.2 Bayesian Optimisation and the Tree-Parzen-Estimator

In Bayesian Optimisation (BO), past evaluations are used to make informed decisions about which hyperparameters to test next. This approach is critical for models where evaluating the true objective function $\mathcal{L}$ is computationally expensive (e.g., when fully training an LSTM) (Hutter et al., 2019). In BO, this is achieved by constructing a probabilistic surrogate model $\hat{f}$, which approximates the true, expensive loss function based on all previously evaluated configurations. Because the surrogate model $\hat{f}$ is computationally cheap to evaluate, more potential configurations can be tested to identify the most promising candidates before resources are committed to a full model training.

To decide which configuration to evaluate next based on the given surrogate model, an acquisition function is utilised. A commonly applied acquisition function is Expected Improvement (EI) (Baratchi et al., 2024). In EI, a candidate is evaluated by combining two essential terms: the expected value of the function at a given point, and the associated variance. This combination provides a trade-off between two competing goals:

- **Exploitation:** By considering the expected value, the search is focused on promising regions of the search space that contain good candidate solutions with high probability.
- **Exploration:** In contrast, the variance term encourages the exploration of unknown areas that are weakly explored, or where candidate solutions of highly variable quality have been observed (Jones et al., 1998).

Figure C.3 illustrates one iteration of this process.

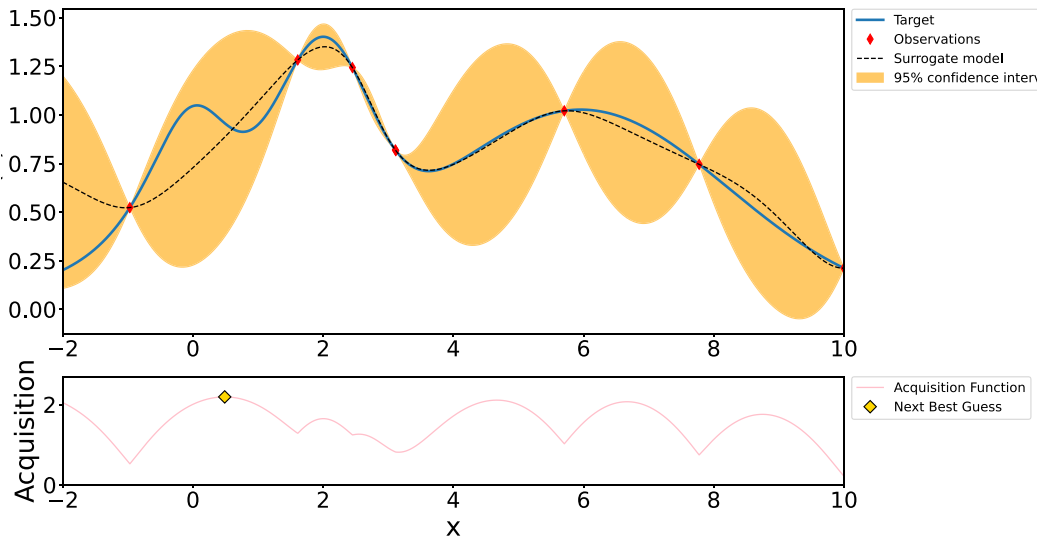


Figure C.3: Bayesian Optimisation (BO) using a surrogate model and Expected Improvement (EI) after several evaluations. The solid line represents the true objective $\mathcal{L}$, the dashed line and shaded region show the surrogate mean and 95% confidence interval, respectively. The acquisition function (bottom) determines the next candidate configuration to evaluate (Baratchi et al., 2024).

While classical BO surrogate models, such as Gaussian Processes, model $p(y \mid \lambda)$ directly, they scale poorly with the number of hyperparameters and struggle with discrete and categorical parameters Baratchi et al., 2024. To address these limitations, the modelling problem is inverted in the Tree-structured Parzen Estimator (TPE) algorithm (Bergstra et al., 2011). Rather than directly modelling $p(y \mid \lambda)$, two separate density functions, $p(\lambda \mid y < \alpha)$ and $p(\lambda \mid y \geq \alpha)$, are constructed in the TPE. With the threshold α , all observations are partitioned into a set of good and bad configurations, each of which is modelled using one-dimensional Parzen windows. Parzen windows

are kernel density estimators that place a smooth Gaussian function at each data point, with their sum forming the density function. The ratio $\frac{p(\lambda|y \geq \alpha)}{p(\lambda|y < \alpha)}$ is closely related to the [EI](#) acquisition function and serves as the criterion for proposing new hyperparameter configurations (Bergstra et al., 2011; Hutter et al., 2019). Figure C.4 illustrates this partitioning and selection process.

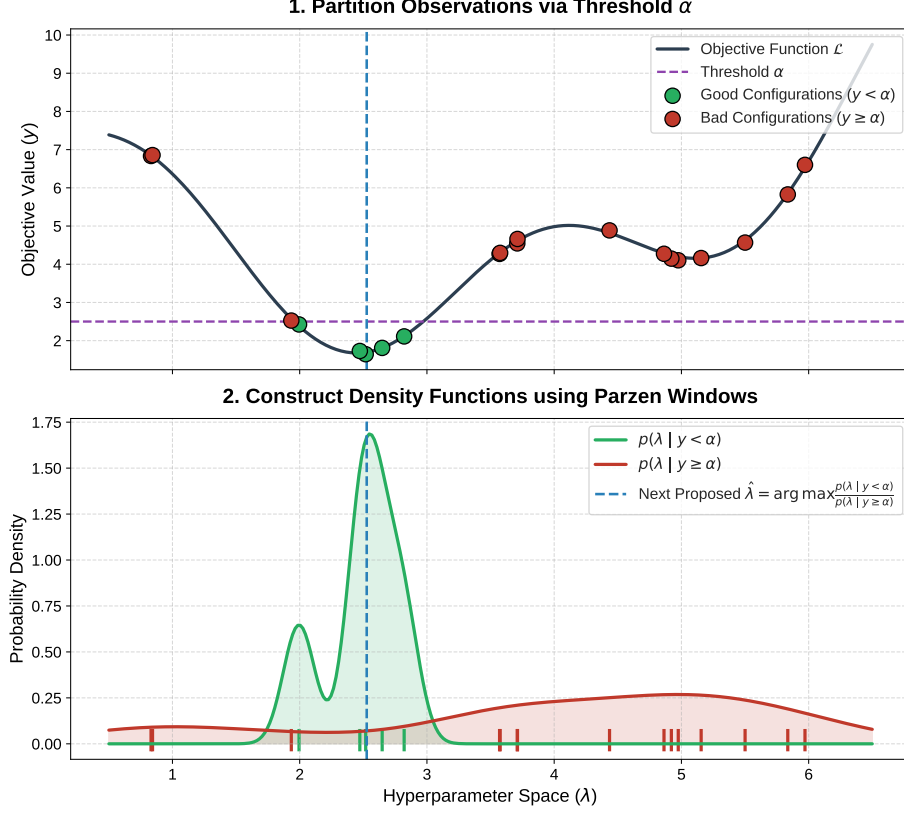


Figure C.4: Mathematical mechanism of the Tree-structured Parzen Estimator (TPE). The upper window illustrates the objective function $\mathcal{L}$, where evaluated configurations are partitioned by the threshold α . Green points represent promising configurations falling below this threshold, whereas red points indicate suboptimal configurations. The lower window displays the corresponding probability density functions constructed using Parzen windows: $p(\lambda | y < \alpha)$ for the good configurations and $p(\lambda | y \geq \alpha)$ for the bad configurations. The next hyperparameter configuration to be evaluated is proposed based on these distributions, indicated by the vertical blue dashed line across both plots.

C.3 Mathematical Details of the Search Space

This appendix provides the formal mathematical definitions for the Automated Machine Learning ([AutoML](#)) search space components introduced in Section 4.2.1. All definitions are adapted from Baratchi et al. (2024) to reflect the fixed training and validation split we used for this work.

Hyperparameter Optimisation. Let A_λ denote a learning algorithm A instantiated with hyperparameter configuration $\lambda \in \Lambda$, where Λ is the hyperparameter search space. Given a dataset $\mathcal{D}$ split into a training set $\mathcal{D}_{train}$ and a validation set $\mathcal{D}_{val}$, the [HPO](#) objective is defined as Equation C.3,

$$\lambda^* = \arg \min_{\lambda \in \Lambda} \mathcal{L}(A_\lambda, \mathcal{D}_{train}, \mathcal{D}_{val}) \quad (\text{C.3})$$

where $\mathcal{L}$ represents a validation loss metric. Since evaluating $\mathcal{L}$ requires fully training A_λ , each function evaluation is expensive, and HPO is a black-box optimisation problem.

Model Selection. The algorithm selection problem is defined as finding the algorithm $A^* \in \mathcal{A}$ that yields the lowest error on the validation set, as defined per Equation C.4.

$$A^* = \arg \min_{A \in \mathcal{A}} \mathcal{L}(A, \mathcal{D}_{train}, \mathcal{D}_{val}) \quad (\text{C.4})$$

CASH Problem. Given a set of learning algorithms $\mathcal{A}$, each algorithm $A^{(j)} \in \mathcal{A}$ is associated with a specific hyperparameter configuration space $\Lambda^{(j)}$. The Combined Algorithm Selection and Hyperparameter Optimisation (CASH) problem is formulated to jointly identify the optimal algorithm A^* and its corresponding optimal hyperparameter configuration λ^* , as defined per Equation C.5:

$$A^*, \lambda^* = \arg \min_{A^{(j)} \in \mathcal{A}, \lambda \in \Lambda^{(j)}} \mathcal{L}(A_\lambda^{(j)}, \mathcal{D}_{train}, \mathcal{D}_{val}) \quad (\text{C.5})$$

C.4 CTRV-EKF Jacobian Derivation

This appendix derives the full Jacobian $\mathbf{A}_k = \partial f / \partial \mathbf{x}|_{\hat{\mathbf{x}}_k^+}$ of the CTRV transition function introduced in Eq. (4.9). The state vector is $\hat{\mathbf{x}}_k = (x_k, y_k, \dot{x}_k, \dot{y}_k, \omega_k)^\top$ and $\Delta\psi = \omega_k \Delta t$.

Case $|\omega_k| \geq \varepsilon$

Position rows with respect to (x_k, y_k) . The new position $x_{k+1} = x_k + g(\dot{x}_k, \dot{y}_k, \omega_k)$ depends on x_k linearly with slope 1, and does not depend on y_k . The partial derivatives are defined by Equation C.6.

$$\frac{\partial x_{k+1}}{\partial x_k} = 1, \quad \frac{\partial x_{k+1}}{\partial y_k} = 0, \quad \frac{\partial y_{k+1}}{\partial x_k} = 0, \quad \frac{\partial y_{k+1}}{\partial y_k} = 1 \quad (\text{C.6})$$

Position rows with respect to $(\dot{x}_k, \dot{y}_k)$. The arc-length update is given by Equation C.7:

$$x_{k+1} = x_k + \frac{\dot{x}_k \sin \Delta\psi + \dot{y}_k (\cos \Delta\psi - 1)}{\omega_k}, \quad y_{k+1} = y_k + \frac{\dot{y}_k \sin \Delta\psi - \dot{x}_k (\cos \Delta\psi - 1)}{\omega_k} \quad (\text{C.7})$$

Because both terms appear linearly, differentiation with respect to $\dot{x}_k$ and $\dot{y}_k$ yields Equation C.8:

$$\frac{\partial x_{k+1}}{\partial \dot{x}_k} = \frac{\sin \Delta\psi}{\omega_k}, \quad \frac{\partial x_{k+1}}{\partial \dot{y}_k} = \frac{\cos \Delta\psi - 1}{\omega_k}, \quad \frac{\partial y_{k+1}}{\partial \dot{x}_k} = \frac{1 - \cos \Delta\psi}{\omega_k}, \quad \frac{\partial y_{k+1}}{\partial \dot{y}_k} = \frac{\sin \Delta\psi}{\omega_k} \quad (\text{C.8})$$

Position rows with respect to ω_k . By applying the quotient rule to $x_{k+1} = x_k + u(\omega_k)/\omega_k$, where $u = \dot{x}_k \sin \Delta\psi + \dot{y}_k (\cos \Delta\psi - 1)$ and $d\Delta\psi/d\omega_k = \Delta t$, the derivative is obtained as shown in Equation C.9.

$$\frac{\partial x_{k+1}}{\partial \omega_k} = \frac{\omega_k \cdot \frac{du}{d\omega_k} - u}{\omega_k^2} = \frac{\Delta t (\dot{x}_k \cos \Delta\psi - \dot{y}_k \sin \Delta\psi) \cdot \omega_k - (\dot{x}_k \sin \Delta\psi + \dot{y}_k (\cos \Delta\psi - 1))}{\omega_k^2} \quad (\text{C.9})$$

The numerator factor $\dot{x}_k \cos \Delta\psi - \dot{y}_k \sin \Delta\psi$ corresponds to the rotated velocity $\dot{x}_{k+1}$ from f , which results in the compact form defined by Equation C.10:

$$a_{15} = \frac{\Delta\psi \dot{x}_{k+1} - (\dot{x}_k \sin \Delta\psi + \dot{y}_k (\cos \Delta\psi - 1))}{\omega_k^2} \quad (\text{C.10})$$

Analogously, the derivation for a_{25} yields Equation C.11:

$$a_{25} = \frac{\Delta\psi \dot{y}_{k+1} - (\dot{y}_k \sin \Delta\psi - \dot{x}_k (\cos \Delta\psi - 1))}{\omega_k^2} \quad (\text{C.11})$$

Velocity rows with respect to $(\dot{x}_k, \dot{y}_k)$. The velocity update is a rotation by $\Delta\psi$, represented by Equation C.12.

$$\begin{pmatrix} \dot{x}_{k+1} \\ \dot{y}_{k+1} \end{pmatrix} = \mathbf{R}(\Delta\psi) \begin{pmatrix} \dot{x}_k \\ \dot{y}_k \end{pmatrix}, \quad \mathbf{R}(\Delta\psi) = \begin{pmatrix} \cos \Delta\psi & -\sin \Delta\psi \\ \sin \Delta\psi & \cos \Delta\psi \end{pmatrix} \quad (\text{C.12})$$

Since the update is linear in $(\dot{x}_k, \dot{y}_k)$, the Jacobian block is $\mathbf{R}(\Delta\psi)$ itself.

Velocity rows with respect to ω_k . Differentiating $\dot{x}_{k+1} = \dot{x}_k \cos \Delta\psi - \dot{y}_k \sin \Delta\psi$ with respect to ω_k , using $d\Delta\psi/d\omega_k = \Delta t$, results in Equation C.13.

$$\frac{\partial \dot{x}_{k+1}}{\partial \omega_k} = (-\dot{x}_k \sin \Delta\psi - \dot{y}_k \cos \Delta\psi) \Delta t, \quad \frac{\partial \dot{y}_{k+1}}{\partial \omega_k} = (\dot{x}_k \cos \Delta\psi - \dot{y}_k \sin \Delta\psi) \Delta t \quad (\text{C.13})$$

Turn-rate row. Since $\omega_{k+1} = \omega_k$, all partial derivatives are zero except $\partial \omega_{k+1} / \partial \omega_k = 1$.

Case $|\omega_k| < \varepsilon$: Taylor Approximation

To avoid division by zero, we substitute:

$$\sin \Delta\psi \approx \Delta\psi, \quad \cos \Delta\psi \approx 1. \quad (\text{C.14})$$

To avoid division by zero, we apply the Taylor approximations as defined by Equation C.15.

$$\sin \Delta\psi \approx \Delta\psi, \quad \cos \Delta\psi \approx 1. \quad (\text{C.15})$$

The position update simplifies to the constant-velocity step $x_{k+1} \approx x_k + \dot{x}_k \Delta t$, $y_{k+1} \approx y_k + \dot{y}_k \Delta t$, while the Jacobian is reduced to the form defined by Equation C.16.

$$\mathbf{A}_k|_{|\omega_k| < \varepsilon} = \begin{pmatrix} 1 & 0 & \Delta t & 0 & -\frac{1}{2}\dot{y}_k \Delta t^2 \\ 0 & 1 & 0 & \Delta t & \frac{1}{2}\dot{x}_k \Delta t^2 \\ 0 & 0 & 1 & 0 & -\dot{y}_k \Delta t \\ 0 & 0 & 0 & 1 & \dot{x}_k \Delta t \\ 0 & 0 & 0 & 0 & 1 \end{pmatrix} \quad (\text{C.16})$$

Appendix D

Codebase Overview

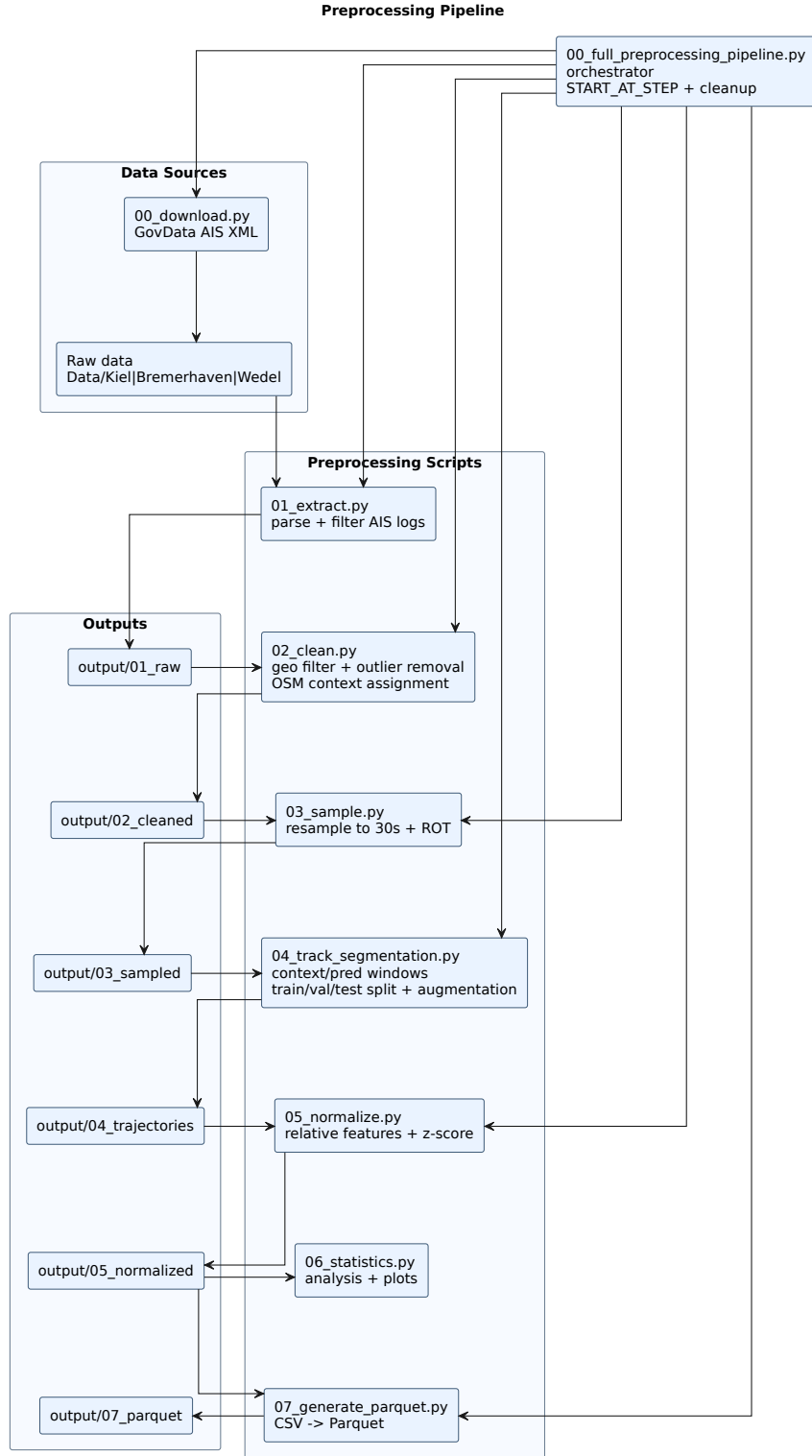


Figure D.1: Preprocessing pipeline file interactions. The pipeline is orchestrated by `00_full_preprocessing_pipeline.py`, which invokes steps 01–07 sequentially and can resume from any intermediate step via `START_AT_STEP`. Raw AIS data is fetched by `00_download.py` and stored under `Data/`. Each script reads from the previous output folder and writes into the next: `01_extract.py` → `output/01_raw` → `02_clean.py` → `output/02_cleaned` → `03_sample.py` → `output/03_sampled` → `04_track_segmentation.py` → `output/04_trajectories` → `05_normalize.py` → `output/05_normalized` → `07_generate_parquet.py` → `output/07_parquet`. `06_statistics.py` reads from `output/05_normalized` to generate analysis figures and produces no further output folder.

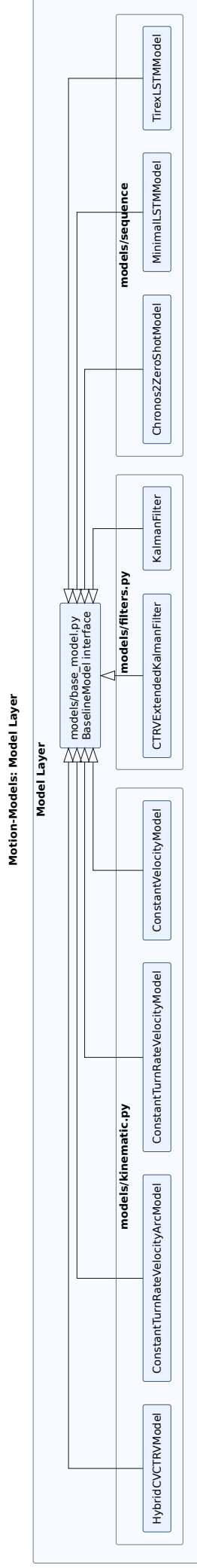


Figure D.2: Model layer class hierarchy. All models inherit from `BaselineModel (models/base_model.py)`, which defines the common interface used by the evaluation framework: `predict(context_df, n_pred_steps)`. The *kinematic* group (`models/kinematic.py`) contains: `ConstantVelocityModel (CV)`, `ConstantTurnRateVelocityModel (CTRV)`, `ConstantTurnRateVelocityArcModel (CTRV Arc)`, and `HybridCVCTRVMModel`, which selects between `CV` and `CTRV` per sample based on a rotation-rate threshold. The *filter* group (`models/filters.py`) contains `KalmanFilter` and `CTRVExtendedKalmanFilter`. The *sequence* group (`models/sequence/`) contains data-driven models: `TirexLSTMModel (TiRex)`, `MinimalLSTMModel (LSTM)`, and `Chronos2ZeroShotModel (Chronos-2)`.

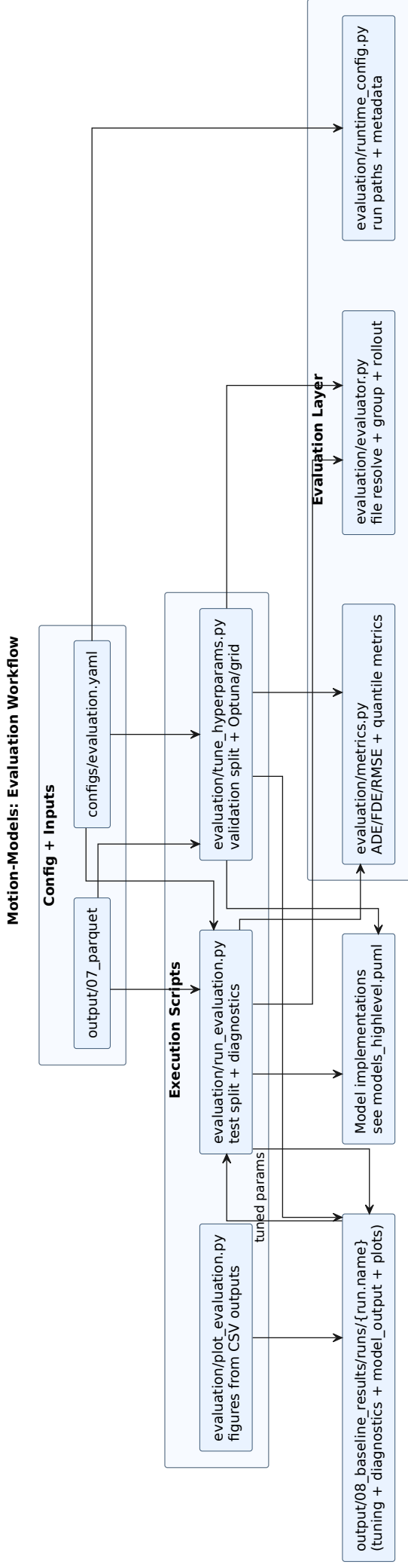


Figure D.3: Evaluation workflow file and module interactions. The evaluation framework takes two external inputs: the Parquet dataset produced by the pre-processing pipeline (`output/07_parquet`) and a YAML configuration file (`configs/evaluation.yaml`) that specifies enabled models, hyperparameter search spaces, and output paths. `evaluation/runtime_config.py` resolves run-specific paths and metadata from the configuration. **Tuning** (`tune_hyperparams.py`) operates exclusively on the validation split: single-parameter models (CV, CTRV, CTRV Arc, TiReX LSTM) are tuned via exhaustive grid search in parallel worker processes, while multi-parameter models (Hybrid, Kalman, CTRV EKF) are tuned with Optuna TPE. Tuning results are written as CSV files under `output/08_baseline_results/runs/{run.name}/tuning/`. **Final evaluation** (`run_evaluation.py`) reads the tuned parameters and evaluates all enabled models on the test split, writing per-file and per-month diagnostic tables and optional per-model prediction exports to the same run folder. Both scripts delegate to `evaluation/evaluator.py` for file resolution, track grouping, model rollout, and metric aggregation, and to `evaluation/metrics.py` for computing ADE, FDE, RMSE, and quantile metrics. `evaluation/plot_evaluation.py` reads the saved CSV outputs to generate all evaluation figures independently of the evaluation runs themselves.

Eidesstattliche Versicherung Declaration of Academic Integrity

Klein, Dustin

Name, Vorname/Last Name, First Name

445455

Matrikelnummer (freiwillige Angabe)
Student ID Number (optional)

Ich versichere hiermit an Eides Statt, dass ich die vorliegende ~~Arbeit~~/Bachelorarbeit/
~~Masterarbeit~~* mit dem Titel

I hereby declare under penalty of perjury that I have completed the present ~~paper~~/bachelor's thesis/~~master's thesis~~* entitled

Short-Term Trajectory Prediction for Inland Vessels using AIS Data

(Trajektorienvorhersage für Binnenschiffe mithilfe von AIS-Daten)

selbstständig und ohne unzulässige fremde Hilfe (insbes. akademisches Ghostwriting) erbracht habe. Ich habe keine anderen als die angegebenen Quellen und Hilfsmittel benutzt; dies umfasst insbesondere auch Software und Dienste zur Sprach-, Text- und Medienproduktion. Ich erkläre, dass für den Fall, dass die Arbeit in unterschiedlichen Formen eingereicht wird (z.B. elektronisch, gedruckt, geplottet, auf einem Datenträger) alle eingereichten Versionen vollständig übereinstimmen. Die Arbeit hat in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen.

independently and without unauthorized assistance from third parties (in particular academic ghostwriting). I have not used any other sources or aids than those indicated; this includes in particular software and services for language, text, and media production. In the event that the work is submitted in different formats (e.g. electronically, printed, plotted, on a data carrier), I declare that all the submitted versions are fully identical. I have not previously submitted this work, either in the same or a similar form to an examination body.

Milton Keynes, 28. 08. 2026

Ort, Datum/City, Date



Unterschrift/Signature

*Nichtzutreffendes bitte streichen/Please delete as appropriate

Belehrung:

Official Notification:

§ 156 StGB: Falsche Versicherung an Eides Statt

Wer vor einer zur Abnahme einer Versicherung an Eides Statt zuständigen Behörde eine solche Versicherung falsch abgibt oder unter Berufung auf eine solche Versicherung falsch aussagt, wird mit Freiheitsstrafe bis zu drei Jahren oder mit Geldstrafe bestraft.

§ 156 StGB (German Criminal Code): False Unsworn Declarations

Whosoever before a public authority competent to administer unsworn declarations (including Declarations of Academic Integrity) falsely submits such a declaration or falsely testifies while referring to such a declaration shall be liable to imprisonment for a term not exceeding three years or to a fine.

§ 161 StGB: Fahrlässiger Falscheid; fahrlässige falsche Versicherung an Eides Statt

(1) Wenn eine der in den §§ 154 bis 156 bezeichneten Handlungen aus Fahrlässigkeit begangen worden ist, so tritt Freiheitsstrafe bis zu einem Jahr oder Geldstrafe ein.

(2) Straflosigkeit tritt ein, wenn der Täter die falsche Angabe rechtzeitig berichtigt. Die Vorschriften des § 158 Abs. 2 und 3 gelten entsprechend.

§ 161 StGB (German Criminal Code): False Unsworn Declarations Due to Negligence

(1) If an individual commits one of the offenses listed in §§ 154 to 156 due to negligence, they are liable to imprisonment for a term not exceeding one year or to a fine.

(2) The offender shall be exempt from liability if they correct their false testimony in time. The provisions of § 158 (2) and (3) shall apply accordingly.

Die vorstehende Belehrung habe ich zur Kenntnis genommen:

I have read and understood the above official notification:

Milton Keynes, 28. 08. 2026

Ort, Datum/City, Date



Unterschrift/Signature